MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

ELEVANCESKILL INTERNSHIP

ON

INTELLIGENT EV BATTERY MANAGEMENT SYSTEM

A Real-Time Multi-Cell Battery Intelligence,

Safety, Fault-Tolerant and Cloud Telemetry System.

[BMS / EV ICON]

SUBMITTED BY,

VIVEK BADGUJAR

B.Tech – Electronics & Telecommunication Engineering

MIT Academy of Engineering, Alandi

INTERNSHIP ORGANIZATION

ELEVANCESKILL

DOMAIN

Electrics Vehicle – Battery Management System

UNDER THE GUIDANCE OF

Mentor: Mr. Jayanth Sir

Date

20/08/2026

Offer Letter / Internship Selection Letter

ELEVANCE SKILLS

elevanceskills.com
support@elevanceskills.com
+91 9134772858
2/180, PUNGAMPATTI VILLAGE, BARUR POST,
POCHAMPALLI TALUK, KRISHNAGIRI, , 635201

June 22, 2026

Dear Vivek Badgujar,

We are pleased to offer you the opportunity to join ElevanceSkills as an **Embedded Systems** Intern.

This offer is conditional upon Annexure A: Terms and Conditions attached below and your successful completion of the required training program. Upon fulfilling the training criteria, you will embark on a journey of professional growth and real-world experience with us.

Congratulations!

Jay Kumar,
Chief Human Resources Officer

CERTIFICATE

Declaration

I hereby declare that the internship project report entitled **"Intelligent EV Battery Management System"** is an original work carried out by me during my internship at **ElevanceSkills** under the guidance of the assigned mentor.

This project has been developed as an integrated **Electric Vehicle Battery Management System (EV-BMS)** using **ESP32, Wokwi, Arduino C++, and Blynk IoT**. The work presented in this report covers the implementation, integration, testing, and analysis of the six internship tasks assigned as part of the internship program.

I further declare that the information presented in this report is based on the work carried out during the internship and has not been submitted previously, in whole or in part, for the award of any other academic qualification or certification.

Name: Vivek Badgujar
Branch: Electronics & Telecommunication Engineering
College: MIT Academy of Engineering, Alandi, Pune

Place: Alandi, Pune
Date: 20/08/2026

Signature of Student

Vivek Ravindra Badgujar

Acknowledgement

I would like to express my sincere gratitude to **ElevanceSkills** for providing me with the opportunity to undertake this internship in the field of **Electric Vehicle (EV) Battery Management Systems**.

I am thankful to my internship mentor for providing continuous guidance, technical support, and valuable suggestions throughout the development of the project. The mentor's feedback helped me understand the practical aspects of battery monitoring, safety protection, fault-tolerant embedded systems, cloud telemetry, and IoT-based dashboard development.

I would also like to express my sincere gratitude to the faculty members and management of **MIT Academy of Engineering, Alandi, Pune**, for their support and encouragement during the internship.

I am thankful to everyone who directly or indirectly contributed to the successful completion of this project. This internship provided me with valuable practical experience in **ESP32, embedded C/C++, Wokwi simulation, Blynk IoT, battery intelligence, event-driven programming, fault handling, and embedded system development**.

Abstract

The increasing adoption of electric vehicles (EVs) requires reliable and intelligent Battery Management Systems (BMS) for continuous monitoring, safety protection, fault detection, and battery condition assessment. This internship project presents the design and implementation of an **Intelligent EV Battery Management System** developed as a single integrated embedded platform using **ESP32, Wokwi Simulation, and Blynk IoT**.

The system is designed to monitor a simulated **four-cell lithium battery pack** in real time. It acquires individual cell voltage data and calculates important battery parameters such as pack voltage, average cell voltage, voltage imbalance, and the strongest and weakest cells. Based on these parameters, the system classifies the battery condition into different health states.

An event-driven safety protection mechanism is implemented using a non-blocking millis()-based architecture. The system detects conditions including under-voltage, over-voltage, sensor anomalies, and rapid voltage changes, and responds through relay protection, buzzer alerts, and LCD warning messages. A fault-tolerant runtime subsystem further detects conditions such as invalid sensor readings, frozen ADC behaviour, and relay mismatch while managing different operational states including **NORMAL, DEGRADED, FAILSAFE, and SHUTDOWN**.

The project also incorporates an embedded LCD-based Human-Machine Interface (HMI) with automatic diagnostic page rotation and priority-based fault display. Fault events are timestamped and logged for diagnostic purposes. For remote monitoring, an event-driven cloud telemetry architecture using **Blynk IoT** transmits important data during state changes, anomalies, and significant voltage variations while supporting network reconnection and event synchronization.

Finally, an executive battery intelligence dashboard provides live battery information, historical voltage trends, risk and diagnostic status, fault history, and operator recommendations. The complete system demonstrates the integration of **battery intelligence, safety protection, fault tolerance, embedded HMI, cloud telemetry, and IoT-based visualization** into a single simulated EV-BMS platform.

Table of Contents

7. SYSTEM OVERVIEW

7.1 Introduction to the Intelligent EV Battery Management System

The increasing adoption of Electric Vehicles (EVs) requires reliable Battery Management Systems (BMS) for continuous battery monitoring, safety protection, fault detection, and operational analysis. This project presents an **Intelligent EV Battery Management System** developed as a single integrated embedded system using an **ESP32 microcontroller, Wokwi simulation, and Blynk IoT**.

The developed system monitors a simulated four-cell lithium battery pack in real time. It measures individual cell voltages and calculates important battery parameters such as pack voltage, average cell voltage, voltage imbalance, and the strongest and weakest cells. Based on the calculated parameters, the system determines the battery health condition.

The system integrates six major functions: **Adaptive Multi-Cell Battery Intelligence, Event-Driven Safety Protection, Intelligent Embedded HMI, Fault-Tolerant Runtime Management, Intelligent Cloud Telemetry, and Executive Battery Intelligence Dashboard**. These functions operate together as a single integrated EV-BMS platform.

7.2 Problem Statement

Electric vehicle battery packs require continuous monitoring to ensure safe and reliable operation of individual cells. Abnormal voltage conditions, sensor faults, and system failures can affect battery performance and safety. This project develops an integrated EV-BMS for battery monitoring, safety protection, fault handling, cloud telemetry, and dashboard-based diagnostics.

7.3 Project Objectives

i. **To develop a real-time four-cell battery intelligence system** for measuring cell voltages, pack voltage, average voltage, imbalance, and identifying the strongest and weakest cells.

ii. **To classify battery health and implement safety protection** by detecting under-voltage, over-voltage, rapid voltage changes, and sensor anomalies using a non-blocking millis()-based architecture.

iii. **To develop an intelligent embedded HMI and fault-tolerant runtime system** for LCD diagnostics, protection status, fault detection, runtime modes, fault logging, and recovery.

iv. **To implement intelligent cloud telemetry using Blynk IoT** for event-based battery data transmission, Wi-Fi reconnection, and communication monitoring.

v. **To develop an executive battery intelligence dashboard** providing live monitoring, risk analysis, historical voltage trends, diagnostics, fault history, and operator recommendations.

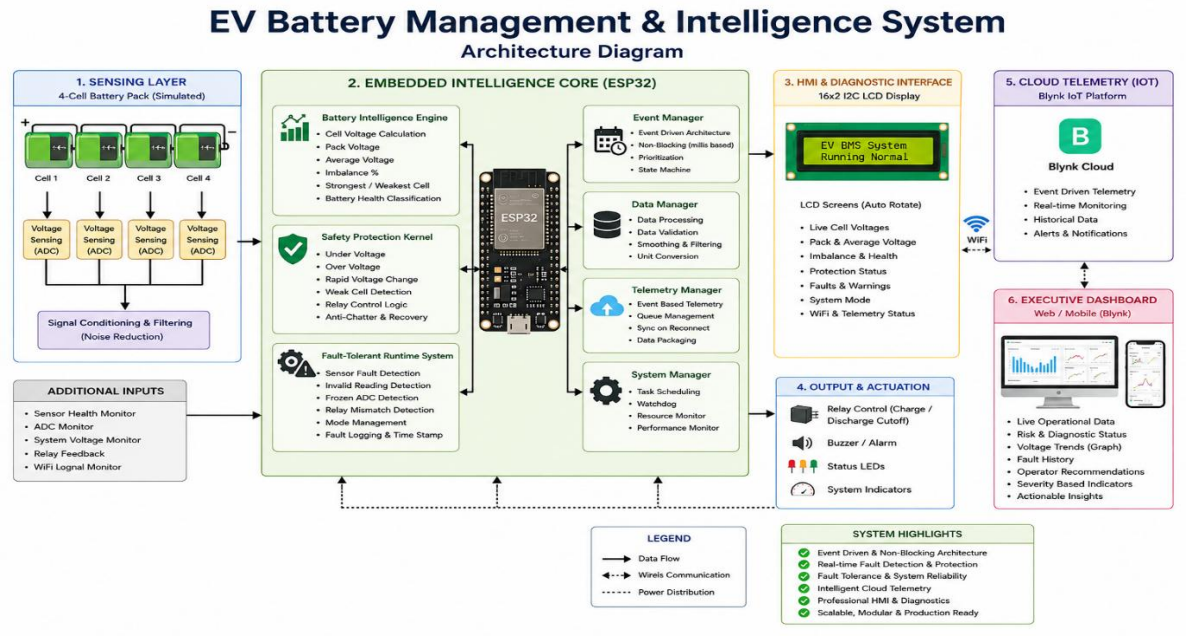
7.4 Scope of the Project

The project focuses on developing a **four-cell EV-BMS using ESP32 and Wokwi** for battery monitoring, safety protection, fault management, LCD diagnostics, Blynk cloud telemetry, and dashboard visualization. All six internship tasks are integrated into a **single system** for real-time simulation and validation.

7.5 System Components and Technologies

Category	Component / Technology
Microcontroller	ESP32
Battery	4-Cell Lithium Battery (For Simulation Potentiometer)
Display	16×2 I2C LCD
Protection	Relay & Buzzer
Indicators	LEDs
Programming	Arduino C++
Simulation	Wokwi
Cloud	Blynk IoT
Communication	Wi-Fi
Timing	millis()-based architecture

7.6 System Architecture Diagram



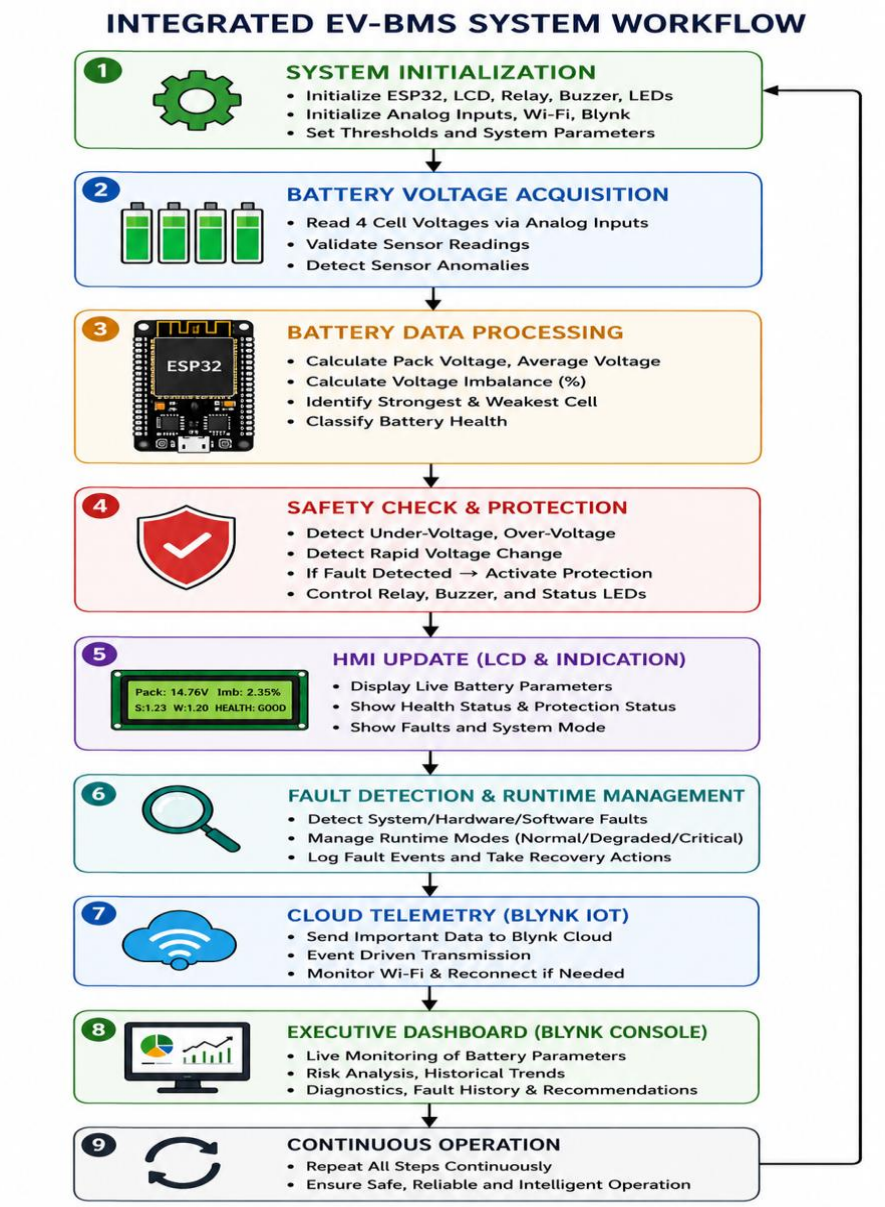
7.7 Architecture Description

The proposed EV-BMS architecture integrates **battery sensing, ESP32-based intelligence, safety protection, fault management, HMI, cloud telemetry, and dashboard monitoring** into a single system.

The **four-cell battery pack** provides individual cell voltage inputs to the sensing stage. The ESP32 processes these inputs to calculate battery parameters, detect abnormal conditions, manage faults, and control the protection outputs.

The processed information is displayed locally through the **16×2 I2C LCD** and transmitted through **Wi-Fi to the Blynk Cloud** for remote monitoring. The Blynk dashboard provides live battery data, risk status, historical trends, fault history, and operator recommendations.

7.8 System Workflow Diagram



8. Task 1: Battery Health Monitoring System (4-Cell pack)

8.1 Task 1 Problem

Adaptive Multi-Cell Battery Intelligence Engine: In this task, candidates are required to design and implement a production-grade battery intelligence engine capable of monitoring a simulated 4-cell lithium battery pack in real time. The system must read multiple analog inputs, calculate individual cell voltages, pack averages, imbalance percentages, and identify the weakest and strongest cells. It should also classify battery health into different states such as Healthy, Minor Imbalance, Critical Imbalance, and Pack Failure using scalable and modular embedded architecture principles.

8.2 Components:

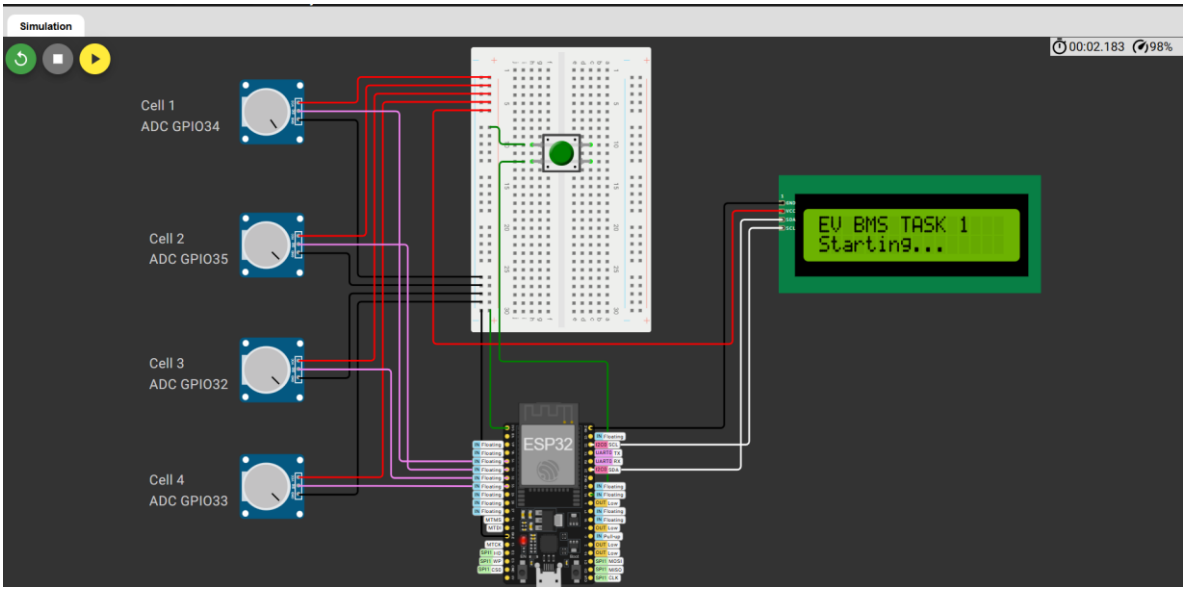
1. Microcontroller (ESP32- DevKit)
2. Breadboard
3. Pack of Cell (4-Cell) / Potentiometer (10K)
4. LCD 20×4 I²C
5. Push Button

8.3 Circuit Diagram Connection:

Component	Pin/Terminal	ESP32 Connection
Cell 1 Potentiometer	SIG	GPIO 34 (ADC)
	VCC	3.3 V
	Ground (GND)	GND
Cell 2 Potentiometer	SIG	GPIO 35 (ADC)
	VCC	3.3 V
	Ground (GND)	GND
Cell 3 Potentiometer	SIG	GPIO 32 (ADC)
	VCC	3.3 V
	Ground (GND)	GND
Cell 4 Potentiometer	SIG	GPIO 33 (ADC)
	VCC	3.3 V
	Ground (GND)	GND
16×2 I²C LCD	SDA	GPIO 21
	SCL	GPIO 22
	VCC	3.3 V

	GND	GND

8.4 Simulation Circuit :



8.5 Code:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ========================================================
// ADAPTIVE MULTI-CELL BATTERY INTELLIGENCE ENGINE
// TASK 1
//
// Hardware:
// ESP32 DevKit V1
// 4 x Potentiometers -> simulated 4 cells
// 16x2 I2C LCD
// 4 LEDs -> battery health indication
//
// ADC:
// Cell 1 -> GPIO34
// Cell 2 -> GPIO35
// Cell 3 -> GPIO32
// Cell 4 -> GPIO33
// ========================================================

// ========================================================
// 1. PIN CONFIGURATION
// ========================================================

// Cell ADC pins
#define CELL1_PIN 34
#define CELL2_PIN 35
#define CELL3_PIN 32
#define CELL4_PIN 33

// LCD I2C pins
```

```
#define SDA_PIN 21
#define SCL_PIN 22

// Health LEDs
#define LED_GREEN  4
#define LED_YELLOW 2
#define LED_ORANGE 15
#define LED_RED    5

// ==================================================
// 2. LCD CONFIGURATION
// ==================================================

LiquidCrystal_I2C lcd(0x27, 16, 2);

// ==================================================
// 3. BATTERY CONFIGURATION
// ==================================================

#define NUM_CELLS 4

// ESP32 ADC
const int ADC_MAX = 4095;
const float ADC_MAX_VOLTAGE = 3.3;

// Simulated cell voltage range
const float CELL_MIN_VOLTAGE = 2.5;
const float CELL_MAX_VOLTAGE = 4.2;

// ==================================================
// 4. HEALTH THRESHOLDS
// ==================================================
//
// These are demonstration thresholds for the project.
// They are NOT universal lithium battery safety limits.
//

const float HEALTHY_LIMIT = 2.0;
const float MINOR_LIMIT = 5.0;
const float CRITICAL_LIMIT = 10.0;

// ==================================================
// 5. TIMING
// ==================================================

const unsigned long SAMPLE_INTERVAL = 1000;
const unsigned long LCD_INTERVAL = 3000;
const unsigned long SERIAL_INTERVAL = 2000;

unsigned long previousSampleTime = 0;
unsigned long previousLCDTime = 0;
unsigned long previousSerialTime = 0;

// ==================================================
// 6. LCD PAGES
// ==================================================

int currentPage = 0;

const int TOTAL_PAGES = 4;

// ==================================================
// 7. BATTERY DATA
// ==================================================

float cellVoltage[NUM_CELLS];
```

```
float packVoltage = 0.0;
float averageVoltage = 0.0;

float maxVoltage = 0.0;
float minVoltage = 0.0;

float imbalanceVoltage = 0.0;
float imbalancePercent = 0.0;

int strongestCell = 0;
int weakestCell = 0;

String batteryHealth;

// ========================================================
// 8. SETUP
// ========================================================

void setup()
{
 Serial.begin(115200);

 // ESP32 ADC resolution
 analogReadResolution(12);

 // ADC input pins
 pinMode(CELL1_PIN, INPUT);
 pinMode(CELL2_PIN, INPUT);
 pinMode(CELL3_PIN, INPUT);
 pinMode(CELL4_PIN, INPUT);

 // LED pins
 pinMode(LED_GREEN, OUTPUT);
 pinMode(LED_YELLOW, OUTPUT);
 pinMode(LED_ORANGE, OUTPUT);
 pinMode(LED_RED, OUTPUT);

 // Turn LEDs OFF initially
 digitalWrite(LED_GREEN, LOW);
 digitalWrite(LED_YELLOW, LOW);
 digitalWrite(LED_ORANGE, LOW);
 digitalWrite(LED_RED, LOW);

 // I2C
 Wire.begin(SDA_PIN, SCL_PIN);

 // LCD
 lcd.init();
 lcd.backlight();

 lcd.clear();

 lcd.setCursor(0, 0);
 lcd.print("EV BMS TASK 1");

 lcd.setCursor(0, 1);
 lcd.print("Starting...");

 delay(1000);

 // First measurement
 readCellVoltages();
 calculateBatteryParameters();
 identifyStrongestWeakest();
 calculateImbalance();
```

```
 classifyBatteryHealth();
 updateLEDs();

 displayLCD();
 displaySerialTable();

 Serial.println();
 Serial.println("System started successfully.");
}

// =======================================================
// 9. MAIN LOOP
// =======================================================

void loop()
{
 unsigned long currentTime = millis();

 // -------------------------------------------------------
 // Battery measurement
 // -------------------------------------------------------

 if (currentTime - previousSampleTime >= SAMPLE_INTERVAL)
 {
  previousSampleTime = currentTime;

  readCellVoltages();

  calculateBatteryParameters();

  identifyStrongestWeakest();

  calculateImbalance();

  classifyBatteryHealth();

  updateLEDs();
 }

 // -------------------------------------------------------
 // LCD automatic screen rotation
 // -------------------------------------------------------

 if (currentTime - previousLCDTime >= LCD_INTERVAL)
 {
  previousLCDTime = currentTime;

  currentPage++;

  if (currentPage >= TOTAL_PAGES)
  {
   currentPage = 0;
  }

  displayLCD();
 }

 // -------------------------------------------------------
 // Serial table
 // -------------------------------------------------------

 if (currentTime - previousSerialTime >= SERIAL_INTERVAL)
 {
  previousSerialTime = currentTime;

  displaySerialTable();
```

```
 }
}

// ====================================================
// 10. READ FOUR CELL VOLTAGES
// ====================================================

void readCellVoltages()
{
 int rawCell1 = analogRead(CELL1_PIN);
 int rawCell2 = analogRead(CELL2_PIN);
 int rawCell3 = analogRead(CELL3_PIN);
 int rawCell4 = analogRead(CELL4_PIN);

 cellVoltage[0] = convertADCToCellVoltage(rawCell1);
 cellVoltage[1] = convertADCToCellVoltage(rawCell2);
 cellVoltage[2] = convertADCToCellVoltage(rawCell3);
 cellVoltage[3] = convertADCToCellVoltage(rawCell4);
}

// ====================================================
// 11. ADC TO SIMULATED CELL VOLTAGE
// ====================================================

float convertADCToCellVoltage(int rawADC)
{
 float adcVoltage;

 adcVoltage =
  ((float)rawADC / ADC_MAX) * ADC_MAX_VOLTAGE;

 float simulatedVoltage;

 simulatedVoltage =
  CELL_MIN_VOLTAGE +
  (adcVoltage / ADC_MAX_VOLTAGE) *
  (CELL_MAX_VOLTAGE - CELL_MIN_VOLTAGE);

 return simulatedVoltage;
}

// ====================================================
// 12. CALCULATE BATTERY PARAMETERS
// ====================================================

void calculateBatteryParameters()
{
 packVoltage = 0.0;

 // Calculate pack voltage
 for (int i = 0; i < NUM_CELLS; i++)
 {
  packVoltage += cellVoltage[i];
 }

 // Average cell voltage
 averageVoltage =
  packVoltage / NUM_CELLS;

 // Initial maximum and minimum
 maxVoltage = cellVoltage[0];
 minVoltage = cellVoltage[0];

 // Find maximum and minimum
 for (int i = 1; i < NUM_CELLS; i++)
 {
```

```
  if (cellVoltage[i] > maxVoltage)
  {
   maxVoltage = cellVoltage[i];
  }

  if (cellVoltage[i] < minVoltage)
  {
   minVoltage = cellVoltage[i];
  }
 }
}

// ======================================================
// 13. IDENTIFY STRONGEST AND WEAKEST CELL
// ======================================================

void identifyStrongestWeakest()
{
 strongestCell = 0;
 weakestCell = 0;

 for (int i = 1; i < NUM_CELLS; i++)
 {
  if (cellVoltage[i] >
     cellVoltage[strongestCell])
  {
   strongestCell = i;
  }

  if (cellVoltage[i] <
     cellVoltage[weakestCell])
  {
   weakestCell = i;
  }
 }
}

// ======================================================
// 14. CALCULATE IMBALANCE
// ======================================================

void calculateImbalance()
{
 // Difference between highest and lowest cell
 imbalanceVoltage =
  maxVoltage - minVoltage;

 // Percentage imbalance
 if (averageVoltage > 0)
 {
  imbalancePercent =
   (imbalanceVoltage /
    averageVoltage) * 100.0;
 }
 else
 {
  imbalancePercent = 0.0;
 }
}

// ======================================================
// 15. BATTERY HEALTH CLASSIFICATION
// ======================================================

void classifyBatteryHealth()
{
```

```
 if (imbalancePercent < HEALTHY_LIMIT)
 {
  batteryHealth = "HEALTHY";
 }

 else if (imbalancePercent < MINOR_LIMIT)
 {
  batteryHealth = "MINOR IMBALANCE";
 }

 else if (imbalancePercent < CRITICAL_LIMIT)
 {
  batteryHealth = "CRITICAL IMBALANCE";
 }

 else
 {
  batteryHealth = "PACK FAILURE";
 }
}

// ======================================================
// 16. UPDATE HEALTH LEDs
// ======================================================

void updateLEDs()
{
 // Turn everything OFF
 digitalWrite(LED_GREEN, LOW);
 digitalWrite(LED_YELLOW, LOW);
 digitalWrite(LED_ORANGE, LOW);
 digitalWrite(LED_RED, LOW);

 if (batteryHealth == "HEALTHY")
 {
  digitalWrite(LED_GREEN, HIGH);
 }

 else if (batteryHealth == "MINOR IMBALANCE")
 {
  digitalWrite(LED_YELLOW, HIGH);
 }

 else if (batteryHealth == "CRITICAL IMBALANCE")
 {
  digitalWrite(LED_ORANGE, HIGH);
 }

 else
 {
  digitalWrite(LED_RED, HIGH);
 }
}

// ======================================================
// 17. LCD DISPLAY
// ======================================================

void displayLCD()
{
 lcd.clear();

 // ----------------------------------------------------
 // PAGE 1: CELL VOLTAGES
 // ----------------------------------------------------
```

```
if (currentPage == 0)
{
  lcd.setCursor(0, 0);

  lcd.print("C1:");
  lcd.print(cellVoltage[0], 2);

  lcd.print(" C2:");
  lcd.print(cellVoltage[1], 2);

  lcd.setCursor(0, 1);

  lcd.print("C3:");
  lcd.print(cellVoltage[2], 2);

  lcd.print(" C4:");
  lcd.print(cellVoltage[3], 2);
}

// ------------------------------------------------------
// PAGE 2: PACK PARAMETERS
// ------------------------------------------------------

else if (currentPage == 1)
{
  lcd.setCursor(0, 0);

  lcd.print("PACK:");
  lcd.print(packVoltage, 2);

  lcd.print("V");

  lcd.setCursor(0, 1);

  lcd.print("AVG:");
  lcd.print(averageVoltage, 2);

  lcd.print("V");
}

// ------------------------------------------------------
// PAGE 3: IMBALANCE
// ------------------------------------------------------

else if (currentPage == 2)
{
  lcd.setCursor(0, 0);

  lcd.print("IMB:");
  lcd.print(imbalancePercent, 2);

  lcd.print("%");

  lcd.setCursor(0, 1);

  lcd.print("W:C");
  lcd.print(weakestCell + 1);

  lcd.print(" S:C");
  lcd.print(strongestCell + 1);
}

// ------------------------------------------------------
// PAGE 4: HEALTH
// ------------------------------------------------------
```

```
 else if (currentPage == 3)
 {
  lcd.setCursor(0, 0);

  if (batteryHealth == "HEALTHY")
  {
   lcd.print("STATUS: HEALTHY");
  }

  else if (batteryHealth == "MINOR IMBALANCE")
  {
   lcd.print("STATUS: MINOR");
  }

  else if (batteryHealth == "CRITICAL IMBALANCE")
  {
   lcd.print("STATUS: CRITICAL");
  }

  else
  {
   lcd.print("STATUS: FAILURE");
  }

  lcd.setCursor(0, 1);

  lcd.print("DV:");
  lcd.print(imbalanceVoltage, 2);

  lcd.print("V");
 }
}

// ======================================================
// 18. SERIAL MONITOR TABLE
// ======================================================

void displaySerialTable()
{
 Serial.println();
 Serial.println("================================================");
 Serial.println("      ADAPTIVE BATTERY INTELLIGENCE");
 Serial.println("================================================");

 Serial.println("Parameter              Value");
 Serial.println("------------------------------------------------");

 Serial.print("Cell 1 Voltage         ");
 Serial.print(cellVoltage[0], 2);
 Serial.println(" V");

 Serial.print("Cell 2 Voltage         ");
 Serial.print(cellVoltage[1], 2);
 Serial.println(" V");

 Serial.print("Cell 3 Voltage         ");
 Serial.print(cellVoltage[2], 2);
 Serial.println(" V");

 Serial.print("Cell 4 Voltage         ");
 Serial.print(cellVoltage[3], 2);
 Serial.println(" V");

 Serial.println("------------------------------------------------");

 Serial.print("Pack Voltage           ");
```

```
  Serial.print(packVoltage, 2);
  Serial.println(" V");

  Serial.print("Average Voltage        ");
  Serial.print(averageVoltage, 2);
  Serial.println(" V");

  Serial.print("Maximum Voltage        ");
  Serial.print(maxVoltage, 2);
  Serial.println(" V");

  Serial.print("Minimum Voltage        ");
  Serial.print(minVoltage, 2);
  Serial.println(" V");

  Serial.print("Imbalance Voltage      ");
  Serial.print(imbalanceVoltage, 2);
  Serial.println(" V");

  Serial.print("Imbalance Percentage   ");
  Serial.print(imbalancePercent, 2);
  Serial.println(" %");

  Serial.print("Strongest Cell         Cell ");
  Serial.println(strongestCell + 1);

  Serial.print("Weakest Cell           Cell ");
  Serial.println(weakestCell + 1);

  Serial.print("Battery Health         ");
  Serial.println(batteryHealth);

  Serial.println("------------------------------------------------");
  Serial.println("          END OF SAMPLE");
  Serial.println("================================================");
}
```

8.6 Imbalance Calculation formulas

a. **Imbalance Voltage:**

$$\Delta V = V_{max} - V_{min}$$

b. **Imbalance Percentage:**

$$\text{Imbalance\%} = [\ (V_{max} - V_{min}) / V_{avg}\ ] \times 100$$

Example

If: Cell 1 = 3.70 V
Cell 2 = 3.68 V
Cell 3 = 3.72 V
Cell 4 = 3.65 V

Then:

Pack Voltage : 3.70+3.68+3.72+3.65=14.75V

Average Voltage : 14.75/4=3.6875V

8.7 Strongest & Weakest Cell:

Strongest Cell: 3.72 V → Cell 3

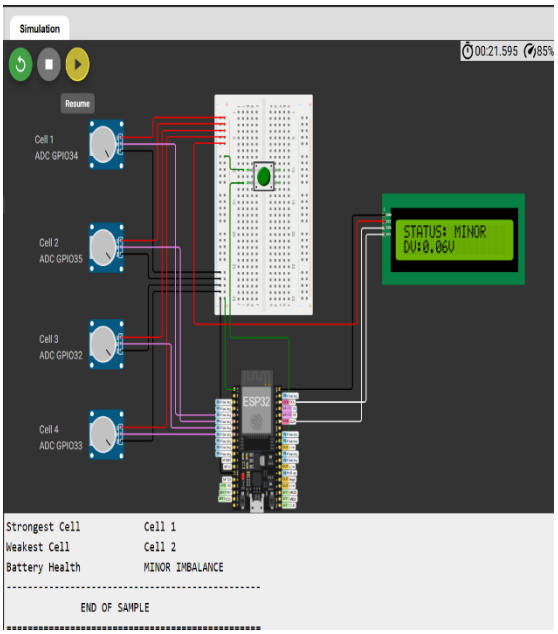
Weakest Cell: 3.65 V → Cell 4

Imbalance : 3.6875

$(3.72-3.65) \times 100 \approx 1.90\%$

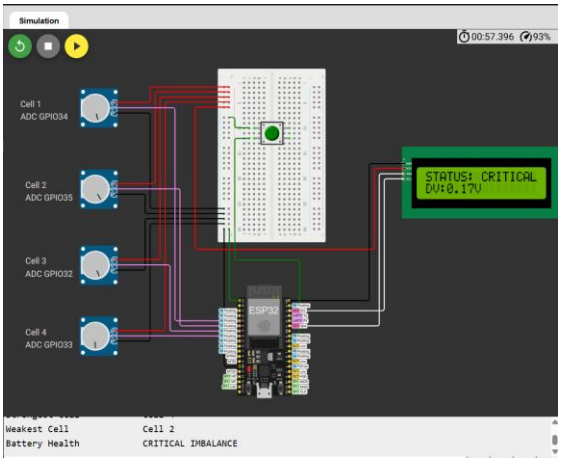
8.8 Simulation Running Photos with correct Output:

Battery Health = Minor Imbalance



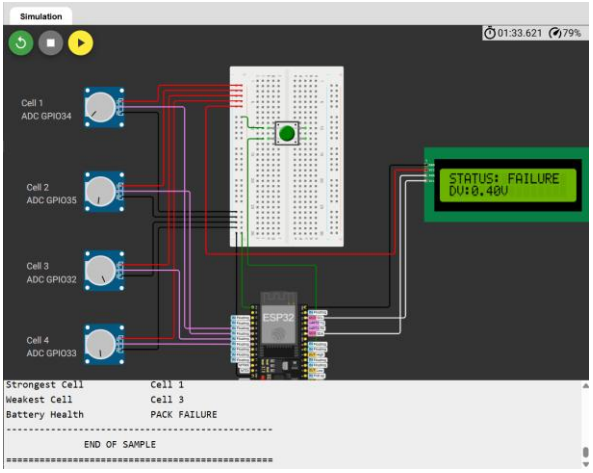

```
System started successfully.

================================================
       ADAPTIVE BATTERY INTELLIGENCE
================================================
Parameter                Value
------------------------------------------------
Cell 1 Voltage           2.56 V
Cell 2 Voltage           2.50 V
Cell 3 Voltage           2.50 V
Cell 4 Voltage           2.50 V
------------------------------------------------
Pack Voltage             10.06 V
Average Voltage          2.52 V
Maximum Voltage          2.56 V
Minimum Voltage          2.50 V
Imbalance Voltage        0.06 V
Imbalance Percentage     2.51 %
Strongest Cell           Cell 1
Weakest Cell             Cell 2
Battery Health           MINOR IMBALANCE
------------------------------------------------
              END OF SAMPLE
================================================
```

Battery Health = Critical Imbalance



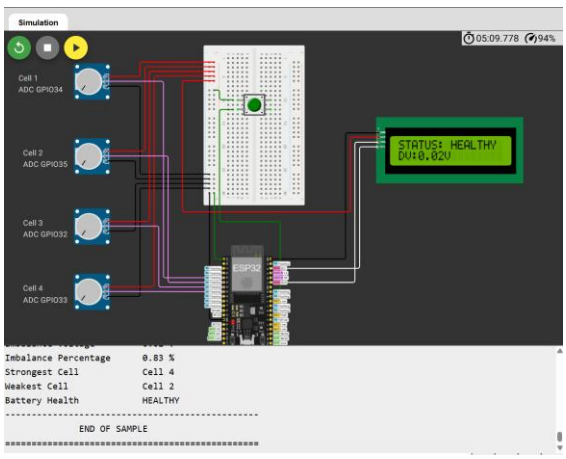

```
       ADAPTIVE BATTERY INTELLIGENCE
================================================
Parameter                 Value
------------------------------------------------
Cell 1 Voltage            2.68 V
Cell 2 Voltage            2.59 V
Cell 3 Voltage            2.64 V
Cell 4 Voltage            2.76 V
------------------------------------------------
Pack Voltage              10.67 V
Average Voltage           2.67 V
Maximum Voltage           2.76 V
Minimum Voltage           2.59 V
Imbalance Voltage         0.17 V
Imbalance Percentage      6.35 %
Strongest Cell            Cell 4
Weakest Cell              Cell 2
Battery Health            CRITICAL IMBALANCE
------------------------------------------------
              END OF SAMPLE
================================================
```

Battery Health = Pack Failure




```
================================================
       ADAPTIVE BATTERY INTELLIGENCE
================================================
Parameter                 Value
------------------------------------------------
Cell 1 Voltage            3.04 V
Cell 2 Voltage            2.83 V
Cell 3 Voltage            2.64 V
Cell 4 Voltage            2.76 V
------------------------------------------------
Pack Voltage              11.26 V
Average Voltage           2.82 V
Maximum Voltage           3.04 V
Minimum Voltage           2.64 V
Imbalance Voltage         0.40 V
Imbalance Percentage      14.34 %
Strongest Cell            Cell 1
Weakest Cell              Cell 3
Battery Health            PACK FAILURE
------------------------------------------------
              END OF SAMPLE
================================================
```

Battery Health = Pack Failure




```
================================================
       ADAPTIVE BATTERY INTELLIGENCE
================================================
Parameter                 Value
------------------------------------------------
Cell 1 Voltage            3.00 V
Cell 2 Voltage            2.99 V
Cell 3 Voltage            2.99 V
Cell 4 Voltage            3.02 V
------------------------------------------------
Pack Voltage              12.00 V
Average Voltage           3.00 V
Maximum Voltage           3.02 V
Minimum Voltage           2.99 V
Imbalance Voltage         0.02 V
Imbalance Percentage      0.83 %
Strongest Cell            Cell 4
Weakest Cell              Cell 2
Battery Health            HEALTHY
------------------------------------------------
              END OF SAMPLE
================================================
```

8.9 Important Notes:

[Threshold Limits]

Imbalance % / Condition	Health State	LED Indication
0 – <3%	HEALTHY	Green
3 – 10%	MINOR_IMBAL	Yellow
>10%	CRITICAL_IMBAL	Orange
Any cell UV	PACK_FAILURE	Red

8.10 Link of Simulation :

Task 1 Click Here

8.11 Task 1 Video Demonstration Link:

Click Here

9. Task 2: Battery Intelligence Engine Safety Kernel Module

9.1 Task 2 Problem

Event-Driven Safety Protection Kernel: This task focuses on developing an automotive-grade safety protection controller using a fully non-blocking event-driven architecture. Candidates must detect conditions such as weak cell voltage, overvoltage, sensor anomalies, and rapid voltage fluctuations while controlling relay cutoffs, buzzer alerts, and LCD warning messages. The implementation must rely entirely on millis()-based timing without using delay() and should include recovery logic, anti-relay chatter protection, and stable state management.

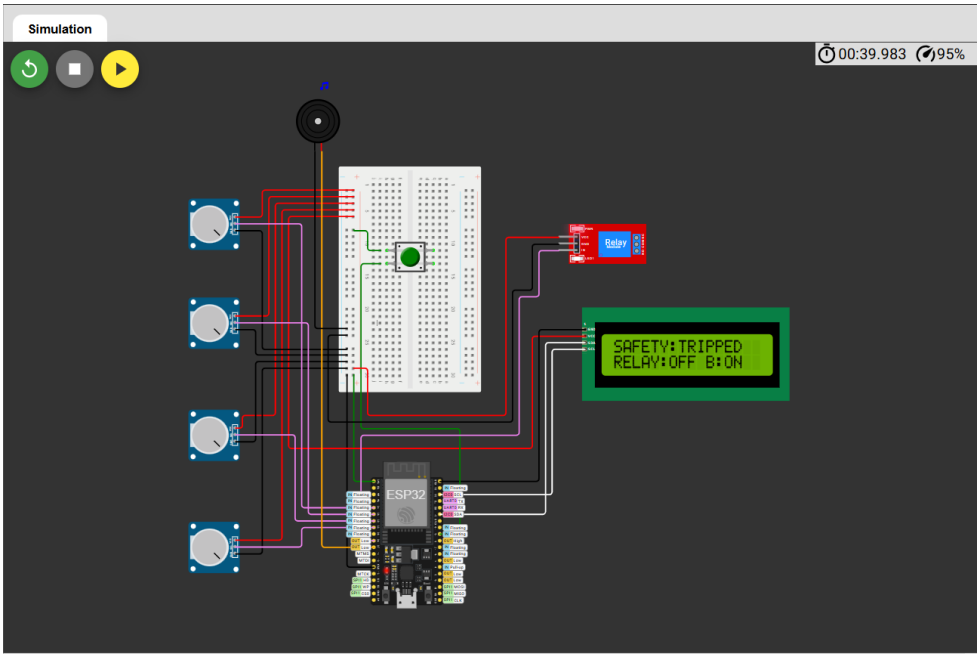
9.2 Components

1. Microcontroller (ESP32- DevKit)
2. Breadboard
3. Pack of Cell (4-Cell) / Potentiometer (10K)
4. LCD 20×4 I²C
5. Push Button

New Components

6. Relay Module
7. Buzzer

9.3 Simulation Circuit :



9.4 Code :

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ======================================================
// EV BATTERY MANAGEMENT SYSTEM
// TASK 1 + TASK 2
//
// TASK 1:
// Adaptive Multi-Cell Battery Intelligence Engine
//
// TASK 2:
// Event-Driven Safety Protection Kernel
//
// Hardware:
// ESP32 DevKit V1
// 4 x Potentiometers = simulated cells
// 16x2 I2C LCD
// 4 x Status LEDs
// Relay Module
```

```
// Buzzer
//
// Cell 1 -> GPIO34
// Cell 2 -> GPIO35
// Cell 3 -> GPIO32
// Cell 4 -> GPIO33
//
// LCD SDA -> GPIO21
// LCD SCL -> GPIO22
//
// Green LED  -> GPIO4
// Yellow LED -> GPIO2
// Orange LED -> GPIO15
// Red LED    -> GPIO5
//
// Relay -> GPIO26
// Buzzer -> GPIO27
// ================================================

// ================================================
// 1. PIN CONFIGURATION
// ================================================

// -------- Cell ADC pins --------

#define CELL1_PIN 34
#define CELL2_PIN 35
#define CELL3_PIN 32
#define CELL4_PIN 33

// -------- LCD --------

#define SDA_PIN 21
#define SCL_PIN 22

// -------- Health LEDs --------

#define LED_GREEN  4
#define LED_YELLOW 2
#define LED_ORANGE 15
#define LED_RED    5

// -------- Task 2 protection --------

#define RELAY_PIN  26
#define BUZZER_PIN 27

// ================================================
// 2. LCD CONFIGURATION
// ================================================

LiquidCrystal_I2C lcd(0x27, 16, 2);

// ================================================
// 3. BATTERY CONFIGURATION
// ================================================

#define NUM_CELLS 4

const int ADC_MAX = 4095;

const float ADC_MAX_VOLTAGE = 3.3;

// Simulated cell voltage range

const float CELL_MIN_VOLTAGE = 2.5;
const float CELL_MAX_VOLTAGE = 4.2;

// ================================================
// 4. TASK 1 HEALTH THRESHOLDS
// ================================================
```

```
const float HEALTHY_LIMIT = 2.0;

const float MINOR_LIMIT = 5.0;

const float CRITICAL_LIMIT = 10.0;

// ======================================================
// 5. TASK 2 SAFETY THRESHOLDS
// ======================================================

// Under-voltage threshold
const float UV_THRESHOLD = 2.8;

// Over-voltage threshold
const float OV_THRESHOLD = 4.15;

// Rapid voltage change threshold
const float RAPID_CHANGE_THRESHOLD = 0.10;

// ======================================================
// 6. TASK 2 TIMING
// ======================================================

// Battery sampling
const unsigned long SAMPLE_INTERVAL = 200;

// LCD update
const unsigned long LCD_INTERVAL = 3000;

// Serial report
const unsigned long SERIAL_INTERVAL = 2000;

// Fault confirmation
const unsigned long FAULT_CONFIRM_TIME = 500;

// Recovery time
const unsigned long RECOVERY_TIME = 5000;

// Minimum relay switching time
const unsigned long RELAY_LOCK_TIME = 3000;

// Buzzer interval
const unsigned long BUZZER_INTERVAL = 300;

// ======================================================
// 7. TIMERS
// ======================================================

unsigned long previousSampleTime = 0;

unsigned long previousLCDTime = 0;

unsigned long previousSerialTime = 0;

unsigned long faultStartTime = 0;

unsigned long recoveryStartTime = 0;

unsigned long lastRelayChangeTime = 0;

unsigned long previousBuzzerTime = 0;

// ======================================================
// 8. BATTERY DATA
// ======================================================

float cellVoltage[NUM_CELLS];

float previousCellVoltage[NUM_CELLS];
```

```
float packVoltage = 0.0;

float averageVoltage = 0.0;

float maxVoltage = 0.0;

float minVoltage = 0.0;

float imbalanceVoltage = 0.0;

float imbalancePercent = 0.0;

int strongestCell = 0;

int weakestCell = 0;

String batteryHealth;

// ====================================================
// 9. SAFETY STATE MACHINE
// ====================================================

enum SafetyState
{
  SAFETY_NORMAL,
  SAFETY_WARNING,
  SAFETY_TRIPPED,
  SAFETY_RECOVERY
};

SafetyState safetyState = SAFETY_NORMAL;

// ====================================================
// 10. FAULT TYPES
// ====================================================

bool underVoltageFault = false;

bool overVoltageFault = false;

bool rapidChangeFault = false;

bool sensorFault = false;

bool faultDetected = false;

bool faultConfirmed = false;

// ====================================================
// 11. RELAY STATE
// ====================================================

bool relayState = true;

// ====================================================
// 12. LCD PAGES
// ====================================================

int currentPage = 0;

const int TOTAL_PAGES = 5;

// ====================================================
// SETUP
// ====================================================

void setup()
{
  Serial.begin(115200);

  // ---------- ADC ----------
```

```
analogReadResolution(12);

pinMode(CELL1_PIN, INPUT);
pinMode(CELL2_PIN, INPUT);
pinMode(CELL3_PIN, INPUT);
pinMode(CELL4_PIN, INPUT);

// ---------- LEDs ----------

pinMode(LED_GREEN, OUTPUT);
pinMode(LED_YELLOW, OUTPUT);
pinMode(LED_ORANGE, OUTPUT);
pinMode(LED_RED, OUTPUT);

// ---------- Relay ----------

pinMode(RELAY_PIN, OUTPUT);

// Relay initially ON
digitalWrite(RELAY_PIN, HIGH);

relayState = true;

// ---------- Buzzer ----------

pinMode(BUZZER_PIN, OUTPUT);

digitalWrite(BUZZER_PIN, LOW);

// ---------- I2C ----------

Wire.begin(SDA_PIN, SCL_PIN);

// ---------- LCD ----------

lcd.init();

lcd.backlight();

lcd.clear();

lcd.setCursor(0, 0);

lcd.print("EV BMS TASK 2");

lcd.setCursor(0, 1);

lcd.print("Safety Kernel");

delay(1500);

// ---------- Initial reading ----------

readCellVoltages();

calculateBatteryParameters();

identifyStrongestWeakest();

calculateImbalance();

classifyBatteryHealth();

// Store initial voltage
for (int i = 0; i < NUM_CELLS; i++)
{
  previousCellVoltage[i] = cellVoltage[i];
}

updateLEDs();
```

```
  updateLCD();

  displaySerialReport();

  Serial.println();

  Serial.println("=====================================");

  Serial.println("TASK 2 SAFETY KERNEL READY");

  Serial.println("=====================================");
}

// ====================================================
// MAIN LOOP
// ====================================================

void loop()
{
  unsigned long currentTime = millis();

  // ================================================
  // 1. BATTERY SAMPLING
  // ================================================

  if (currentTime - previousSampleTime >= SAMPLE_INTERVAL)
  {
    previousSampleTime = currentTime;

    // Task 1
    readCellVoltages();

    calculateBatteryParameters();

    identifyStrongestWeakest();

    calculateImbalance();

    classifyBatteryHealth();

    // Task 2
    detectSafetyConditions();

    updateSafetyState();

    updateRelay();

    updateBuzzer();

    updateLEDs();
  }

  // ================================================
  // 2. LCD PAGE ROTATION
  // ================================================

  if (currentTime - previousLCDTime >= LCD_INTERVAL)
  {
    previousLCDTime = currentTime;

    currentPage++;

    if (currentPage >= TOTAL_PAGES)
    {
      currentPage = 0;
    }

    updateLCD();
  }
```

```
  // ====================================================
  // 3. SERIAL REPORT
  // ====================================================

  if (currentTime - previousSerialTime >= SERIAL_INTERVAL)
  {
    previousSerialTime = currentTime;

    displaySerialReport();
  }
}

// ====================================================
// TASK 1
// READ FOUR CELL VOLTAGES
// ====================================================

void readCellVoltages()
{
  int rawCell1 = analogRead(CELL1_PIN);

  int rawCell2 = analogRead(CELL2_PIN);

  int rawCell3 = analogRead(CELL3_PIN);

  int rawCell4 = analogRead(CELL4_PIN);

  cellVoltage[0] =
    convertADCToCellVoltage(rawCell1);

  cellVoltage[1] =
    convertADCToCellVoltage(rawCell2);

  cellVoltage[2] =
    convertADCToCellVoltage(rawCell3);

  cellVoltage[3] =
    convertADCToCellVoltage(rawCell4);
}

// ====================================================
// TASK 1
// ADC TO SIMULATED CELL VOLTAGE
// ====================================================

float convertADCToCellVoltage(int rawADC)
{
  float adcVoltage;

  adcVoltage =
    ((float)rawADC / ADC_MAX)
    * ADC_MAX_VOLTAGE;

  float simulatedVoltage;

  simulatedVoltage =
    CELL_MIN_VOLTAGE
    +
    (adcVoltage / ADC_MAX_VOLTAGE)
    *
    (CELL_MAX_VOLTAGE -
     CELL_MIN_VOLTAGE);

  return simulatedVoltage;
}

// ====================================================
// TASK 1
// CALCULATE BATTERY PARAMETERS
// ====================================================
```

```
void calculateBatteryParameters()
{
 packVoltage = 0.0;

 // Pack voltage

 for (int i = 0; i < NUM_CELLS; i++)
 {
  packVoltage += cellVoltage[i];
 }

 // Average

 averageVoltage =
  packVoltage / NUM_CELLS;

 // Initial min/max

 maxVoltage = cellVoltage[0];

 minVoltage = cellVoltage[0];

 // Find min/max

 for (int i = 1; i < NUM_CELLS; i++)
 {
  if (cellVoltage[i] > maxVoltage)
  {
   maxVoltage = cellVoltage[i];
  }

  if (cellVoltage[i] < minVoltage)
  {
   minVoltage = cellVoltage[i];
  }
 }
}

// =====================================================
// TASK 1
// IDENTIFY STRONGEST / WEAKEST CELL
// =====================================================

void identifyStrongestWeakest()
{
 strongestCell = 0;

 weakestCell = 0;

 for (int i = 1; i < NUM_CELLS; i++)
 {
  if (cellVoltage[i] >
     cellVoltage[strongestCell])
  {
   strongestCell = i;
  }

  if (cellVoltage[i] <
     cellVoltage[weakestCell])
  {
   weakestCell = i;
  }
 }
}

// =====================================================
// TASK 1
// CALCULATE IMBALANCE
// =====================================================

void calculateImbalance()
```

```
{
 imbalanceVoltage =
  maxVoltage - minVoltage;

 if (averageVoltage > 0)
 {
  imbalancePercent =
   (imbalanceVoltage /
    averageVoltage)
   * 100.0;
 }
 else
 {
  imbalancePercent = 0.0;
 }
}

// ======================================================
// TASK 1
// HEALTH CLASSIFICATION
// ======================================================

void classifyBatteryHealth()
{
 if (imbalancePercent < HEALTHY_LIMIT)
 {
  batteryHealth = "HEALTHY";
 }

 else if (imbalancePercent < MINOR_LIMIT)
 {
  batteryHealth = "MINOR IMBALANCE";
 }

 else if (imbalancePercent < CRITICAL_LIMIT)
 {
  batteryHealth = "CRITICAL IMBALANCE";
 }

 else
 {
  batteryHealth = "PACK FAILURE";
 }
}

// ======================================================
// TASK 2
// DETECT SAFETY CONDITIONS
// ======================================================

void detectSafetyConditions()
{
 underVoltageFault = false;

 overVoltageFault = false;

 rapidChangeFault = false;

 sensorFault = false;

 // ------------------------------------------------
 // Check all cells
 // ------------------------------------------------

 for (int i = 0; i < NUM_CELLS; i++)
 {
  // Sensor anomaly
  if (cellVoltage[i] < CELL_MIN_VOLTAGE ||
     cellVoltage[i] > CELL_MAX_VOLTAGE)
  {
   sensorFault = true;
```

```
    }

    // Under-voltage
    if (cellVoltage[i] < UV_THRESHOLD)
    {
      underVoltageFault = true;
    }

    // Over-voltage
    if (cellVoltage[i] > OV_THRESHOLD)
    {
      overVoltageFault = true;
    }

    // Rapid voltage change
    float voltageChange =
      fabs(cellVoltage[i] -
          previousCellVoltage[i]);

    if (voltageChange >
       RAPID_CHANGE_THRESHOLD)
    {
      rapidChangeFault = true;
    }
  }

  // ------------------------------------------------
  // Store current readings
  // ------------------------------------------------

  for (int i = 0; i < NUM_CELLS; i++)
  {
    previousCellVoltage[i] =
      cellVoltage[i];
  }

  // ------------------------------------------------
  // Overall fault
  // ------------------------------------------------

  faultDetected =
    underVoltageFault ||
    overVoltageFault ||
    rapidChangeFault ||
    sensorFault;
}

// ======================================================
// TASK 2
// SAFETY STATE MACHINE
// ======================================================

void updateSafetyState()
{
  unsigned long now = millis();

  switch (safetyState)
  {

    // ================================================
    // NORMAL
    // ================================================

    case SAFETY_NORMAL:

      if (faultDetected)
      {
        safetyState = SAFETY_WARNING;

        faultStartTime = now;
      }
```

```
    break;

  // ================================================
  // WARNING
  // ================================================

  case SAFETY_WARNING:

    // Fault disappeared
    if (!faultDetected)
    {
      safetyState = SAFETY_NORMAL;

      faultConfirmed = false;

      break;
    }

    // Fault continues
    if (now - faultStartTime >=
        FAULT_CONFIRM_TIME)
    {
      faultConfirmed = true;

      safetyState = SAFETY_TRIPPED;
    }

    break;

  // ================================================
  // TRIPPED
  // ================================================

  case SAFETY_TRIPPED:

    // Wait until all faults disappear
    if (!faultDetected)
    {
      recoveryStartTime = now;

      safetyState = SAFETY_RECOVERY;
    }

    break;

  // ================================================
  // RECOVERY
  // ================================================

  case SAFETY_RECOVERY:

    // Fault came back
    if (faultDetected)
    {
      safetyState = SAFETY_TRIPPED;

      break;
    }

    // Battery stable long enough
    if (now - recoveryStartTime >=
        RECOVERY_TIME)
    {
      safetyState = SAFETY_NORMAL;

      faultConfirmed = false;
    }

    break;
  }
```

```
}

// ====================================================
// TASK 2
// RELAY CONTROL
// ====================================================

void updateRelay()
{
  unsigned long now = millis();

  // ------------------------------------------------
  // TRIPPED → Relay OFF
  // ------------------------------------------------

  if (safetyState == SAFETY_TRIPPED ||
     safetyState == SAFETY_WARNING)
  {
    if (relayState == true)
    {
      if (now - lastRelayChangeTime >=
         RELAY_LOCK_TIME)
      {
        digitalWrite(RELAY_PIN, LOW);

        relayState = false;

        lastRelayChangeTime = now;

        Serial.println(
          ">>> RELAY CUT-OFF <<<"
        );
      }
    }
  }

  // ------------------------------------------------
  // RECOVERY / NORMAL → Relay ON
  // ------------------------------------------------

  else if (safetyState ==
        SAFETY_NORMAL)
  {
    if (relayState == false)
    {
      if (now - lastRelayChangeTime >=
         RELAY_LOCK_TIME)
      {
        digitalWrite(RELAY_PIN, HIGH);

        relayState = true;

        lastRelayChangeTime = now;

        Serial.println(
          ">>> RELAY RESTORED <<<"
        );
      }
    }
  }
}

// ====================================================
// TASK 2
// BUZZER CONTROL
// ====================================================

void updateBuzzer()
{
  unsigned long now = millis();
```

```
 if (safetyState ==
    SAFETY_WARNING ||
    safetyState ==
    SAFETY_TRIPPED)
 {
  if (now - previousBuzzerTime >=
     BUZZER_INTERVAL)
  {
    previousBuzzerTime = now;

    // Toggle buzzer
    digitalWrite(
     BUZZER_PIN,
     !digitalRead(BUZZER_PIN)
    );
  }
 }

 else
 {
  digitalWrite(BUZZER_PIN, LOW);
 }
}

// ====================================================
// LED HEALTH INDICATION
// ====================================================

void updateLEDs()
{
 digitalWrite(LED_GREEN, LOW);

 digitalWrite(LED_YELLOW, LOW);

 digitalWrite(LED_ORANGE, LOW);

 digitalWrite(LED_RED, LOW);

 // Safety fault gets priority

 if (safetyState ==
    SAFETY_WARNING ||
    safetyState ==
    SAFETY_TRIPPED)
 {
  digitalWrite(LED_RED, HIGH);

  return;
 }

 // Normal health indication

 if (batteryHealth == "HEALTHY")
 {
  digitalWrite(LED_GREEN, HIGH);
 }

 else if (batteryHealth ==
       "MINOR IMBALANCE")
 {
  digitalWrite(LED_YELLOW, HIGH);
 }

 else if (batteryHealth ==
       "CRITICAL IMBALANCE")
 {
  digitalWrite(LED_ORANGE, HIGH);
 }

 else
 {
```

```
    digitalWrite(LED_RED, HIGH);
  }
}

// ======================================================
// LCD DISPLAY
// ======================================================

void updateLCD()
{
  lcd.clear();

  // ==================================================
  // PAGE 1
  // CELL VOLTAGES
  // ==================================================

  if (currentPage == 0)
  {
    lcd.setCursor(0, 0);

    lcd.print("C1:");
    lcd.print(cellVoltage[0], 2);

    lcd.print(" C2:");
    lcd.print(cellVoltage[1], 2);

    lcd.setCursor(0, 1);

    lcd.print("C3:");
    lcd.print(cellVoltage[2], 2);

    lcd.print(" C4:");
    lcd.print(cellVoltage[3], 2);
  }

  // ==================================================
  // PAGE 2
  // PACK INFORMATION
  // ==================================================

  else if (currentPage == 1)
  {
    lcd.setCursor(0, 0);

    lcd.print("PK:");
    lcd.print(packVoltage, 2);

    lcd.print(" AV:");
    lcd.print(averageVoltage, 2);

    lcd.setCursor(0, 1);

    lcd.print("IMB:");
    lcd.print(imbalancePercent, 1);

    lcd.print("% W:C");
    lcd.print(weakestCell + 1);
  }

  // ==================================================
  // PAGE 3
  // HEALTH
  // ==================================================

  else if (currentPage == 2)
  {
    lcd.setCursor(0, 0);

    if (batteryHealth ==
      "HEALTHY")
```

```
 {
  lcd.print("HEALTH: HEALTHY");
 }

 else if (batteryHealth ==
       "MINOR IMBALANCE")
 {
  lcd.print("HEALTH: MINOR");
 }

 else if (batteryHealth ==
       "CRITICAL IMBALANCE")
 {
  lcd.print("HEALTH: CRITICAL");
 }

 else
 {
  lcd.print("HEALTH: FAILURE");
 }

 lcd.setCursor(0, 1);

 lcd.print("W:C");
 lcd.print(weakestCell + 1);

 lcd.print(" S:C");
 lcd.print(strongestCell + 1);
}

// ================================================
// PAGE 4
// SAFETY STATUS
// ================================================

else if (currentPage == 3)
{
 lcd.setCursor(0, 0);

 lcd.print("SAFETY:");

 if (safetyState ==
    SAFETY_NORMAL)
 {
  lcd.print("NORMAL");
 }

 else if (safetyState ==
       SAFETY_WARNING)
 {
  lcd.print("WARNING");
 }

 else if (safetyState ==
       SAFETY_TRIPPED)
 {
  lcd.print("TRIPPED");
 }

 else
 {
  lcd.print("RECOVERY");
 }

 lcd.setCursor(0, 1);

 lcd.print("RELAY:");

 if (relayState)
  lcd.print("ON ");
 else
```

```
    lcd.print("OFF");

   lcd.print(" B:");

   if (safetyState ==
      SAFETY_WARNING ||
      safetyState ==
      SAFETY_TRIPPED)
    lcd.print("ON");
   else
    lcd.print("OFF");
  }

  // ================================================
  // PAGE 5
  // FAULT
  // ================================================

  else if (currentPage == 4)
  {
   lcd.setCursor(0, 0);

   if (underVoltageFault)
   {
    lcd.print("FAULT: UNDER V");
   }

   else if (overVoltageFault)
   {
    lcd.print("FAULT: OVER V");
   }

   else if (rapidChangeFault)
   {
    lcd.print("FAULT: RAPID DV");
   }

   else if (sensorFault)
   {
    lcd.print("FAULT: SENSOR");
   }

   else
   {
    lcd.print("NO ACTIVE FAULT");
   }

   lcd.setCursor(0, 1);

   if (faultConfirmed)
   {
    lcd.print("FAULT CONFIRMED");
   }

   else
   {
    lcd.print("MONITORING...");
   }
  }
}

// ================================================
// SERIAL REPORT
// ================================================

void displaySerialReport()
{
 Serial.println();

 Serial.println(
  "================================================"
```

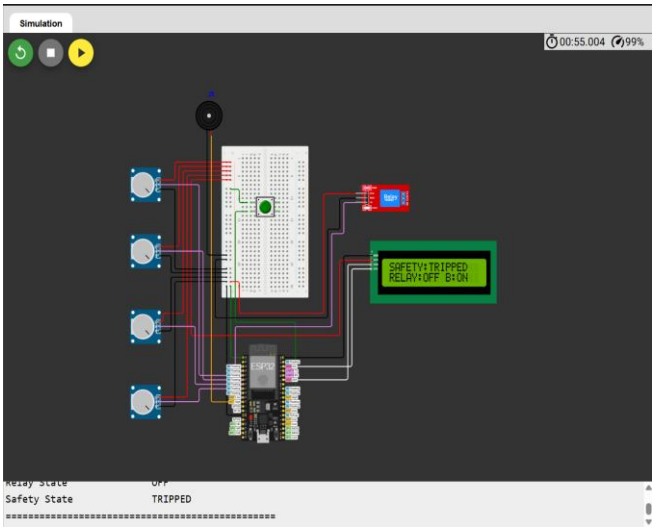
```
);

Serial.println(
  "      EV BMS TASK 1 + TASK 2"
);

Serial.println(
  "================================================"
);

Serial.println(
  "Parameter              Value"
);

Serial.println(
  "------------------------------------------------"
);

// Cell voltages

Serial.print("Cell 1 Voltage          ");

Serial.print(cellVoltage[0], 2);

Serial.println(" V");

Serial.print("Cell 2 Voltage          ");

Serial.print(cellVoltage[1], 2);

Serial.println(" V");

Serial.print("Cell 3 Voltage          ");

Serial.print(cellVoltage[2], 2);

Serial.println(" V");

Serial.print("Cell 4 Voltage          ");

Serial.print(cellVoltage[3], 2);

Serial.println(" V");

Serial.println(
  "------------------------------------------------"
);

// Battery parameters

Serial.print("Pack Voltage            ");

Serial.print(packVoltage, 2);

Serial.println(" V");

Serial.print("Average Voltage         ");

Serial.print(averageVoltage, 2);

Serial.println(" V");

Serial.print("Maximum Voltage         ");

Serial.print(maxVoltage, 2);

Serial.println(" V");

Serial.print("Minimum Voltage         ");

Serial.print(minVoltage, 2);
```

```
Serial.println(" V");

Serial.print("Imbalance Voltage       ");

Serial.print(imbalanceVoltage, 2);

Serial.println(" V");

Serial.print("Imbalance Percentage     ");

Serial.print(imbalancePercent, 2);

Serial.println(" %");

Serial.print("Strongest Cell         Cell ");

Serial.println(strongestCell + 1);

Serial.print("Weakest Cell           Cell ");

Serial.println(weakestCell + 1);

Serial.print("Battery Health         ");

Serial.println(batteryHealth);

Serial.println(
  "------------------------------------------------"
);

// Safety information

Serial.print("Under Voltage Fault     ");

Serial.println(
  underVoltageFault ? "YES" : "NO"
);

Serial.print("Over Voltage Fault      ");

Serial.println(
  overVoltageFault ? "YES" : "NO"
);

Serial.print("Rapid Change Fault      ");

Serial.println(
  rapidChangeFault ? "YES" : "NO"
);

Serial.print("Sensor Fault           ");

Serial.println(
  sensorFault ? "YES" : "NO"
);

Serial.print("Fault Confirmed         ");

Serial.println(
  faultConfirmed ? "YES" : "NO"
);

Serial.print("Relay State            ");

Serial.println(
  relayState ? "ON" : "OFF"
);

Serial.print("Safety State           ");
```

```
 if (safetyState ==
    SAFETY_NORMAL)
 {
  Serial.println("NORMAL");
 }

 else if (safetyState ==
       SAFETY_WARNING)
 {
  Serial.println("WARNING");
 }

 else if (safetyState ==
       SAFETY_TRIPPED)
 {
  Serial.println("TRIPPED");
 }

 else
 {
  Serial.println("RECOVERY");
 }

 Serial.println(
  "================================================"
 );
}
```

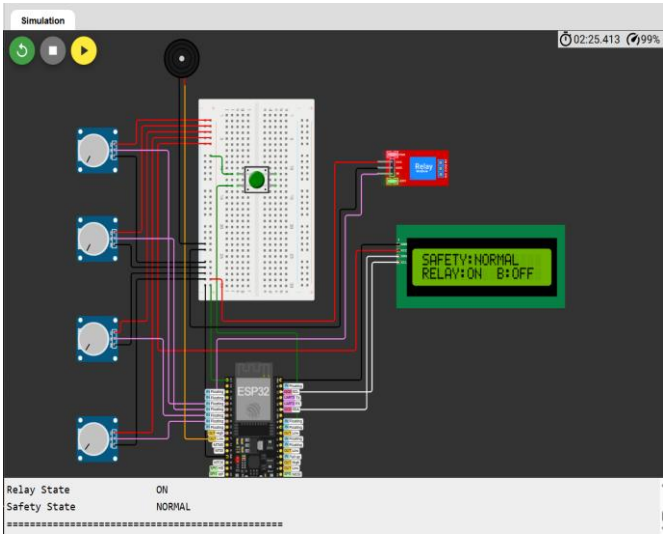
9.5 Simulation Running Photos with correct Output:

[Relay State = off; Safety State = Tripped]




```
================================================
       EV BMS TASK 1 + TASK 2
================================================
Parameter                 Value
------------------------------------------------
Cell 1 Voltage            2.50 V
Cell 2 Voltage            2.50 V
Cell 3 Voltage            2.50 V
Cell 4 Voltage            2.50 V
------------------------------------------------
Pack Voltage              10.00 V
Average Voltage           2.50 V
Maximum Voltage           2.50 V
Minimum Voltage           2.50 V
Imbalance Voltage         0.00 V
Imbalance Percentage      0.00 %
Strongest Cell            Cell 1
Weakest Cell              Cell 1
Battery Health            HEALTHY
------------------------------------------------
Under Voltage Fault       YES
Over Voltage Fault        NO
Rapid Change Fault        NO
Sensor Fault              NO
Fault Confirmed           YES
Relay State               OFF
Safety State              TRIPPED
================================================
```

[Relay State = On; Safety State = Normal]




```
================================================
       EV BMS TASK 1 + TASK 2
================================================
Parameter                 Value
------------------------------------------------
Cell 1 Voltage            2.96 V
Cell 2 Voltage            2.96 V
Cell 3 Voltage            3.00 V
Cell 4 Voltage            3.02 V
------------------------------------------------
Pack Voltage              11.94 V
Average Voltage           2.98 V
Maximum Voltage           3.02 V
Minimum Voltage           2.96 V
Imbalance Voltage         0.06 V
Imbalance Percentage      2.06 %
Strongest Cell            Cell 4
Weakest Cell              Cell 1
Battery Health            MINOR IMBALANCE
------------------------------------------------
Under Voltage Fault       NO
Over Voltage Fault        NO
Rapid Change Fault        NO
Sensor Fault              NO
Fault Confirmed           NO
Relay State               ON
Safety State              NORMAL
================================================
```

9.6 Important Notes:

Test	Input Condition	Expected Result
1. Normal	All cells ≈ 3.8 V	NORMAL, Relay ON
2. Under-Voltage	Any cell < 2.8 V	Relay OFF + Buzzer ON
3. Over-Voltage	Any cell > 4.15 V	Relay OFF + Buzzer ON
4. Rapid Change	Voltage change > 0.10 V/sample	Fault detected
5. Sensor Fault	Invalid/out-of-range reading	Safe state + Warning
6. Recovery	Restore normal voltage	Recovery → Normal → Relay ON
7. Relay Chatter	Repeated fault/normal transition	Relay switching restricted

9.7 ink of Simulation (Wokwi):

Task 2 Click Here

9.8 Task 2 Video Demonstration Link:

Click Here

10. Task 3 : Intelligent Embedded HMI & Diagnostic Interface

10.1 Task 3 Problem

Intelligent Embedded HMI & Diagnostic Interface: In this task, participants must build a professional Human-Machine Interface (HMI) for the embedded system using an LCD display. The interface should automatically rotate between multiple diagnostic screens displaying live battery data, analytics, protection status, and fault diagnostics. The system must support smooth, flicker-free rendering, optimized LCD refresh handling, and automatic fault-priority screen overrides during critical conditions.

10.2 Code :

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ========================================================
// EV BATTERY MANAGEMENT SYSTEM
// TASK 1 + TASK 2 + TASK 3
//
// TASK 1:
// Adaptive Multi-Cell Battery Intelligence Engine
//
// TASK 2:
// Event-Driven Safety Protection Kernel
//
// TASK 3:
// Intelligent Embedded HMI & Diagnostic Interface
//
// Hardware:
// ESP32 DevKit V1
// 4 x Potentiometers = simulated cells
// 16x2 I2C LCD
// 4 x Status LEDs
// Relay Module
// Buzzer
//
// Cell 1 -> GPIO34
// Cell 2 -> GPIO35
// Cell 3 -> GPIO32
// Cell 4 -> GPIO33
//
// LCD SDA -> GPIO21
// LCD SCL -> GPIO22
//
// Green LED  -> GPIO4
```

```
// Yellow LED -> GPIO2
// Orange LED -> GPIO15
// Red LED    -> GPIO5
//
// Relay -> GPIO26
// Buzzer -> GPIO27
// ================================================

// ================================================
// 1. PIN CONFIGURATION
// ================================================

#define CELL1_PIN 34
#define CELL2_PIN 35
#define CELL3_PIN 32
#define CELL4_PIN 33

#define SDA_PIN 21
#define SCL_PIN 22

#define LED_GREEN  4
#define LED_YELLOW 2
#define LED_ORANGE 15
#define LED_RED    5

#define RELAY_PIN  26
#define BUZZER_PIN 27

// ================================================
// 2. LCD CONFIGURATION
// ================================================

LiquidCrystal_I2C lcd(0x27, 16, 2);

// ================================================
// 3. BATTERY CONFIGURATION
// ================================================

#define NUM_CELLS 4

const int ADC_MAX = 4095;

const float ADC_MAX_VOLTAGE = 3.3;

const float CELL_MIN_VOLTAGE = 2.5;
const float CELL_MAX_VOLTAGE = 4.2;

// ================================================
// 4. TASK 1 HEALTH THRESHOLDS
// ================================================

const float HEALTHY_LIMIT = 2.0;

const float MINOR_LIMIT = 5.0;

const float CRITICAL_LIMIT = 10.0;

// ================================================
// 5. TASK 2 SAFETY THRESHOLDS
// ================================================

const float UV_THRESHOLD = 2.8;

const float OV_THRESHOLD = 4.15;

const float RAPID_CHANGE_THRESHOLD = 0.10;
```

```
// ====================================================
// 6. TIMING
// ====================================================

const unsigned long SAMPLE_INTERVAL = 200;

const unsigned long LCD_INTERVAL = 3000;

const unsigned long SERIAL_INTERVAL = 2000;

const unsigned long FAULT_CONFIRM_TIME = 500;

const unsigned long RECOVERY_TIME = 5000;

const unsigned long RELAY_LOCK_TIME = 3000;

const unsigned long BUZZER_INTERVAL = 300;

// ====================================================
// 7. TIMERS
// ====================================================

unsigned long previousSampleTime = 0;

unsigned long previousLCDTime = 0;

unsigned long previousSerialTime = 0;

unsigned long faultStartTime = 0;

unsigned long recoveryStartTime = 0;

unsigned long lastRelayChangeTime = 0;

unsigned long previousBuzzerTime = 0;

// ====================================================
// 8. BATTERY DATA
// ====================================================

float cellVoltage[NUM_CELLS];

float previousCellVoltage[NUM_CELLS];

float packVoltage = 0.0;

float averageVoltage = 0.0;

float maxVoltage = 0.0;

float minVoltage = 0.0;

float imbalanceVoltage = 0.0;

float imbalancePercent = 0.0;

int strongestCell = 0;

int weakestCell = 0;

String batteryHealth;

// ====================================================
// 9. SAFETY STATE MACHINE
// ====================================================
```

```
enum SafetyState
{
  SAFETY_NORMAL,
  SAFETY_WARNING,
  SAFETY_TRIPPED,
  SAFETY_RECOVERY
};

SafetyState safetyState = SAFETY_NORMAL;

// ==================================================
// 10. FAULT TYPES
// ==================================================

bool underVoltageFault = false;

bool overVoltageFault = false;

bool rapidChangeFault = false;

bool sensorFault = false;

bool faultDetected = false;

bool faultConfirmed = false;

// ==================================================
// 11. RELAY STATE
// ==================================================

bool relayState = true;

// ==================================================
// 12. TASK 3 HMI
// ==================================================

int currentPage = 0;

const int TOTAL_PAGES = 5;

// Stores previous page
int previousPage = -1;

// Prevents continuous fault-screen redraw
bool faultScreenShown = false;

// ==================================================
// SETUP
// ==================================================

void setup()
{
  Serial.begin(115200);

  // ---------- ADC ----------

  analogReadResolution(12);

  pinMode(CELL1_PIN, INPUT);
  pinMode(CELL2_PIN, INPUT);
  pinMode(CELL3_PIN, INPUT);
  pinMode(CELL4_PIN, INPUT);

  // ---------- LEDs ----------
```

```
pinMode(LED_GREEN, OUTPUT);
pinMode(LED_YELLOW, OUTPUT);
pinMode(LED_ORANGE, OUTPUT);
pinMode(LED_RED, OUTPUT);

// ---------- Relay ----------

pinMode(RELAY_PIN, OUTPUT);

digitalWrite(RELAY_PIN, HIGH);

relayState = true;

// ---------- Buzzer ----------

pinMode(BUZZER_PIN, OUTPUT);

digitalWrite(BUZZER_PIN, LOW);

// ---------- I2C ----------

Wire.begin(SDA_PIN, SCL_PIN);

// ---------- LCD ----------

lcd.init();

lcd.backlight();

lcd.clear();

lcd.setCursor(0, 0);

lcd.print("EV BMS TASK 3");

lcd.setCursor(0, 1);

lcd.print("Smart HMI");

// Startup delay only
// Not used in safety/HMI operation
delay(1500);

// ---------- Initial reading ----------

readCellVoltages();

calculateBatteryParameters();

identifyStrongestWeakest();

calculateImbalance();

classifyBatteryHealth();

// Store initial voltage

for (int i = 0; i < NUM_CELLS; i++)
{
  previousCellVoltage[i] = cellVoltage[i];
}

updateLEDs();

updateLCD();
```

```
  displaySerialReport();

  Serial.println();

  Serial.println("=====================================");

  Serial.println("TASK 3 HMI READY");

  Serial.println("=====================================");
}

// ====================================================
// MAIN LOOP
// ====================================================

void loop()
{
  unsigned long currentTime = millis();

  // ================================================
  // 1. BATTERY + SAFETY PROCESSING
  // ================================================

  if (currentTime - previousSampleTime >= SAMPLE_INTERVAL)
  {
    previousSampleTime = currentTime;

    // ---------- TASK 1 ----------

    readCellVoltages();

    calculateBatteryParameters();

    identifyStrongestWeakest();

    calculateImbalance();

    classifyBatteryHealth();

    // ---------- TASK 2 ----------

    detectSafetyConditions();

    updateSafetyState();

    updateRelay();

    updateBuzzer();

    updateLEDs();
  }

  // ================================================
  // 2. TASK 3 HMI
  // ================================================

  if (currentTime - previousLCDTime >= LCD_INTERVAL)
  {
    previousLCDTime = currentTime;

    // Rotate only when there is NO fault

    if (!faultDetected)
    {
      currentPage++;
```

```
    if (currentPage >= TOTAL_PAGES)
    {
      currentPage = 0;
    }
   }

   updateLCD();
  }

  // ================================================
  // 3. SERIAL REPORT
  // ================================================

  if (currentTime - previousSerialTime >= SERIAL_INTERVAL)
  {
   previousSerialTime = currentTime;

   displaySerialReport();
  }
}

// ================================================
// TASK 1
// READ FOUR CELL VOLTAGES
// ================================================

void readCellVoltages()
{
 int rawCell1 = analogRead(CELL1_PIN);

 int rawCell2 = analogRead(CELL2_PIN);

 int rawCell3 = analogRead(CELL3_PIN);

 int rawCell4 = analogRead(CELL4_PIN);

 cellVoltage[0] =
  convertADCToCellVoltage(rawCell1);

 cellVoltage[1] =
  convertADCToCellVoltage(rawCell2);

 cellVoltage[2] =
  convertADCToCellVoltage(rawCell3);

 cellVoltage[3] =
  convertADCToCellVoltage(rawCell4);
}

// ================================================
// TASK 1
// ADC TO SIMULATED CELL VOLTAGE
// ================================================

float convertADCToCellVoltage(int rawADC)
{
 float adcVoltage;

 adcVoltage =
  ((float)rawADC / ADC_MAX)
  * ADC_MAX_VOLTAGE;

 float simulatedVoltage;

 simulatedVoltage =
  CELL_MIN_VOLTAGE
```

```
    +
    (adcVoltage / ADC_MAX_VOLTAGE)
    *
    (CELL_MAX_VOLTAGE -
     CELL_MIN_VOLTAGE);

  return simulatedVoltage;
}

// ====================================================
// TASK 1
// CALCULATE BATTERY PARAMETERS
// ====================================================

void calculateBatteryParameters()
{
  packVoltage = 0.0;

  for (int i = 0; i < NUM_CELLS; i++)
  {
    packVoltage += cellVoltage[i];
  }

  averageVoltage =
    packVoltage / NUM_CELLS;

  maxVoltage = cellVoltage[0];

  minVoltage = cellVoltage[0];

  for (int i = 1; i < NUM_CELLS; i++)
  {
    if (cellVoltage[i] > maxVoltage)
    {
      maxVoltage = cellVoltage[i];
    }

    if (cellVoltage[i] < minVoltage)
    {
      minVoltage = cellVoltage[i];
    }
  }
}

// ====================================================
// TASK 1
// IDENTIFY STRONGEST / WEAKEST CELL
// ====================================================

void identifyStrongestWeakest()
{
  strongestCell = 0;

  weakestCell = 0;

  for (int i = 1; i < NUM_CELLS; i++)
  {
    if (cellVoltage[i] >
        cellVoltage[strongestCell])
    {
      strongestCell = i;
    }

    if (cellVoltage[i] <
        cellVoltage[weakestCell])
    {
```

```
      weakestCell = i;
    }
  }
}

// ====================================================
// TASK 1
// CALCULATE IMBALANCE
// ====================================================

void calculateImbalance()
{
  imbalanceVoltage =
    maxVoltage - minVoltage;

  if (averageVoltage > 0)
  {
    imbalancePercent =
      (imbalanceVoltage /
       averageVoltage)
      * 100.0;
  }
  else
  {
    imbalancePercent = 0.0;
  }
}

// ====================================================
// TASK 1
// HEALTH CLASSIFICATION
// ====================================================

void classifyBatteryHealth()
{
  if (imbalancePercent < HEALTHY_LIMIT)
  {
    batteryHealth = "HEALTHY";
  }

  else if (imbalancePercent < MINOR_LIMIT)
  {
    batteryHealth = "MINOR IMBALANCE";
  }

  else if (imbalancePercent < CRITICAL_LIMIT)
  {
    batteryHealth = "CRITICAL IMBALANCE";
  }

  else
  {
    batteryHealth = "PACK FAILURE";
  }
}

// ====================================================
// TASK 2
// DETECT SAFETY CONDITIONS
// ====================================================

void detectSafetyConditions()
{
  underVoltageFault = false;

  overVoltageFault = false;
```

```
 rapidChangeFault = false;

 sensorFault = false;

 for (int i = 0; i < NUM_CELLS; i++)
 {
  // Sensor anomaly

  if (cellVoltage[i] < CELL_MIN_VOLTAGE ||
     cellVoltage[i] > CELL_MAX_VOLTAGE)
  {
   sensorFault = true;
  }

  // Under-voltage

  if (cellVoltage[i] < UV_THRESHOLD)
  {
   underVoltageFault = true;
  }

  // Over-voltage

  if (cellVoltage[i] > OV_THRESHOLD)
  {
   overVoltageFault = true;
  }

  // Rapid voltage change

  float voltageChange =
   fabs(cellVoltage[i] -
        previousCellVoltage[i]);

  if (voltageChange >
     RAPID_CHANGE_THRESHOLD)
  {
   rapidChangeFault = true;
  }
 }

 // Store current readings

 for (int i = 0; i < NUM_CELLS; i++)
 {
  previousCellVoltage[i] =
   cellVoltage[i];
 }

 // Overall fault

 faultDetected =
  underVoltageFault ||
  overVoltageFault ||
  rapidChangeFault ||
  sensorFault;
}

// ================================================
// TASK 2
// SAFETY STATE MACHINE
// ================================================

void updateSafetyState()
{
```

```
unsigned long now = millis();

switch (safetyState)
{
 // ================================================
 // NORMAL
 // ================================================

 case SAFETY_NORMAL:

  if (faultDetected)
  {
   safetyState = SAFETY_WARNING;

   faultStartTime = now;
  }

  break;

 // ================================================
 // WARNING
 // ================================================

 case SAFETY_WARNING:

  if (!faultDetected)
  {
   safetyState = SAFETY_NORMAL;

   faultConfirmed = false;

   break;
  }

  if (now - faultStartTime >=
     FAULT_CONFIRM_TIME)
  {
   faultConfirmed = true;

   safetyState = SAFETY_TRIPPED;
  }

  break;

 // ================================================
 // TRIPPED
 // ================================================

 case SAFETY_TRIPPED:

  if (!faultDetected)
  {
   recoveryStartTime = now;

   safetyState = SAFETY_RECOVERY;
  }

  break;

 // ================================================
 // RECOVERY
 // ================================================

 case SAFETY_RECOVERY:

  if (faultDetected)
```

```
    {
      safetyState = SAFETY_TRIPPED;

      break;
    }

    if (now - recoveryStartTime >=
        RECOVERY_TIME)
    {
      safetyState = SAFETY_NORMAL;

      faultConfirmed = false;
    }

    break;
  }
}

// ================================================
// TASK 2
// RELAY CONTROL
// ================================================

void updateRelay()
{
  unsigned long now = millis();

  // Relay OFF during warning/tripped

  if (safetyState == SAFETY_TRIPPED ||
     safetyState == SAFETY_WARNING)
  {
    if (relayState == true)
    {
      if (now - lastRelayChangeTime >=
          RELAY_LOCK_TIME)
      {
        digitalWrite(RELAY_PIN, LOW);

        relayState = false;

        lastRelayChangeTime = now;

        Serial.println(
          ">>> RELAY CUT-OFF <<<"
        );
      }
    }
  }

  // Relay ON during normal

  else if (safetyState ==
          SAFETY_NORMAL)
  {
    if (relayState == false)
    {
      if (now - lastRelayChangeTime >=
          RELAY_LOCK_TIME)
      {
        digitalWrite(RELAY_PIN, HIGH);

        relayState = true;

        lastRelayChangeTime = now;
```

```
      Serial.println(
        ">>> RELAY RESTORED <<<"
      );
     }
    }
   }
}

// ====================================================
// TASK 2
// BUZZER CONTROL
// ====================================================

void updateBuzzer()
{
 unsigned long now = millis();

 if (safetyState ==
    SAFETY_WARNING ||
    safetyState ==
    SAFETY_TRIPPED)
 {
  if (now - previousBuzzerTime >=
     BUZZER_INTERVAL)
  {
    previousBuzzerTime = now;

    digitalWrite(
      BUZZER_PIN,
      !digitalRead(BUZZER_PIN)
    );
  }
 }

 else
 {
  digitalWrite(BUZZER_PIN, LOW);
 }
}

// ====================================================
// LED HEALTH INDICATION
// ====================================================

void updateLEDs()
{
 digitalWrite(LED_GREEN, LOW);

 digitalWrite(LED_YELLOW, LOW);

 digitalWrite(LED_ORANGE, LOW);

 digitalWrite(LED_RED, LOW);

 // Safety fault gets priority

 if (safetyState ==
    SAFETY_WARNING ||
    safetyState ==
    SAFETY_TRIPPED)
 {
  digitalWrite(LED_RED, HIGH);

  return;
 }
```

```
 // Battery health indication

 if (batteryHealth == "HEALTHY")
 {
  digitalWrite(LED_GREEN, HIGH);
 }

 else if (batteryHealth ==
       "MINOR IMBALANCE")
 {
  digitalWrite(LED_YELLOW, HIGH);
 }

 else if (batteryHealth ==
       "CRITICAL IMBALANCE")
 {
  digitalWrite(LED_ORANGE, HIGH);
 }

 else
 {
  digitalWrite(LED_RED, HIGH);
 }
}

// ================================================
// TASK 3
// LCD HMI MANAGER
// ================================================

void updateLCD()
{
 // ================================================
 // FAULT PRIORITY
 // ================================================

 if (faultDetected)
 {
  if (!faultScreenShown)
  {
   displayFaultPriority();

   faultScreenShown = true;
  }

  return;
 }

 // ================================================
 // FAULT CLEARED
 // ================================================

 if (faultScreenShown)
 {
  lcd.clear();

  faultScreenShown = false;

  previousPage = -1;
 }

 // ================================================
 // NORMAL HMI
 // ================================================

 // Only redraw when page changes
```

```
if (currentPage == previousPage)
{
  return;
}

previousPage = currentPage;

lcd.clear();

// ================================================
// PAGE 1
// LIVE CELL DATA
// ================================================

if (currentPage == 0)
{
  lcd.setCursor(0, 0);

  lcd.print("C1:");
  lcd.print(cellVoltage[0], 2);

  lcd.print(" C2:");
  lcd.print(cellVoltage[1], 2);

  lcd.setCursor(0, 1);

  lcd.print("C3:");
  lcd.print(cellVoltage[2], 2);

  lcd.print(" C4:");
  lcd.print(cellVoltage[3], 2);
}

// ================================================
// PAGE 2
// PACK ANALYTICS
// ================================================

else if (currentPage == 1)
{
  lcd.setCursor(0, 0);

  lcd.print("PACK:");
  lcd.print(packVoltage, 2);

  lcd.print("V");

  lcd.setCursor(0, 1);

  lcd.print("AVG:");
  lcd.print(averageVoltage, 2);

  lcd.print("V");
}

// ================================================
// PAGE 3
// CELL ANALYTICS
// ================================================

else if (currentPage == 2)
{
  lcd.setCursor(0, 0);

  lcd.print("IMB:");
```

```
  lcd.print(imbalancePercent, 1);

  lcd.print("%");

  lcd.setCursor(0, 1);

  lcd.print("W:C");
  lcd.print(weakestCell + 1);

  lcd.print(" S:C");
  lcd.print(strongestCell + 1);
 }

 // ================================================
 // PAGE 4
 // PROTECTION STATUS
 // ================================================

 else if (currentPage == 3)
 {
  lcd.setCursor(0, 0);

  lcd.print("SAFETY:");

  if (safetyState ==
     SAFETY_NORMAL)
  {
   lcd.print("NORMAL");
  }

  else if (safetyState ==
         SAFETY_WARNING)
  {
   lcd.print("WARNING");
  }

  else if (safetyState ==
         SAFETY_TRIPPED)
  {
   lcd.print("TRIPPED");
  }

  else
  {
   lcd.print("RECOVERY");
  }

  lcd.setCursor(0, 1);

  lcd.print("RELAY:");

  if (relayState)
   lcd.print("ON ");
  else
   lcd.print("OFF");

  lcd.print(" B:");

  if (safetyState ==
     SAFETY_WARNING ||
     safetyState ==
     SAFETY_TRIPPED)
  {
   lcd.print("ON");
  }
  else
```

```
    {
      lcd.print("OFF");
    }
  }

  // ================================================
  // PAGE 5
  // DIAGNOSTICS
  // ================================================

  else if (currentPage == 4)
  {
    lcd.setCursor(0, 0);

    lcd.print("DIAGNOSTICS");

    lcd.setCursor(0, 1);

    if (!faultDetected)
    {
      lcd.print("SYSTEM OK");
    }
    else
    {
      lcd.print("CHECK FAULT");
    }
  }
}

// ================================================
// TASK 3
// FAULT PRIORITY DISPLAY
// ================================================

void displayFaultPriority()
{
  lcd.clear();

  // ================================================
  // UNDER VOLTAGE
  // ================================================

  if (underVoltageFault)
  {
    lcd.setCursor(0, 0);

    lcd.print("FAULT: UNDER V");

    lcd.setCursor(0, 1);

    lcd.print("C");

    lcd.print(weakestCell + 1);

    lcd.print(":");

    lcd.print(cellVoltage[weakestCell], 2);

    lcd.print("V");
  }

  // ================================================
  // OVER VOLTAGE
  // ================================================

  else if (overVoltageFault)
```

```
  {
    lcd.setCursor(0, 0);

    lcd.print("FAULT: OVER V");

    lcd.setCursor(0, 1);

    lcd.print("CHECK CELL");
  }

  // ================================================
  // RAPID VOLTAGE CHANGE
  // ================================================

  else if (rapidChangeFault)
  {
    lcd.setCursor(0, 0);

    lcd.print("FAULT: RAPID DV");

    lcd.setCursor(0, 1);

    lcd.print("CHECK BATTERY");
  }

  // ================================================
  // SENSOR FAULT
  // ================================================

  else if (sensorFault)
  {
    lcd.setCursor(0, 0);

    lcd.print("FAULT: SENSOR");

    lcd.setCursor(0, 1);

    lcd.print("SAFE MODE");
  }
}

// ====================================================
// SERIAL REPORT
// ====================================================

void displaySerialReport()
{
  Serial.println();

  Serial.println(
    "================================================"
  );

  Serial.println(
    "      EV BMS TASK 1 + 2 + 3"
  );

  Serial.println(
    "================================================"
  );

  Serial.println(
    "Parameter              Value"
  );

  Serial.println(
```

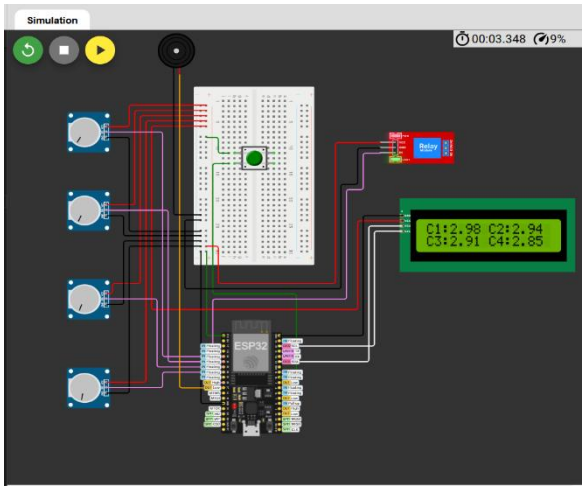
```
  "------------------------------------------------"
);

// Cell voltages

Serial.print("Cell 1 Voltage          ");

Serial.print(cellVoltage[0], 2);

Serial.println(" V");

Serial.print("Cell 2 Voltage          ");

Serial.print(cellVoltage[1], 2);

Serial.println(" V");

Serial.print("Cell 3 Voltage          ");

Serial.print(cellVoltage[2], 2);

Serial.println(" V");

Serial.print("Cell 4 Voltage          ");

Serial.print(cellVoltage[3], 2);

Serial.println(" V");

Serial.println(
  "------------------------------------------------"
);

// Battery parameters

Serial.print("Pack Voltage            ");

Serial.print(packVoltage, 2);

Serial.println(" V");

Serial.print("Average Voltage         ");

Serial.print(averageVoltage, 2);

Serial.println(" V");

Serial.print("Maximum Voltage         ");

Serial.print(maxVoltage, 2);

Serial.println(" V");

Serial.print("Minimum Voltage         ");

Serial.print(minVoltage, 2);

Serial.println(" V");

Serial.print("Imbalance Voltage       ");

Serial.print(imbalanceVoltage, 2);

Serial.println(" V");

Serial.print("Imbalance Percentage    ");
```

```
Serial.print(imbalancePercent, 2);

Serial.println(" %");

Serial.print("Strongest Cell          Cell ");

Serial.println(strongestCell + 1);

Serial.print("Weakest Cell            Cell ");

Serial.println(weakestCell + 1);

Serial.print("Battery Health          ");

Serial.println(batteryHealth);

Serial.println(
  "-------------------------------------------------"
);

// Safety information

Serial.print("Under Voltage Fault     ");

Serial.println(
  underVoltageFault ? "YES" : "NO"
);

Serial.print("Over Voltage Fault      ");

Serial.println(
  overVoltageFault ? "YES" : "NO"
);

Serial.print("Rapid Change Fault      ");

Serial.println(
  rapidChangeFault ? "YES" : "NO"
);

Serial.print("Sensor Fault            ");

Serial.println(
  sensorFault ? "YES" : "NO"
);

Serial.print("Fault Confirmed         ");

Serial.println(
  faultConfirmed ? "YES" : "NO"
);

Serial.print("Relay State             ");

Serial.println(
  relayState ? "ON" : "OFF"
);

Serial.print("Safety State            ");

if (safetyState ==
    SAFETY_NORMAL)
{
  Serial.println("NORMAL");
}
```

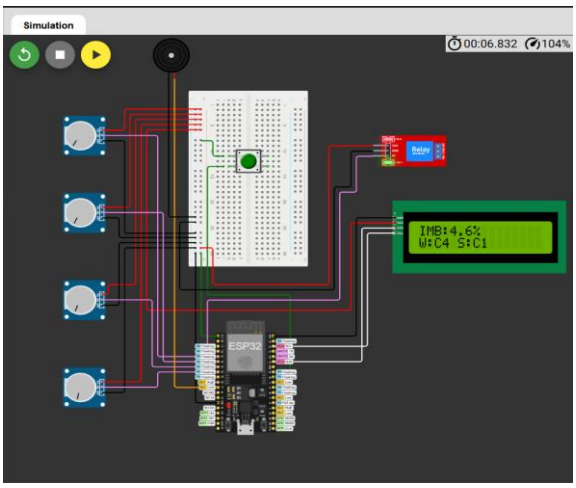
```
 else if (safetyState ==
       SAFETY_WARNING)
 {
  Serial.println("WARNING");
 }

 else if (safetyState ==
       SAFETY_TRIPPED)
 {
  Serial.println("TRIPPED");
 }

 else
 {
  Serial.println("RECOVERY");
 }

 Serial.println(
  "================================================"
 );
}
```

10.3 Simulation Running Photos with correct Output:

1. Live Cell Data

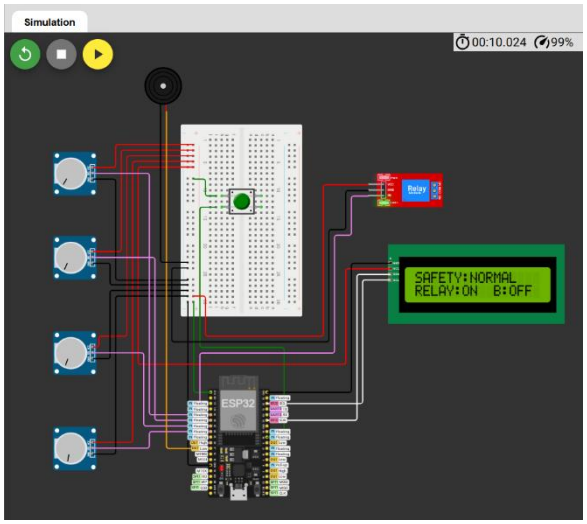


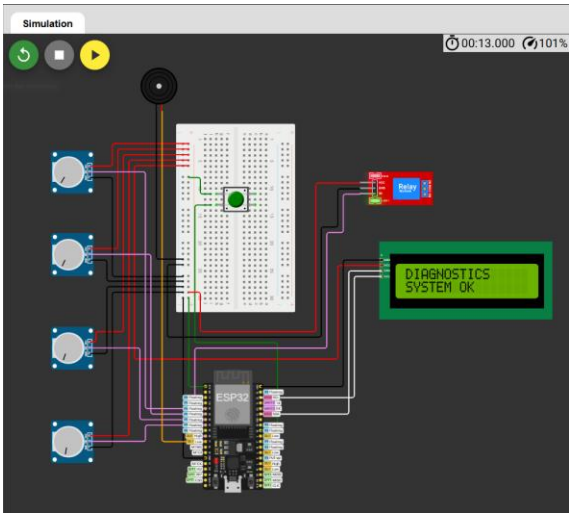
2. Cell Analytics



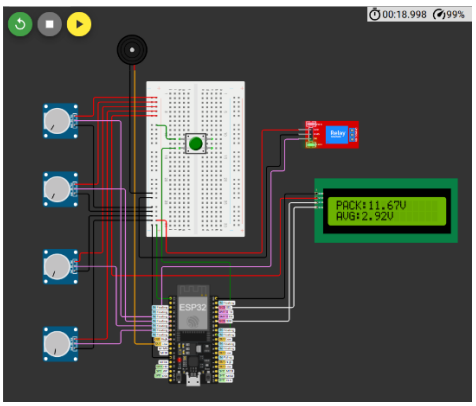
3. Protection :

4. Diagnostics :

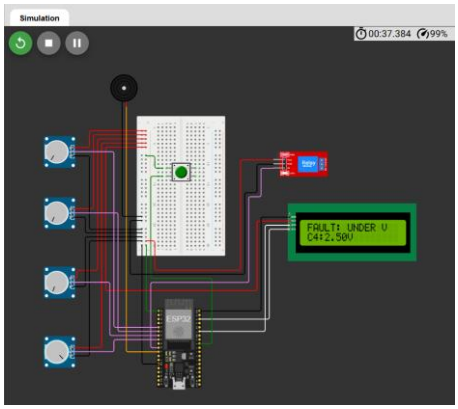




5. Battery Analytics



Fault Comes



10.4 Task 3 Wokwi Link :

Task 3 Click Here

10.5 Task 3 Video Demonstration Link:

Click Here

11. Task 4 : Fault-Tolerant Embedded Runtime System

11.1 Task 4 Problem

Fault-Tolerant Embedded Runtime System: This task requires candidates to create a fault-tolerant runtime subsystem capable of detecting and handling hardware and software failures autonomously. The implementation should identify issues such as sensor disconnection, invalid readings, frozen ADC conditions, and relay mismatches while transitioning between operational modes like NORMAL, DEGRADED, FAILSAFE, and SHUTDOWN. The system must isolate faults without affecting healthy modules and maintain timestamped fault logs with structured recovery mechanisms.

11.2 Code:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ====================================================
// EV BATTERY MANAGEMENT SYSTEM
// TASK 1 + TASK 2 + TASK 3 + TASK 4
//
// TASK 1:
// Adaptive Multi-Cell Battery Intelligence Engine
//
// TASK 2:
// Event-Driven Safety Protection Kernel
//
// TASK 3:
// Intelligent Embedded HMI & Diagnostic Interface
//
// TASK 4:
// Fault-Tolerant Embedded Runtime System
// ====================================================

// ====================================================
// 1. PIN CONFIGURATION
// ====================================================

#define CELL1_PIN 34
#define CELL2_PIN 35
#define CELL3_PIN 32
#define CELL4_PIN 33

#define SDA_PIN 21
#define SCL_PIN 22

#define LED_GREEN  4
#define LED_YELLOW 2
#define LED_ORANGE 15
#define LED_RED    5

#define RELAY_PIN  26
#define BUZZER_PIN 27
```

```
// ====================================================
// 2. LCD
// ====================================================

LiquidCrystal_I2C lcd(0x27, 16, 2);

// ====================================================
// 3. BATTERY CONFIGURATION
// ====================================================

#define NUM_CELLS 4

const int ADC_MAX = 4095;

const float ADC_MAX_VOLTAGE = 3.3;

const float CELL_MIN_VOLTAGE = 2.5;
const float CELL_MAX_VOLTAGE = 4.2;

// ====================================================
// 4. TASK 1 HEALTH THRESHOLDS
// ====================================================

const float HEALTHY_LIMIT = 2.0;
const float MINOR_LIMIT = 5.0;
const float CRITICAL_LIMIT = 10.0;

// ====================================================
// 5. TASK 2 SAFETY THRESHOLDS
// ====================================================

const float UV_THRESHOLD = 2.8;
const float OV_THRESHOLD = 4.15;
const float RAPID_CHANGE_THRESHOLD = 0.10;

// ====================================================
// 6. TIMING
// ====================================================

const unsigned long SAMPLE_INTERVAL = 200;
const unsigned long LCD_INTERVAL = 3000;
const unsigned long SERIAL_INTERVAL = 2000;

const unsigned long FAULT_CONFIRM_TIME = 500;
const unsigned long RECOVERY_TIME = 5000;
const unsigned long RELAY_LOCK_TIME = 3000;
const unsigned long BUZZER_INTERVAL = 300;

// ====================================================
// TASK 4 TIMING
// ====================================================

// Same ADC value for this many samples
// = frozen ADC suspicion

const int FROZEN_LIMIT = 10;

// Time after which a degraded condition
// can return to NORMAL

const unsigned long DEGRADED_RECOVERY_TIME = 3000;

// ====================================================
// 7. TIMERS
// ====================================================
```

```
unsigned long previousSampleTime = 0;
unsigned long previousLCDTime = 0;
unsigned long previousSerialTime = 0;

unsigned long faultStartTime = 0;
unsigned long recoveryStartTime = 0;

unsigned long lastRelayChangeTime = 0;
unsigned long previousBuzzerTime = 0;

unsigned long degradedStartTime = 0;

// ====================================================
// 8. BATTERY DATA
// ====================================================

float cellVoltage[NUM_CELLS];

float previousCellVoltage[NUM_CELLS];

int cellADC[NUM_CELLS];

int previousADC[NUM_CELLS];

int frozenCount[NUM_CELLS];

float packVoltage = 0.0;
float averageVoltage = 0.0;

float maxVoltage = 0.0;
float minVoltage = 0.0;

float imbalanceVoltage = 0.0;
float imbalancePercent = 0.0;

int strongestCell = 0;
int weakestCell = 0;

String batteryHealth;

// ====================================================
// 9. TASK 2 SAFETY STATE
// ====================================================

enum SafetyState
{
  SAFETY_NORMAL,
  SAFETY_WARNING,
  SAFETY_TRIPPED,
  SAFETY_RECOVERY
};

SafetyState safetyState = SAFETY_NORMAL;

// ====================================================
// 10. TASK 2 FAULT FLAGS
// ====================================================

bool underVoltageFault = false;
bool overVoltageFault = false;
bool rapidChangeFault = false;
bool sensorFault = false;

bool faultDetected = false;
bool faultConfirmed = false;
```

```
// ====================================================
// 11. TASK 4 RUNTIME MODES
// ====================================================

enum RuntimeMode
{
 RUNTIME_NORMAL,
 RUNTIME_DEGRADED,
 RUNTIME_FAILSAFE,
 RUNTIME_SHUTDOWN
};

RuntimeMode runtimeMode = RUNTIME_NORMAL;

// ====================================================
// 12. TASK 4 FAULT TYPES
// ====================================================

bool invalidSensorFault = false;
bool frozenADCFault = false;
bool relayMismatchFault = false;

// ====================================================
// 13. RELAY
// ====================================================

bool relayState = true;

// ====================================================
// 14. TASK 4 SOFTWARE RELAY MISMATCH
// ====================================================

// FALSE = normal operation
// TRUE  = simulate relay mismatch

bool simulateRelayMismatch = false;

// ====================================================
// 15. TASK 3 HMI
// ====================================================

int currentPage = 0;

const int TOTAL_PAGES = 5;

int previousPage = -1;

bool faultScreenShown = false;

// ====================================================
// 16. TASK 4 FAULT LOG
// ====================================================

struct FaultLog
{
 unsigned long timestamp;

 String faultName;

 String mode;
};

#define MAX_FAULT_LOGS 10

FaultLog faultLogs[MAX_FAULT_LOGS];
```

```
int faultLogCount = 0;

// ================================================
// SETUP
// ================================================

void setup()
{
 Serial.begin(115200);

 // ================================================
 // ADC
 // ================================================

 analogReadResolution(12);

 pinMode(CELL1_PIN, INPUT);
 pinMode(CELL2_PIN, INPUT);
 pinMode(CELL3_PIN, INPUT);
 pinMode(CELL4_PIN, INPUT);

 // ================================================
 // LEDs
 // ================================================

 pinMode(LED_GREEN, OUTPUT);
 pinMode(LED_YELLOW, OUTPUT);
 pinMode(LED_ORANGE, OUTPUT);
 pinMode(LED_RED, OUTPUT);

 // ================================================
 // RELAY
 // ================================================

 pinMode(RELAY_PIN, OUTPUT);

 digitalWrite(RELAY_PIN, HIGH);

 relayState = true;

 // ================================================
 // BUZZER
 // ================================================

 pinMode(BUZZER_PIN, OUTPUT);

 digitalWrite(BUZZER_PIN, LOW);

 // ================================================
 // I2C
 // ================================================

 Wire.begin(SDA_PIN, SCL_PIN);

 // ================================================
 // LCD
 // ================================================

 lcd.init();

 lcd.backlight();

 lcd.clear();

 lcd.setCursor(0, 0);
```

```
  lcd.print("EV BMS TASK 4");

  lcd.setCursor(0, 1);
  lcd.print("Fault Runtime");

  // Startup display only
  delay(1500);

  // ================================================
  // INITIAL BATTERY READING
  // ================================================

  readCellVoltages();

  calculateBatteryParameters();

  identifyStrongestWeakest();

  calculateImbalance();

  classifyBatteryHealth();

  // Store initial ADC and voltage values

  for (int i = 0; i < NUM_CELLS; i++)
  {
    previousCellVoltage[i] = cellVoltage[i];

    previousADC[i] = cellADC[i];

    frozenCount[i] = 0;
  }

  updateLEDs();

  updateLCD();

  displaySerialReport();

  Serial.println();
  Serial.println("====================================");
  Serial.println("TASK 4 FAULT-TOLERANT RUNTIME READY");
  Serial.println("====================================");
}

// ======================================================
// MAIN LOOP
// ======================================================

void loop()
{
  unsigned long currentTime = millis();

  // ================================================
  // 1. BATTERY + SAFETY + RUNTIME
  // ================================================

  if (currentTime - previousSampleTime >= SAMPLE_INTERVAL)
  {
    previousSampleTime = currentTime;

    // ---------- TASK 1 ----------

    readCellVoltages();

    calculateBatteryParameters();
```

```
    identifyStrongestWeakest();

    calculateImbalance();

    classifyBatteryHealth();

    // ---------- TASK 2 ----------

    detectSafetyConditions();

    updateSafetyState();

    updateRelay();

    updateBuzzer();

    // ---------- TASK 4 ----------

    checkSensorHealth();

    checkFrozenADC();

    checkRelayMismatch();

    updateRuntimeMode();

    updateLEDs();
  }

  // ================================================
  // 2. HMI
  // ================================================

  if (currentTime - previousLCDTime >= LCD_INTERVAL)
  {
    previousLCDTime = currentTime;

    if (!faultDetected &&
        runtimeMode == RUNTIME_NORMAL)
    {
      currentPage++;

      if (currentPage >= TOTAL_PAGES)
      {
        currentPage = 0;
      }
    }

    updateLCD();
  }

  // ================================================
  // 3. SERIAL
  // ================================================

  if (currentTime - previousSerialTime >= SERIAL_INTERVAL)
  {
    previousSerialTime = currentTime;

    displaySerialReport();
  }
}

// ================================================
// TASK 1
```

```
// READ CELL VOLTAGES
// ======================================================

void readCellVoltages()
{
 cellADC[0] = analogRead(CELL1_PIN);
 cellADC[1] = analogRead(CELL2_PIN);
 cellADC[2] = analogRead(CELL3_PIN);
 cellADC[3] = analogRead(CELL4_PIN);

 for (int i = 0; i < NUM_CELLS; i++)
 {
  cellVoltage[i] =
   convertADCToCellVoltage(cellADC[i]);
 }
}

// ======================================================
// TASK 1
// ADC TO CELL VOLTAGE
// ======================================================

float convertADCToCellVoltage(int rawADC)
{
 float adcVoltage =
  ((float)rawADC / ADC_MAX)
  * ADC_MAX_VOLTAGE;

 float simulatedVoltage =
  CELL_MIN_VOLTAGE
  +
  (adcVoltage / ADC_MAX_VOLTAGE)
  *
  (CELL_MAX_VOLTAGE -
   CELL_MIN_VOLTAGE);

 return simulatedVoltage;
}

// ======================================================
// TASK 1
// BATTERY PARAMETERS
// ======================================================

void calculateBatteryParameters()
{
 packVoltage = 0.0;

 for (int i = 0; i < NUM_CELLS; i++)
 {
  packVoltage += cellVoltage[i];
 }

 averageVoltage =
  packVoltage / NUM_CELLS;

 maxVoltage = cellVoltage[0];

 minVoltage = cellVoltage[0];

 for (int i = 1; i < NUM_CELLS; i++)
 {
  if (cellVoltage[i] > maxVoltage)
  {
   maxVoltage = cellVoltage[i];
  }
```

```
   if (cellVoltage[i] < minVoltage)
   {
     minVoltage = cellVoltage[i];
   }
  }
}

// ======================================================
// TASK 1
// STRONGEST / WEAKEST CELL
// ======================================================

void identifyStrongestWeakest()
{
 strongestCell = 0;

 weakestCell = 0;

 for (int i = 1; i < NUM_CELLS; i++)
 {
   if (cellVoltage[i] >
      cellVoltage[strongestCell])
   {
     strongestCell = i;
   }

   if (cellVoltage[i] <
      cellVoltage[weakestCell])
   {
     weakestCell = i;
   }
 }
}

// ======================================================
// TASK 1
// IMBALANCE
// ======================================================

void calculateImbalance()
{
 imbalanceVoltage =
   maxVoltage - minVoltage;

 if (averageVoltage > 0)
 {
   imbalancePercent =
     (imbalanceVoltage /
      averageVoltage)
     * 100.0;
 }
 else
 {
   imbalancePercent = 0.0;
 }
}

// ======================================================
// TASK 1
// HEALTH
// ======================================================

void classifyBatteryHealth()
{
 if (imbalancePercent < HEALTHY_LIMIT)
```

```
 {
  batteryHealth = "HEALTHY";
 }

 else if (imbalancePercent < MINOR_LIMIT)
 {
  batteryHealth = "MINOR IMBALANCE";
 }

 else if (imbalancePercent < CRITICAL_LIMIT)
 {
  batteryHealth = "CRITICAL IMBALANCE";
 }

 else
 {
  batteryHealth = "PACK FAILURE";
 }
}

// ======================================================
// TASK 2
// SAFETY DETECTION
// ======================================================

void detectSafetyConditions()
{
 underVoltageFault = false;

 overVoltageFault = false;

 rapidChangeFault = false;

 sensorFault = false;

 for (int i = 0; i < NUM_CELLS; i++)
 {
  if (cellVoltage[i] < CELL_MIN_VOLTAGE ||
     cellVoltage[i] > CELL_MAX_VOLTAGE)
  {
   sensorFault = true;
  }

  if (cellVoltage[i] < UV_THRESHOLD)
  {
   underVoltageFault = true;
  }

  if (cellVoltage[i] > OV_THRESHOLD)
  {
   overVoltageFault = true;
  }

  float voltageChange =
   fabs(cellVoltage[i] -
       previousCellVoltage[i]);

  if (voltageChange >
     RAPID_CHANGE_THRESHOLD)
  {
   rapidChangeFault = true;
  }
 }

 for (int i = 0; i < NUM_CELLS; i++)
 {
```

```
    previousCellVoltage[i] =
      cellVoltage[i];
  }

  faultDetected =
    underVoltageFault ||
    overVoltageFault ||
    rapidChangeFault ||
    sensorFault;
}

// =======================================================
// TASK 2
// SAFETY STATE MACHINE
// =======================================================

void updateSafetyState()
{
  unsigned long now = millis();

  switch (safetyState)
  {
    case SAFETY_NORMAL:

      if (faultDetected)
      {
        safetyState = SAFETY_WARNING;

        faultStartTime = now;
      }

      break;

    case SAFETY_WARNING:

      if (!faultDetected)
      {
        safetyState = SAFETY_NORMAL;

        faultConfirmed = false;

        break;
      }

      if (now - faultStartTime >=
          FAULT_CONFIRM_TIME)
      {
        faultConfirmed = true;

        safetyState = SAFETY_TRIPPED;
      }

      break;

    case SAFETY_TRIPPED:

      if (!faultDetected)
      {
        recoveryStartTime = now;

        safetyState = SAFETY_RECOVERY;
      }

      break;

    case SAFETY_RECOVERY:
```

```
    if (faultDetected)
    {
      safetyState = SAFETY_TRIPPED;

      break;
    }

    if (now - recoveryStartTime >=
        RECOVERY_TIME)
    {
      safetyState = SAFETY_NORMAL;

      faultConfirmed = false;
    }

    break;
  }
}

// =====================================================
// TASK 2
// RELAY CONTROL
// =====================================================

void updateRelay()
{
  unsigned long now = millis();

  if (safetyState == SAFETY_TRIPPED ||
      safetyState == SAFETY_WARNING)
  {
    if (relayState == true)
    {
      if (now - lastRelayChangeTime >=
          RELAY_LOCK_TIME)
      {
        digitalWrite(RELAY_PIN, LOW);

        relayState = false;

        lastRelayChangeTime = now;

        Serial.println(">>> RELAY CUT-OFF <<<");
      }
    }
  }

  else if (safetyState ==
           SAFETY_NORMAL)
  {
    if (relayState == false)
    {
      if (now - lastRelayChangeTime >=
          RELAY_LOCK_TIME)
      {
        digitalWrite(RELAY_PIN, HIGH);

        relayState = true;

        lastRelayChangeTime = now;

        Serial.println(">>> RELAY RESTORED <<<");
      }
    }
  }
```

```
}

// ======================================================
// TASK 2
// BUZZER
// ======================================================

void updateBuzzer()
{
 unsigned long now = millis();

 if (safetyState ==
    SAFETY_WARNING ||
    safetyState ==
    SAFETY_TRIPPED)
 {
  if (now - previousBuzzerTime >=
     BUZZER_INTERVAL)
  {
    previousBuzzerTime = now;

    digitalWrite(
     BUZZER_PIN,
     !digitalRead(BUZZER_PIN)
    );
  }
 }

 else
 {
  digitalWrite(BUZZER_PIN, LOW);
 }
}

// ======================================================
// TASK 4
// SENSOR HEALTH
// ======================================================

void checkSensorHealth()
{
 invalidSensorFault = false;

 for (int i = 0; i < NUM_CELLS; i++)
 {
  // Extreme ADC values simulate
  // disconnected/invalid sensor

  if (cellADC[i] <= 5 ||
     cellADC[i] >= 4090)
  {
    invalidSensorFault = true;

    String fault =
     "C" + String(i + 1)
     + " INVALID SENSOR";

    logFault(fault, "DEGRADED");
  }
 }
}

// ======================================================
// TASK 4
// FROZEN ADC DETECTION
// ======================================================
```

```
void checkFrozenADC()
{
 frozenADCFault = false;

 for (int i = 0; i < NUM_CELLS; i++)
 {
  if (cellADC[i] == previousADC[i])
  {
   frozenCount[i]++;
  }
  else
  {
   frozenCount[i] = 0;
  }

  if (frozenCount[i] >= FROZEN_LIMIT)
  {
   frozenADCFault = true;

   String fault =
    "C" + String(i + 1)
    + " ADC FROZEN";

   logFault(fault, "DEGRADED");

   // Prevent excessive counter growth
   frozenCount[i] = FROZEN_LIMIT;
  }

  previousADC[i] = cellADC[i];
 }
}

// ====================================================
// TASK 4
// RELAY MISMATCH
// ====================================================

void checkRelayMismatch()
{
 relayMismatchFault = false;

 // Software simulation only
 if (simulateRelayMismatch)
 {
  relayMismatchFault = true;

  logFault(
   "RELAY MISMATCH",
   "FAILSAFE"
  );
 }
}

// ====================================================
// TASK 4
// RUNTIME MODE
// ====================================================

void updateRuntimeMode()
{
 unsigned long now = millis();

 // ================================================
 // CRITICAL FAULT → FAILSAFE
```

```
// ================================================

if (relayMismatchFault ||
    overVoltageFault ||
    underVoltageFault)
{
  if (runtimeMode != RUNTIME_FAILSAFE)
  {
    logFault(
      "CRITICAL SAFETY FAULT",
      "FAILSAFE"
    );
  }

  runtimeMode = RUNTIME_FAILSAFE;

  degradedStartTime = 0;

  return;
}

// ================================================
// SHUTDOWN CONDITION
// ================================================

if (sensorFault &&
    invalidSensorFault &&
    frozenADCFault)
{
  runtimeMode = RUNTIME_SHUTDOWN;

  logFault(
    "MULTIPLE SYSTEM FAILURE",
    "SHUTDOWN"
  );

  return;
}

// ================================================
// DEGRADED
// ================================================

if (invalidSensorFault ||
    frozenADCFault ||
    rapidChangeFault)
{
  if (runtimeMode ==
      RUNTIME_NORMAL)
  {
    runtimeMode = RUNTIME_DEGRADED;

    degradedStartTime = now;

    logFault(
      "SYSTEM DEGRADED",
      "DEGRADED"
    );
  }

  return;
}

// ================================================
// RECOVERY FROM FAILSAFE
// ================================================
```

```
if (runtimeMode ==
   RUNTIME_FAILSAFE)
{
 if (!faultDetected &&
    !relayMismatchFault)
 {
  if (degradedStartTime == 0)
  {
   degradedStartTime = now;
  }

  if (now - degradedStartTime >=
     RECOVERY_TIME)
  {
   runtimeMode =
    RUNTIME_NORMAL;

   degradedStartTime = 0;

   logFault(
    "FAILSAFE RECOVERED",
    "NORMAL"
   );
  }
 }

 return;
}

// ================================================
// RECOVERY FROM DEGRADED
// ================================================

if (runtimeMode ==
   RUNTIME_DEGRADED)
{
 if (!invalidSensorFault &&
    !frozenADCFault &&
    !rapidChangeFault)
 {
  if (degradedStartTime == 0)
  {
   degradedStartTime = now;
  }

  if (now - degradedStartTime >=
     DEGRADED_RECOVERY_TIME)
  {
   runtimeMode =
    RUNTIME_NORMAL;

   degradedStartTime = 0;

   logFault(
    "DEGRADED RECOVERED",
    "NORMAL"
   );
  }
 }

 return;
}

// ================================================
// NORMAL
```

```
  // ================================================

  if (!faultDetected &&
     !invalidSensorFault &&
     !frozenADCFault &&
     !relayMismatchFault)
  {
   runtimeMode = RUNTIME_NORMAL;
  }
}

// ======================================================
// TASK 4
// FAULT LOGGING
// ======================================================

void logFault(String faultName,
          String mode)
{
  // Avoid filling the log repeatedly
  // with the same event every sample

  static String lastFault = "";

  if (faultName == lastFault)
  {
   return;
  }

  lastFault = faultName;

  if (faultLogCount < MAX_FAULT_LOGS)
  {
   faultLogs[faultLogCount].timestamp =
     millis();

   faultLogs[faultLogCount].faultName =
     faultName;

   faultLogs[faultLogCount].mode =
     mode;

   faultLogCount++;
  }

  Serial.println();
  Serial.println("******** FAULT LOG ********");

  Serial.print("Time  : ");
  Serial.print(millis());
  Serial.println(" ms");

  Serial.print("Fault : ");
  Serial.println(faultName);

  Serial.print("Mode  : ");
  Serial.println(mode);

  Serial.println("****************************");
}

// ======================================================
// LED CONTROL
// ======================================================

void updateLEDs()
```

```
{
 digitalWrite(LED_GREEN, LOW);

 digitalWrite(LED_YELLOW, LOW);

 digitalWrite(LED_ORANGE, LOW);

 digitalWrite(LED_RED, LOW);

 // ================================================
 // TASK 4 PRIORITY
 // ================================================

 if (runtimeMode ==
    RUNTIME_SHUTDOWN)
 {
  digitalWrite(LED_RED, HIGH);

  return;
 }

 if (runtimeMode ==
    RUNTIME_FAILSAFE)
 {
  digitalWrite(LED_RED, HIGH);

  return;
 }

 if (runtimeMode ==
    RUNTIME_DEGRADED)
 {
  digitalWrite(LED_YELLOW, HIGH);

  return;
 }

 // ================================================
 // TASK 2 SAFETY
 // ================================================

 if (safetyState ==
    SAFETY_WARNING ||
    safetyState ==
    SAFETY_TRIPPED)
 {
  digitalWrite(LED_RED, HIGH);

  return;
 }

 // ================================================
 // TASK 1 HEALTH
 // ================================================

 if (batteryHealth == "HEALTHY")
 {
  digitalWrite(LED_GREEN, HIGH);
 }

 else if (batteryHealth ==
       "MINOR IMBALANCE")
 {
  digitalWrite(LED_YELLOW, HIGH);
 }
```

```
 else if (batteryHealth ==
       "CRITICAL IMBALANCE")
 {
  digitalWrite(LED_ORANGE, HIGH);
 }

 else
 {
  digitalWrite(LED_RED, HIGH);
 }
}

// ======================================================
// TASK 3 + TASK 4
// LCD HMI
// ======================================================

void updateLCD()
{
 // ================================================
 // TASK 4 SHUTDOWN
 // ================================================

 if (runtimeMode ==
    RUNTIME_SHUTDOWN)
 {
  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("SYSTEM SHUTDOWN");

  lcd.setCursor(0, 1);
  lcd.print("CHECK FAULT");

  return;
 }

 // ================================================
 // TASK 4 FAILSAFE
 // ================================================

 if (runtimeMode ==
    RUNTIME_FAILSAFE)
 {
  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("RUNTIME:FAILSAFE");

  lcd.setCursor(0, 1);
  lcd.print("RELAY:OFF");

  return;
 }

 // ================================================
 // TASK 4 DEGRADED
 // ================================================

 if (runtimeMode ==
    RUNTIME_DEGRADED)
 {
  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("RUNTIME:DEGRADE");
```

```
 lcd.setCursor(0, 1);

 if (invalidSensorFault)
 {
  lcd.print("SENSOR FAULT");
 }

 else if (frozenADCFault)
 {
  lcd.print("ADC FROZEN");
 }

 else
 {
  lcd.print("CHECK SYSTEM");
 }

 return;
}

// ================================================
// TASK 3 FAULT PRIORITY
// ================================================

if (faultDetected)
{
 if (!faultScreenShown)
 {
   displayFaultPriority();

   faultScreenShown = true;
 }

 return;
}

// Fault cleared

if (faultScreenShown)
{
 lcd.clear();

 faultScreenShown = false;

 previousPage = -1;
}

// ================================================
// NORMAL HMI
// ================================================

if (currentPage == previousPage)
{
 return;
}

previousPage = currentPage;

lcd.clear();

// ================================================
// PAGE 1
// ================================================

if (currentPage == 0)
```

```
{
  lcd.setCursor(0, 0);

  lcd.print("C1:");
  lcd.print(cellVoltage[0], 2);

  lcd.print(" C2:");
  lcd.print(cellVoltage[1], 2);

  lcd.setCursor(0, 1);

  lcd.print("C3:");
  lcd.print(cellVoltage[2], 2);

  lcd.print(" C4:");
  lcd.print(cellVoltage[3], 2);
}

// ================================================
// PAGE 2
// ================================================

else if (currentPage == 1)
{
  lcd.setCursor(0, 0);

  lcd.print("PACK:");
  lcd.print(packVoltage, 2);

  lcd.print("V");

  lcd.setCursor(0, 1);

  lcd.print("AVG:");
  lcd.print(averageVoltage, 2);

  lcd.print("V");
}

// ================================================
// PAGE 3
// ================================================

else if (currentPage == 2)
{
  lcd.setCursor(0, 0);

  lcd.print("IMB:");
  lcd.print(imbalancePercent, 1);

  lcd.print("%");

  lcd.setCursor(0, 1);

  lcd.print("W:C");
  lcd.print(weakestCell + 1);

  lcd.print(" S:C");
  lcd.print(strongestCell + 1);
}

// ================================================
// PAGE 4
// ================================================

else if (currentPage == 3)
```

```
 {
  lcd.setCursor(0, 0);

  lcd.print("SAFETY:");

  if (safetyState ==
     SAFETY_NORMAL)
  {
   lcd.print("NORMAL");
  }

  else if (safetyState ==
        SAFETY_WARNING)
  {
   lcd.print("WARNING");
  }

  else if (safetyState ==
        SAFETY_TRIPPED)
  {
   lcd.print("TRIPPED");
  }

  else
  {
   lcd.print("RECOVERY");
  }

  lcd.setCursor(0, 1);

  lcd.print("RELAY:");

  if (relayState)
   lcd.print("ON ");
  else
   lcd.print("OFF");
 }

 // ================================================
 // PAGE 5
 // ================================================

 else if (currentPage == 4)
 {
  lcd.setCursor(0, 0);

  lcd.print("DIAGNOSTICS");

  lcd.setCursor(0, 1);

  lcd.print("SYSTEM OK");
 }
}

// ====================================================
// TASK 3
// FAULT PRIORITY DISPLAY
// ====================================================

void displayFaultPriority()
{
 lcd.clear();

 if (underVoltageFault)
 {
  lcd.setCursor(0, 0);
```

```
  lcd.print("FAULT: UNDER V");

  lcd.setCursor(0, 1);

  lcd.print("C");
  lcd.print(weakestCell + 1);
  lcd.print(":");
  lcd.print(cellVoltage[weakestCell], 2);
  lcd.print("V");
 }

 else if (overVoltageFault)
 {
  lcd.setCursor(0, 0);

  lcd.print("FAULT: OVER V");

  lcd.setCursor(0, 1);

  lcd.print("CHECK CELL");
 }

 else if (rapidChangeFault)
 {
  lcd.setCursor(0, 0);

  lcd.print("FAULT: RAPID DV");

  lcd.setCursor(0, 1);

  lcd.print("CHECK BATTERY");
 }

 else if (sensorFault)
 {
  lcd.setCursor(0, 0);

  lcd.print("FAULT: SENSOR");

  lcd.setCursor(0, 1);

  lcd.print("SAFE MODE");
 }
}

// ====================================================
// SERIAL REPORT
// ====================================================

void displaySerialReport()
{
 Serial.println();

 Serial.println(
  "================================================"
 );

 Serial.println(
  "      EV BMS TASK 1 + 2 + 3 + 4"
 );

 Serial.println(
  "================================================"
 );
```

```
// ================================================
// BATTERY
// ================================================

Serial.println("BATTERY PARAMETERS");

Serial.println(
  "------------------------------------------------"
);

for (int i = 0; i < NUM_CELLS; i++)
{
  Serial.print("Cell ");
  Serial.print(i + 1);

  Serial.print(" Voltage     ");

  Serial.print(cellVoltage[i], 2);

  Serial.println(" V");
}

Serial.print("Pack Voltage          ");
Serial.print(packVoltage, 2);
Serial.println(" V");

Serial.print("Average Voltage       ");
Serial.print(averageVoltage, 2);
Serial.println(" V");

Serial.print("Imbalance             ");
Serial.print(imbalancePercent, 2);
Serial.println(" %");

Serial.print("Strongest Cell        C");
Serial.println(strongestCell + 1);

Serial.print("Weakest Cell          C");
Serial.println(weakestCell + 1);

Serial.print("Battery Health        ");
Serial.println(batteryHealth);

// ================================================
// TASK 2
// ================================================

Serial.println();

Serial.println("SAFETY STATUS");

Serial.println(
  "------------------------------------------------"
);

Serial.print("Under Voltage         ");
Serial.println(
  underVoltageFault ? "YES" : "NO"
);

Serial.print("Over Voltage          ");
Serial.println(
  overVoltageFault ? "YES" : "NO"
);

Serial.print("Rapid Voltage Change    ");
```

```
Serial.println(
 rapidChangeFault ? "YES" : "NO"
);

Serial.print("Safety State          ");

if (safetyState ==
   SAFETY_NORMAL)
{
 Serial.println("NORMAL");
}

else if (safetyState ==
      SAFETY_WARNING)
{
 Serial.println("WARNING");
}

else if (safetyState ==
      SAFETY_TRIPPED)
{
 Serial.println("TRIPPED");
}

else
{
 Serial.println("RECOVERY");
}

Serial.print("Relay                ");

Serial.println(
 relayState ? "ON" : "OFF"
);

// ================================================
// TASK 4
// ================================================

Serial.println();

Serial.println("FAULT-TOLERANT RUNTIME");

Serial.println(
 "------------------------------------------------"
);

Serial.print("Runtime Mode          ");

if (runtimeMode ==
   RUNTIME_NORMAL)
{
 Serial.println("NORMAL");
}

else if (runtimeMode ==
      RUNTIME_DEGRADED)
{
 Serial.println("DEGRADED");
}

else if (runtimeMode ==
      RUNTIME_FAILSAFE)
{
 Serial.println("FAILSAFE");
}
```

```
else
{
 Serial.println("SHUTDOWN");
}

Serial.print("Invalid Sensor         ");

Serial.println(
 invalidSensorFault ? "YES" : "NO"
);

Serial.print("Frozen ADC            ");

Serial.println(
 frozenADCFault ? "YES" : "NO"
);

Serial.print("Relay Mismatch        ");

Serial.println(
 relayMismatchFault ? "YES" : "NO"
);

// ================================================
// FAULT LOG
// ================================================

Serial.println();

Serial.println("FAULT LOG");

Serial.println(
 "------------------------------------------------"
);

if (faultLogCount == 0)
{
 Serial.println("No faults logged");
}

else
{
 for (int i = 0; i < faultLogCount; i++)
 {
  Serial.print("#");
  Serial.print(i + 1);

  Serial.print("  ");

  Serial.print(
   faultLogs[i].timestamp
  );

  Serial.print(" ms  ");

  Serial.print(
   faultLogs[i].faultName
  );

  Serial.print("  [");

  Serial.print(
   faultLogs[i].mode
  );
```

```
    Serial.println("]");
  }
 }

 Serial.println(
  "================================================"
 );
}
```

11.4 Task 4 Wokwi):

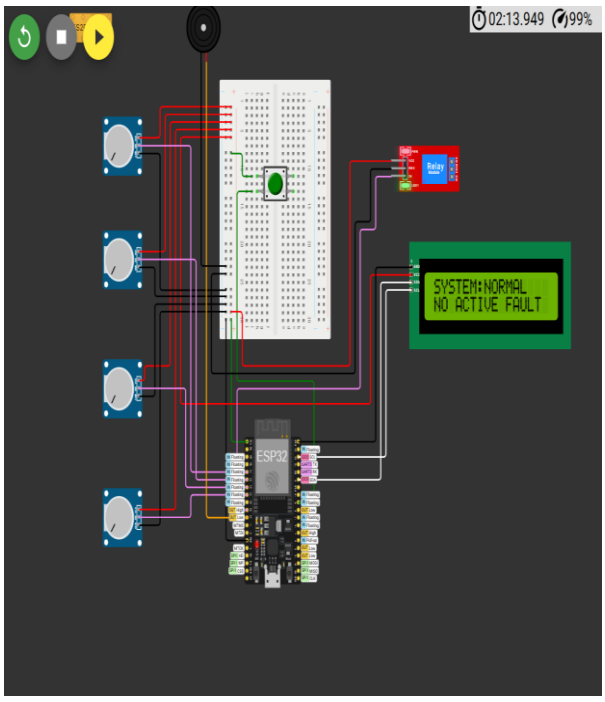
[Task-4 Click Here]

11.5 Task 4 Video Demonstration Link

[Click Here]

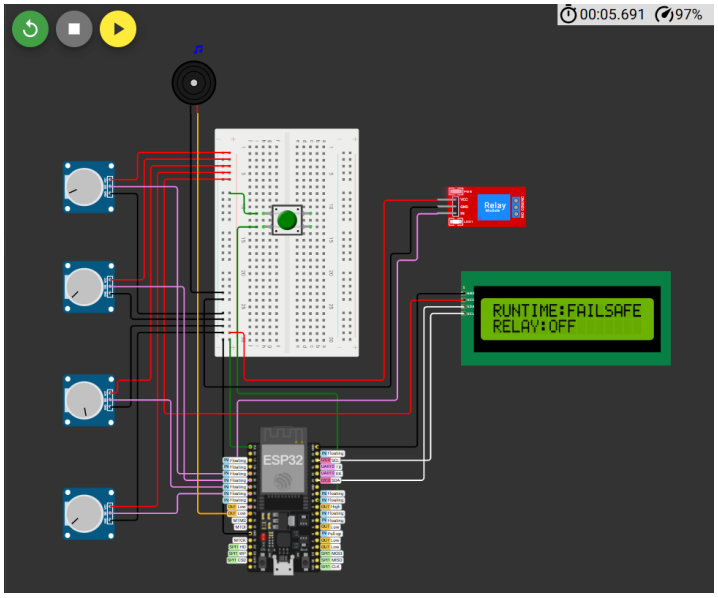
11.3 Simulation Running Photos with correct Output:

Case 1 — Normal




```
================================================
       EV BMS TASK 1 + 2 + 3 + 4
================================================
BATTERY PARAMETERS
------------------------------------------------
Cell 1 Voltage      3.00 V
Cell 2 Voltage      3.00 V
Cell 3 Voltage      2.99 V
Cell 4 Voltage      3.01 V
Pack Voltage             12.00 V
Average Voltage          3.00 V
Imbalance                0.61 %
Strongest Cell           C4
Weakest Cell             C3
Battery Health           HEALTHY

SAFETY STATUS
------------------------------------------------
Under Voltage            NO
Over Voltage             NO
Rapid Voltage Change     NO
Safety State             NORMAL
Relay                    ON

FAULT-TOLERANT RUNTIME
------------------------------------------------
Runtime Mode             NORMAL
Invalid Sensor           NO
Frozen ADC               NO
Relay Mismatch           NO

FAULT LOG
------------------------------------------------
No faults logged
------------------------------------------------
```

Case 2 — FAILSAFE / Relay Mismatch




```
================================================
       EV BMS TASK 1 + 2 + 3 + 4
================================================
BATTERY PARAMETERS
------------------------------------------------
Cell 1 Voltage      3.19 V
Cell 2 Voltage      3.07 V
Cell 3 Voltage      2.72 V
Cell 4 Voltage      3.08 V
Pack Voltage            12.07 V
Average Voltage         3.02 V
Imbalance               15.65 %
Strongest Cell          C1
Weakest Cell            C3
Battery Health          CRITICAL IMBALANCE

SAFETY STATUS
------------------------------------------------
Under Voltage           YES
Over Voltage            NO
Rapid Voltage Change    NO
Safety State            TRIPPED
Relay                   OFF

FAULT-TOLERANT RUNTIME
------------------------------------------------
Runtime Mode            FAILSAFE
Invalid Sensor          NO
Frozen ADC              NO
Relay Mismatch          NO

FAULT LOG
------------------------------------------------
#1  14290 ms  CRITICAL SAFETY FAULT  [FAILSAFE]
#2  14302 ms  UNDER VOLTAGE  [FAILSAFE]
#3  14313 ms  RAPID VOLTAGE CHANGE  [DEGRADED]
================================================
```

12. Task 5 : Intelligent Cloud Telemetry Architecture:

12.1 Task 5 Problem

In this task, candidates must design a smart IoT telemetry system using the Blynk platform for cloud communication. Instead of fixed-interval telemetry, the system should transmit data only during state changes, anomalies, or voltage threshold violations. The implementation must include WiFi reconnect handling, event queue synchronization after reconnection, signal quality monitoring, and asynchronous cloud communication while ensuring the embedded system continues functioning normally during network failures.

12.2 Code :

```
void updateSafetyState()

{

  unsigned long now = millis();#define BLYNK_TEMPLATE_ID "TMPL36OnLHkD5"

#define BLYNK_TEMPLATE_NAME "EV BMS Task5"

#define BLYNK_AUTH_TOKEN "nKkqxnUij0v4qdk0oJ-niWzcGSOd8tmh"

#define CLOUD_TEST_BUTTON 25


#include <WiFi.h>

#include <BlynkSimpleEsp32.h>
```

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <math.h>

const char* WIFI_SSID = "Wokwi-GUEST";
const char* WIFI_PASSWORD = "";

// =====================================================
// EV BMS - TASK 1 + TASK 2 + TASK 3 + TASK 4
//
// TASK 1 : Adaptive Multi-Cell Battery Intelligence
// TASK 2 : Event-Driven Safety Protection
// TASK 3 : Intelligent HMI & Diagnostic Interface
// TASK 4 : Fault-Tolerant Embedded Runtime
//
// Hardware:
// ESP32 DevKit V1
// 4 x Potentiometers
// 16x2 I2C LCD
// 4 x LEDs
// Relay
// Buzzer
// =====================================================

// =====================================================
// 1. PIN CONFIGURATION
// =====================================================

#define CELL1_PIN 34
#define CELL2_PIN 35
#define CELL3_PIN 32
#define CELL4_PIN 33

#define SDA_PIN 21
#define SCL_PIN 22

#define LED_GREEN  4
#define LED_YELLOW 2
```

```
#define LED_ORANGE 15
#define LED_RED    5

#define RELAY_PIN  26
#define BUZZER_PIN 27

// ====================================================
// 2. LCD
// ====================================================

LiquidCrystal_I2C lcd(0x27, 16, 2);

// ====================================================
// 3. BATTERY CONFIGURATION
// ====================================================

#define NUM_CELLS 4

const int ADC_MAX = 4095;

const float ADC_MAX_VOLTAGE = 3.3;

const float CELL_MIN_VOLTAGE = 2.5;
const float CELL_MAX_VOLTAGE = 4.2;

// ====================================================
// 4. TASK 1 HEALTH THRESHOLDS
// ====================================================

const float HEALTHY_LIMIT = 3.0;
const float MINOR_LIMIT   = 10.0;
const float CRITICAL_LIMIT = 10.0;

// ====================================================
// 5. TASK 2 SAFETY THRESHOLDS
// ====================================================

const float UV_THRESHOLD = 2.8;
```

```
const float OV_THRESHOLD = 4.15;

const float RAPID_CHANGE_THRESHOLD = 0.10;

// =======================================================
// 6. TIMING
// =======================================================

const unsigned long SAMPLE_INTERVAL = 200;

const unsigned long LCD_PAGE_INTERVAL = 3000;

const unsigned long LCD_REFRESH_INTERVAL = 500;

const unsigned long SERIAL_INTERVAL = 2000;

const unsigned long FAULT_CONFIRM_TIME = 500;

const unsigned long SAFETY_RECOVERY_TIME = 5000;

const unsigned long RUNTIME_RECOVERY_TIME = 3000;

const unsigned long RELAY_LOCK_TIME = 3000;

const unsigned long BUZZER_INTERVAL = 300;

// =======================================================
// 7. TASK 4 DIAGNOSTIC SETTINGS
// =======================================================

// IMPORTANT:
//
// Keep all FALSE for NORMAL operation.
//
// Change ONE at a time for testing.
//
// -------------------------------------------------------
```

```
bool simulateFrozenADC = false;

bool simulateRelayMismatch = false;

bool simulateShutdown = false;

bool cloudTestMode = false;

// Frozen ADC threshold
const int FROZEN_LIMIT = 10;

// ====================================================
// 8. TIMERS
// ====================================================

unsigned long previousSampleTime = 0;

unsigned long previousLCDPageTime = 0;

unsigned long previousLCDRefreshTime = 0;

unsigned long previousSerialTime = 0;

unsigned long faultStartTime = 0;

unsigned long safetyRecoveryStart = 0;

unsigned long runtimeRecoveryStart = 0;

unsigned long lastRelayChangeTime = 0;

unsigned long previousBuzzerTime = 0;

// ====================================================
// 9. BATTERY DATA
// ====================================================

float cellVoltage[NUM_CELLS];
```

```
float previousCellVoltage[NUM_CELLS];

int cellADC[NUM_CELLS];

int previousADC[NUM_CELLS];

int frozenCount[NUM_CELLS];

bool cellSensorHealthy[NUM_CELLS];

float packVoltage = 0.0;

float averageVoltage = 0.0;

float maxVoltage = 0.0;

float minVoltage = 0.0;

float imbalanceVoltage = 0.0;

float imbalancePercent = 0.0;

int strongestCell = 0;

int weakestCell = 0;

String batteryHealth;

// ====================================================
// 10. TASK 2 SAFETY STATE
// ====================================================

enum SafetyState
{
  SAFETY_NORMAL,
  SAFETY_WARNING,
  SAFETY_TRIPPED,
```

```
 SAFETY_RECOVERY
};

SafetyState safetyState = SAFETY_NORMAL;

// ====================================================
// 11. TASK 2 FAULT FLAGS
// ====================================================

bool underVoltageFault = false;

bool overVoltageFault = false;

bool rapidChangeFault = false;

bool sensorFault = false;

bool faultDetected = false;

bool faultConfirmed = false;

// ====================================================
// 12. TASK 4 RUNTIME STATES
// ====================================================

enum RuntimeMode
{
 RUNTIME_NORMAL,
 RUNTIME_DEGRADED,
 RUNTIME_FAILSAFE,
 RUNTIME_SHUTDOWN
};

RuntimeMode runtimeMode = RUNTIME_NORMAL;

// ====================================================
// 13. TASK 4 FAULT FLAGS
// ====================================================
```

```
bool invalidSensorFault = false;

bool frozenADCFault = false;

bool relayMismatchFault = false;

bool multipleFailureFault = false;

// ====================================================

// 14. RELAY

// ====================================================

bool relayState = true;

// ====================================================

// 15. LCD HMI

// ====================================================

int currentPage = 0;

const int TOTAL_PAGES = 5;

bool faultPriorityActive = false;

RuntimeMode previousRuntimeMode = RUNTIME_NORMAL;

SafetyState previousSafetyState = SAFETY_NORMAL;

int previousPage = -1;

// ====================================================

// 16. FAULT LOG

// ====================================================

struct FaultLog

{

  unsigned long timestamp;
```

```
  String faultName;

  String mode;
};

#define MAX_FAULT_LOGS 10

FaultLog faultLogs[MAX_FAULT_LOGS];

int faultLogCount = 0;

// ====================================================
// 17. FAULT EVENT MEMORY
// ====================================================

bool previousInvalidSensorFault = false;

bool previousFrozenADCFault = false;

bool previousRelayMismatchFault = false;

bool previousUnderVoltageFault = false;

bool previousOverVoltageFault = false;

bool previousRapidChangeFault = false;


// ====================================================
// TASK 5 - INTELLIGENT CLOUD TELEMETRY
// ====================================================
// Blynk virtual datastream mapping - MATCHES THE USER'S DASHBOARD:
 // V0  Cell 1 Voltage
 // V1  Cell 2 Voltage
 // V2  Cell 3 Voltage
 // V3  Cell 4 Voltage
 // V4  Pack Voltage
```

```
// V5  Average Voltage
// V6  Imbalance %
// V7  Battery Health
// V8  Runtime Mode
// V9  Safety State
// V10 Relay State
// V11 Fault / Event Message
// V12 WiFi RSSI
//
// IMPORTANT: Create the same datastream VPINs in Blynk.
// Event code used below: bms_fault
// ================================================

#define ENABLE_BLYNK_EVENTS 1

const unsigned long WIFI_RECONNECT_INTERVAL = 5000;
const unsigned long BLYNK_RECONNECT_INTERVAL = 5000;
const float TELEMETRY_VOLTAGE_CHANGE = 0.05;
const float TELEMETRY_IMBALANCE_CHANGE = 0.5;

unsigned long previousWiFiReconnect = 0;
unsigned long previousBlynkReconnect = 0;

bool previousCloudConnected = false;
bool cloudSyncRequired = true;

// Last values actually transmitted to Blynk
float sentCellVoltage[NUM_CELLS] = {-100, -100, -100, -100};
float sentPackVoltage = -100;
float sentAverageVoltage = -100;
float sentImbalancePercent = -100;
String sentRuntimeMode = "";
String sentSafetyState = "";
String sentHealth = "";
String sentFaultText = "";
int sentRelayState = -1;
int sentRSSI = 999;
```

```
// Small event queue for faults/state changes while cloud is offline
#define CLOUD_EVENT_QUEUE_SIZE 10
String cloudEventQueue[CLOUD_EVENT_QUEUE_SIZE];
int cloudEventHead = 0;
int cloudEventTail = 0;
int cloudEventCount = 0;

void queueCloudEvent(const String &eventText)
{
 if (cloudEventCount < CLOUD_EVENT_QUEUE_SIZE)
 {
  cloudEventQueue[cloudEventTail] = eventText;
  cloudEventTail = (cloudEventTail + 1) % CLOUD_EVENT_QUEUE_SIZE;
  cloudEventCount++;
 }
 else
 {
  // Keep the newest event when the queue is full.
  cloudEventQueue[cloudEventHead] = eventText;
  cloudEventHead = (cloudEventHead + 1) % CLOUD_EVENT_QUEUE_SIZE;
  cloudEventTail = cloudEventHead;
 }
}

bool cloudEventAvailable()
{
 return cloudEventCount > 0;
}

String popCloudEvent()
{
 if (cloudEventCount == 0) return "";

 String e = cloudEventQueue[cloudEventHead];
 cloudEventHead = (cloudEventHead + 1) % CLOUD_EVENT_QUEUE_SIZE;
 cloudEventCount--;
 return e;
}
```

```
String getRuntimeModeText()
{
 switch (runtimeMode)
 {
  case RUNTIME_NORMAL:   return "NORMAL";
  case RUNTIME_DEGRADED: return "DEGRADED";
  case RUNTIME_FAILSAFE: return "FAILSAFE";
  default:               return "SHUTDOWN";
 }
}

String getSafetyStateText()
{
 switch (safetyState)
 {
  case SAFETY_NORMAL:   return "NORMAL";
  case SAFETY_WARNING:  return "WARNING";
  case SAFETY_TRIPPED:  return "TRIPPED";
  default:              return "RECOVERY";
 }
}

String getMainFaultText()
{
 if (overVoltageFault) return "OVER VOLTAGE";
 if (underVoltageFault) return "UNDER VOLTAGE";
 if (relayMismatchFault) return "RELAY MISMATCH";
 if (invalidSensorFault) return "INVALID SENSOR";
 if (frozenADCFault) return "ADC FROZEN";
 if (rapidChangeFault) return "RAPID VOLTAGE CHANGE";
 return "NO ACTIVE FAULT";
}

void sendBlynkEvent(const String &eventText)
{
#if ENABLE_BLYNK_EVENTS
 if (Blynk.connected())
```

```
 {
   Blynk.logEvent("bms_fault", eventText);
 }
#else
 (void)eventText;
#endif
}

void syncAllTelemetry()
{
 if (!Blynk.connected()) return;

 // Full synchronization is performed ONLY after a cloud connection/reconnection.
 Blynk.virtualWrite(V0, cellVoltage[0]);
 Blynk.virtualWrite(V1, cellVoltage[1]);
 Blynk.virtualWrite(V2, cellVoltage[2]);
 Blynk.virtualWrite(V3, cellVoltage[3]);
 Blynk.virtualWrite(V4, packVoltage);
 Blynk.virtualWrite(V5, averageVoltage);
 Blynk.virtualWrite(V6, imbalancePercent);
 Blynk.virtualWrite(V7, batteryHealth);
 Blynk.virtualWrite(V8, getRuntimeModeText());
 Blynk.virtualWrite(V9, getSafetyStateText());
 Blynk.virtualWrite(V10, relayState ? 1 : 0);
 Blynk.virtualWrite(V11, getMainFaultText());
 Blynk.virtualWrite(V12, WiFi.RSSI());

 for (int i = 0; i < NUM_CELLS; i++)
   sentCellVoltage[i] = cellVoltage[i];

 sentPackVoltage = packVoltage;
 sentAverageVoltage = averageVoltage;
 sentImbalancePercent = imbalancePercent;
 sentRuntimeMode = getRuntimeModeText();
 sentSafetyState = getSafetyStateText();
 sentRelayState = relayState ? 1 : 0;
 sentRSSI = WiFi.RSSI();
 sentHealth = batteryHealth;
```

```
  sentFaultText = getMainFaultText();

  cloudSyncRequired = false;
}

void flushCloudEvents()
{
  if (!Blynk.connected()) return;

  // Send queued events after reconnect, then remove them from the queue.
  while (cloudEventAvailable())
  {
    String eventText = popCloudEvent();
    Blynk.virtualWrite(V10, eventText);
    sendBlynkEvent(eventText);
  }
}

void publishChangedTelemetry()
{
  if (!Blynk.connected()) return;

  // V0-V3: cell voltage; send only after a meaningful change.
  for (int i = 0; i < NUM_CELLS; i++)
  {
    if (fabs(cellVoltage[i] - sentCellVoltage[i]) >= TELEMETRY_VOLTAGE_CHANGE)
    {
      Blynk.virtualWrite(V0 + i, cellVoltage[i]);
      sentCellVoltage[i] = cellVoltage[i];
    }
  }

  // V4: pack voltage.
  if (fabs(packVoltage - sentPackVoltage) >= TELEMETRY_VOLTAGE_CHANGE)
  {
    Blynk.virtualWrite(V4, packVoltage);
    sentPackVoltage = packVoltage;
  }
```

```
// V5: average voltage.
if (fabs(averageVoltage - sentAverageVoltage) >= TELEMETRY_VOLTAGE_CHANGE)
{
  Blynk.virtualWrite(V5, averageVoltage);
  sentAverageVoltage = averageVoltage;
}

// V6: imbalance percentage.
if (fabs(imbalancePercent - sentImbalancePercent) >= TELEMETRY_IMBALANCE_CHANGE)
{
  Blynk.virtualWrite(V6, imbalancePercent);
  sentImbalancePercent = imbalancePercent;
}

// V7: battery health.
if (batteryHealth != sentHealth)
{
  Blynk.virtualWrite(V7, batteryHealth);
  sentHealth = batteryHealth;
}

// V8: runtime mode.
String runtimeText = getRuntimeModeText();
if (runtimeText != sentRuntimeMode)
{
  Blynk.virtualWrite(V8, runtimeText);
  sentRuntimeMode = runtimeText;
}

// V9: safety state.
String safetyText = getSafetyStateText();
if (safetyText != sentSafetyState)
{
  Blynk.virtualWrite(V9, safetyText);
  sentSafetyState = safetyText;
}
```

```
  // V10: relay state.
  int relay = relayState ? 1 : 0;
  if (relay != sentRelayState)
  {
    Blynk.virtualWrite(V10, relay);
    sentRelayState = relay;
  }

  // V11: fault/event text.
  String faultText = getMainFaultText();
  if (faultText != sentFaultText)
  {
    Blynk.virtualWrite(V11, faultText);
    sentFaultText = faultText;
  }

  // V12: Wi-Fi RSSI.
  int rssi = WiFi.RSSI();
  if (abs(rssi - sentRSSI) >= 5)
  {
    Blynk.virtualWrite(V12, rssi);
    sentRSSI = rssi;
  }
}

void processCloudTelemetry()
{
  unsigned long now = millis();

  // WiFi reconnect is non-blocking: no delay() is used here.
  if (WiFi.status() != WL_CONNECTED)
  {
    if (now - previousWiFiReconnect >= WIFI_RECONNECT_INTERVAL)
    {
      previousWiFiReconnect = now;
      WiFi.disconnect();
      WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    }
```

```
    previousCloudConnected = false;
    return;
  }

  // Blynk reconnect is attempted periodically,
 // unless cloud test mode intentionally disables it.
 if (cloudTestMode)
 {
  if (Blynk.connected())
  {
    Blynk.disconnect();
  }

  previousCloudConnected = false;
  return;
 }

 if (!Blynk.connected())
 {
  if (now - previousBlynkReconnect >= BLYNK_RECONNECT_INTERVAL)
  {
    previousBlynkReconnect = now;
    Blynk.connect(1000);
  }

  previousCloudConnected = false;
  return;
 }

  // Detect a new cloud connection.
  if (!previousCloudConnected)
  {
    previousCloudConnected = true;
    cloudSyncRequired = true;
    Serial.println(">>> BLYNK CONNECTED - CLOUD SYNC <<<");
  }
```

```
  if (cloudSyncRequired)
  {
    syncAllTelemetry();
    flushCloudEvents();
  }
  else
  {
    publishChangedTelemetry();
  }
}


void connectWiFi()
{
  Serial.println();
  Serial.println("Starting Wi-Fi connection...");
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
}

void handleCloudTestButton()
{
  bool pressed = (digitalRead(CLOUD_TEST_BUTTON) == LOW);

  if (pressed && !cloudTestMode)
  {
    cloudTestMode = true;

    Serial.println();
    Serial.println("=======================================");
    Serial.println(">>> CLOUD TEST MODE: OFFLINE <<<");
    Serial.println("Blynk communication intentionally disabled");
    Serial.println("BMS operation continues normally");
    Serial.println("=======================================");
  }

  if (!pressed && cloudTestMode)
  {
```

```
    cloudTestMode = false;

    Serial.println();
    Serial.println("======================================");
    Serial.println(">>> CLOUD TEST MODE: ONLINE <<<");
    Serial.println("Blynk communication restored");
    Serial.println("Queued events will synchronize");
    Serial.println("======================================");
  }
}
// ====================================================
// SETUP
// ====================================================

void setup()
{
  Serial.begin(115200);

  connectWiFi();

  Blynk.config(BLYNK_AUTH_TOKEN);

  Serial.println("Blynk configured; cloud connection will be handled in loop.");

  pinMode(CLOUD_TEST_BUTTON, INPUT_PULLUP);
  // ================================================
  // ADC
  // ================================================

  analogReadResolution(12);

  pinMode(CELL1_PIN, INPUT);
  pinMode(CELL2_PIN, INPUT);
  pinMode(CELL3_PIN, INPUT);
  pinMode(CELL4_PIN, INPUT);

  // ================================================
  // LEDs
```

```
// ================================================

pinMode(LED_GREEN, OUTPUT);
pinMode(LED_YELLOW, OUTPUT);
pinMode(LED_ORANGE, OUTPUT);
pinMode(LED_RED, OUTPUT);

digitalWrite(LED_GREEN, LOW);
digitalWrite(LED_YELLOW, LOW);
digitalWrite(LED_ORANGE, LOW);
digitalWrite(LED_RED, LOW);

// ================================================
// RELAY
// ================================================

pinMode(RELAY_PIN, OUTPUT);

digitalWrite(RELAY_PIN, HIGH);

relayState = true;

lastRelayChangeTime = millis();

// ================================================
// BUZZER
// ================================================

pinMode(BUZZER_PIN, OUTPUT);

digitalWrite(BUZZER_PIN, LOW);

// ================================================
// I2C
// ================================================

Wire.begin(SDA_PIN, SCL_PIN);
```

```
// ================================================
// LCD
// ================================================

lcd.init();

lcd.backlight();

lcd.clear();

lcd.setCursor(0, 0);
lcd.print("EV BMS TASK 4");

lcd.setCursor(0, 1);
lcd.print("Initializing...");

// ================================================
// INITIAL READING
// ================================================

readCellVoltages();

calculateBatteryParameters();

identifyStrongestWeakest();

calculateImbalance();

classifyBatteryHealth();

for (int i = 0; i < NUM_CELLS; i++)
{
  previousCellVoltage[i] = cellVoltage[i];

  previousADC[i] = cellADC[i];

  frozenCount[i] = 0;
```

```
    cellSensorHealthy[i] = true;
  }

  updateLEDs();

  displayNormalLCD();

  Serial.println();
  Serial.println("================================================");
  Serial.println("    EV BMS TASK 1 + 2 + 3 + 4 READY");
  Serial.println("================================================");
}

// ====================================================
// MAIN LOOP
// ====================================================

void loop()
{
  handleCloudTestButton();

  unsigned long now = millis();

  if (WiFi.status() == WL_CONNECTED)
  {
    if (!cloudTestMode && Blynk.connected())
    {
      Blynk.run();
    }
  }

  // TASK 5: network/cloud manager.
  // Runs independently of battery protection logic.
  processCloudTelemetry();

  // ====================================================
  // BATTERY + SAFETY + RUNTIME
  // ====================================================
```

```
if (now - previousSampleTime >= SAMPLE_INTERVAL)
{
  previousSampleTime = now;

  // ---------------------------------------------------
  // TASK 1
  // ---------------------------------------------------

  readCellVoltages();

  calculateBatteryParameters();

  identifyStrongestWeakest();

  calculateImbalance();

  classifyBatteryHealth();

  // ---------------------------------------------------
  // TASK 2
  // ---------------------------------------------------

  detectSafetyConditions();

  updateSafetyState();

  updateRelay();

  updateBuzzer();

  // ---------------------------------------------------
  // TASK 4
  // ---------------------------------------------------

  checkSensorHealth();

  checkFrozenADC();
```

```
  checkRelayMismatch();

  updateRuntimeMode();

  updateLEDs();

  // ---------------------------------------------------
  // FAULT EVENT LOGGING
  // ---------------------------------------------------

  processFaultEvents();
}

// ===================================================
// TASK 3 - AUTOMATIC PAGE ROTATION
// ===================================================

if (now - previousLCDPageTime >= LCD_PAGE_INTERVAL)
{
  previousLCDPageTime = now;

  // Do not rotate normal pages during critical faults

  if (!faultDetected &&
     runtimeMode == RUNTIME_NORMAL)
  {
    currentPage++;

    if (currentPage >= TOTAL_PAGES)
    {
      currentPage = 0;
    }

    previousPage = -1;
  }
}
```

```
  // ================================================
  // LCD REFRESH
  // ================================================

  if (now - previousLCDRefreshTime >= LCD_REFRESH_INTERVAL)
  {
   previousLCDRefreshTime = now;

   updateLCD();
  }

  // ================================================
  // SERIAL
  // ================================================

  if (now - previousSerialTime >= SERIAL_INTERVAL)
  {
   previousSerialTime = now;

   displaySerialReport();
  }
}

// ================================================
// TASK 1
// READ CELL VOLTAGES
// ================================================

void readCellVoltages()
{
  cellADC[0] = analogRead(CELL1_PIN);

  cellADC[1] = analogRead(CELL2_PIN);

  cellADC[2] = analogRead(CELL3_PIN);

  cellADC[3] = analogRead(CELL4_PIN);
```

```
 for (int i = 0; i < NUM_CELLS; i++)
 {
   cellVoltage[i] =
     convertADCToCellVoltage(cellADC[i]);
 }
}

// ======================================================
// TASK 1
// ADC TO VOLTAGE
// ======================================================

float convertADCToCellVoltage(int rawADC)
{
 float adcVoltage =
   ((float)rawADC / ADC_MAX)
   * ADC_MAX_VOLTAGE;

 float voltage =
   CELL_MIN_VOLTAGE
   +
   (adcVoltage / ADC_MAX_VOLTAGE)
   *
   (CELL_MAX_VOLTAGE -
    CELL_MIN_VOLTAGE);

 return voltage;
}

// ======================================================
// TASK 1
// BATTERY PARAMETERS
// ======================================================

void calculateBatteryParameters()
{
  packVoltage = 0.0;
```

```
  for (int i = 0; i < NUM_CELLS; i++)
  {
    packVoltage += cellVoltage[i];
  }

  averageVoltage =
    packVoltage / NUM_CELLS;

  maxVoltage = cellVoltage[0];

  minVoltage = cellVoltage[0];

  for (int i = 1; i < NUM_CELLS; i++)
  {
    if (cellVoltage[i] > maxVoltage)
    {
      maxVoltage = cellVoltage[i];
    }

    if (cellVoltage[i] < minVoltage)
    {
      minVoltage = cellVoltage[i];
    }
  }
}

// ======================================================
// TASK 1
// STRONGEST / WEAKEST
// ======================================================

void identifyStrongestWeakest()
{
  strongestCell = 0;

  weakestCell = 0;

  for (int i = 1; i < NUM_CELLS; i++)
```

```
 {
  if (cellVoltage[i] >
     cellVoltage[strongestCell])
  {
    strongestCell = i;
  }

  if (cellVoltage[i] <
     cellVoltage[weakestCell])
  {
    weakestCell = i;
  }
 }
}

// ====================================================
// TASK 1
// IMBALANCE
// ====================================================

void calculateImbalance()
{
 imbalanceVoltage =
  maxVoltage - minVoltage;

 if (averageVoltage > 0)
 {
  imbalancePercent =
    (imbalanceVoltage /
     averageVoltage)
    * 100.0;
 }
 else
 {
  imbalancePercent = 0.0;
 }
}
```

```
// ====================================================
// TASK 1
// HEALTH CLASSIFICATION
// ====================================================

void classifyBatteryHealth()
{
 if (imbalancePercent < 3.0)
 {
  batteryHealth = "HEALTHY";
 }

 else if (imbalancePercent < 10.0)
 {
  batteryHealth = "MINOR IMBALANCE";
 }

 else
 {
  batteryHealth = "CRITICAL IMBALANCE";
 }
}

// ====================================================
// TASK 2
// SAFETY DETECTION
// ====================================================

void detectSafetyConditions()
{
 underVoltageFault = false;

 overVoltageFault = false;

 rapidChangeFault = false;

 sensorFault = false;
```

```
for (int i = 0; i < NUM_CELLS; i++)
{
 // ------------------------------------------------
 // Under Voltage
 // ------------------------------------------------

 if (cellVoltage[i] < UV_THRESHOLD)
 {
  underVoltageFault = true;
 }

 // ------------------------------------------------
 // Over Voltage
 // ------------------------------------------------

 if (cellVoltage[i] > OV_THRESHOLD)
 {
  overVoltageFault = true;
 }

 // ------------------------------------------------
 // Rapid Voltage Change
 // ------------------------------------------------

 float change =
  fabs(cellVoltage[i] -
      previousCellVoltage[i]);

 if (change > RAPID_CHANGE_THRESHOLD)
 {
  rapidChangeFault = true;
 }

 // ------------------------------------------------
 // Invalid sensor
 // ------------------------------------------------

 if (cellADC[i] <= 5 ||
```

```
      cellADC[i] >= 4090)
    {
      sensorFault = true;
    }
  }

  // Store previous voltages

  for (int i = 0; i < NUM_CELLS; i++)
  {
    previousCellVoltage[i] =
      cellVoltage[i];
  }

  faultDetected =
    underVoltageFault ||
    overVoltageFault ||
    rapidChangeFault ||
    sensorFault;
}

// ======================================================
// TASK 2
// SAFETY STATE MACHINE
// ======================================================


  switch (safetyState)
  {
    case SAFETY_NORMAL:

      if (faultDetected)
      {
        safetyState = SAFETY_WARNING;

        faultStartTime = now;
      }
```

```
 break;

case SAFETY_WARNING:

 if (!faultDetected)
 {
  safetyState = SAFETY_NORMAL;

  faultConfirmed = false;

  break;
 }

 if (now - faultStartTime >=
    FAULT_CONFIRM_TIME)
 {
  faultConfirmed = true;

  safetyState = SAFETY_TRIPPED;
 }

 break;

case SAFETY_TRIPPED:

 if (!faultDetected)
 {
  safetyRecoveryStart = now;

  safetyState = SAFETY_RECOVERY;
 }

 break;

case SAFETY_RECOVERY:

 if (faultDetected)
 {
```

```
      safetyState = SAFETY_TRIPPED;

      break;
    }

    if (now - safetyRecoveryStart >=
      SAFETY_RECOVERY_TIME)
    {
      safetyState = SAFETY_NORMAL;

      faultConfirmed = false;
    }

    break;
  }
}

// ======================================================
// TASK 2
// RELAY CONTROL
// ======================================================

void updateRelay()
{
  unsigned long now = millis();

  // ------------------------------------------------
  // SAFETY TRIP
  // ------------------------------------------------

  if (safetyState == SAFETY_WARNING ||
    safetyState == SAFETY_TRIPPED)
  {
    if (relayState)
    {
      if (now - lastRelayChangeTime >=
        RELAY_LOCK_TIME)
      {
```

```
      digitalWrite(RELAY_PIN, LOW);

      relayState = false;

      lastRelayChangeTime = now;

      Serial.println(">>> RELAY CUT-OFF <<<");
    }
   }
  }

  // ------------------------------------------------
  // RECOVERY
  // ------------------------------------------------

  else if (safetyState ==
        SAFETY_NORMAL)
  {
   if (!relayState)
   {
    if (now - lastRelayChangeTime >=
       RELAY_LOCK_TIME)
    {
      digitalWrite(RELAY_PIN, HIGH);

      relayState = true;

      lastRelayChangeTime = now;

      Serial.println(">>> RELAY RESTORED <<<");
    }
   }
  }
 }

 // ====================================================
 // TASK 2
 // BUZZER
```

```
// ====================================================

void updateBuzzer()
{
  unsigned long now = millis();

  if (safetyState == SAFETY_WARNING ||
     safetyState == SAFETY_TRIPPED)
  {
    if (now - previousBuzzerTime >=
       BUZZER_INTERVAL)
    {
      previousBuzzerTime = now;

      digitalWrite(
        BUZZER_PIN,
        !digitalRead(BUZZER_PIN)
      );
    }
  }

  else
  {
    digitalWrite(BUZZER_PIN, LOW);
  }
}

// ====================================================
// TASK 4
// SENSOR HEALTH
// ====================================================

void checkSensorHealth()
{
  invalidSensorFault = false;

  for (int i = 0; i < NUM_CELLS; i++)
  {
```

```
    if (cellADC[i] <= 5 ||
        cellADC[i] >= 4090)
    {
      invalidSensorFault = true;

      cellSensorHealthy[i] = false;
    }
    else
    {
      cellSensorHealthy[i] = true;
    }
  }
}

// ======================================================
// TASK 4
// FROZEN ADC
//
// IMPORTANT:
// A stable battery voltage is NOT automatically
// considered an ADC fault.
//
// Frozen ADC is activated only when:
// simulateFrozenADC = true
// ======================================================

void checkFrozenADC()
{
  frozenADCFault = false;

  if (!simulateFrozenADC)
  {
    for (int i = 0; i < NUM_CELLS; i++)
    {
      frozenCount[i] = 0;

      previousADC[i] =
        cellADC[i];
```

```
  }

  return;
 }

 // Diagnostic simulation enabled

 for (int i = 0; i < NUM_CELLS; i++)
 {
  if (cellADC[i] ==
     previousADC[i])
  {
   frozenCount[i]++;
  }
  else
  {
   frozenCount[i] = 0;
  }

  if (frozenCount[i] >=
     FROZEN_LIMIT)
  {
   frozenADCFault = true;

   frozenCount[i] =
    FROZEN_LIMIT;
  }

  previousADC[i] =
   cellADC[i];
 }
}

// ====================================================
// TASK 4
// RELAY MISMATCH
// ====================================================
```

```
void checkRelayMismatch()
{
 relayMismatchFault =
   simulateRelayMismatch;
}

// ======================================================
// TASK 4
// RUNTIME STATE MACHINE
// ======================================================

void updateRuntimeMode()
{
 unsigned long now = millis();

 // ================================================
 // SHUTDOWN
 // ================================================

 if (simulateShutdown)
 {
  if (runtimeMode !=
     RUNTIME_SHUTDOWN)
  {
   logFault(
    "SYSTEM SHUTDOWN",
    "SHUTDOWN"
   );
  }

  runtimeMode =
   RUNTIME_SHUTDOWN;

  digitalWrite(RELAY_PIN, LOW);

  relayState = false;

  return;
```

```
}

// ================================================
// FAILSAFE
// ================================================

if (relayMismatchFault ||
   overVoltageFault ||
   underVoltageFault)
{
 if (runtimeMode !=
    RUNTIME_FAILSAFE)
 {
   logFault(
    "CRITICAL SAFETY FAULT",
    "FAILSAFE"
   );
 }

 runtimeMode =
   RUNTIME_FAILSAFE;

 runtimeRecoveryStart = 0;

 return;
}

// ================================================
// DEGRADED
// ================================================

if (invalidSensorFault ||
   frozenADCFault ||
   rapidChangeFault)
{
 if (runtimeMode ==
    RUNTIME_NORMAL)
 {
```

```
    runtimeRecoveryStart = 0;

    logFault(
      "SYSTEM DEGRADED",
      "DEGRADED"
    );
  }

  runtimeMode =
    RUNTIME_DEGRADED;

  return;
}

// ================================================
// RECOVERY FROM FAILSAFE
// ================================================

if (runtimeMode ==
    RUNTIME_FAILSAFE)
{
  if (!underVoltageFault &&
      !overVoltageFault &&
      !relayMismatchFault)
  {
    if (runtimeRecoveryStart == 0)
    {
      runtimeRecoveryStart = now;
    }

    if (now - runtimeRecoveryStart >=
        RUNTIME_RECOVERY_TIME)
    {
      runtimeMode =
        RUNTIME_NORMAL;

      runtimeRecoveryStart = 0;
```

```
      logFault(
        "FAILSAFE RECOVERED",
        "NORMAL"
      );
    }
   }
   else
   {
    runtimeRecoveryStart = 0;
   }

   return;
}

// ================================================
// RECOVERY FROM DEGRADED
// ================================================

if (runtimeMode ==
    RUNTIME_DEGRADED)
{
  if (!invalidSensorFault &&
      !frozenADCFault &&
      !rapidChangeFault)
  {
    if (runtimeRecoveryStart == 0)
    {
      runtimeRecoveryStart = now;
    }

    if (now - runtimeRecoveryStart >=
        RUNTIME_RECOVERY_TIME)
    {
      runtimeMode =
        RUNTIME_NORMAL;

      runtimeRecoveryStart = 0;
```

```
      logFault(
        "DEGRADED RECOVERED",
        "NORMAL"
      );
    }
   }
   else
   {
    runtimeRecoveryStart = 0;
   }

   return;
  }

  // ================================================
  // NORMAL
  // ================================================

  if (!invalidSensorFault &&
     !frozenADCFault &&
     !relayMismatchFault &&
     !underVoltageFault &&
     !overVoltageFault &&
     !rapidChangeFault)
  {
   runtimeMode =
     RUNTIME_NORMAL;
  }
 }

// ======================================================
// TASK 4
// FAULT EVENT PROCESSING
// ======================================================

void processFaultEvents()
{
  // ------------------------------------------------
```

```
// Invalid sensor event
// ------------------------------------------------

if (invalidSensorFault &&
    !previousInvalidSensorFault)
{
  logFault(
    "INVALID SENSOR",
    "DEGRADED"
  );
}

// ------------------------------------------------
// Frozen ADC event
// ------------------------------------------------

if (frozenADCFault &&
    !previousFrozenADCFault)
{
  logFault(
    "ADC FROZEN",
    "DEGRADED"
  );
}

// ------------------------------------------------
// Relay mismatch
// ------------------------------------------------

if (relayMismatchFault &&
    !previousRelayMismatchFault)
{
  logFault(
    "RELAY MISMATCH",
    "FAILSAFE"
  );
}
```

```
// ------------------------------------------------
// Under voltage
// ------------------------------------------------

if (underVoltageFault &&
    !previousUnderVoltageFault)
{
  logFault(
    "UNDER VOLTAGE",
    "FAILSAFE"
  );
}

// ------------------------------------------------
// Over voltage
// ------------------------------------------------

if (overVoltageFault &&
    !previousOverVoltageFault)
{
  logFault(
    "OVER VOLTAGE",
    "FAILSAFE"
  );
}

// ------------------------------------------------
// Rapid voltage change
// ------------------------------------------------

if (rapidChangeFault &&
    !previousRapidChangeFault)
{
  logFault(
    "RAPID VOLTAGE CHANGE",
    "DEGRADED"
  );
}
```

```
  // Save previous states

  previousInvalidSensorFault =
    invalidSensorFault;

  previousFrozenADCFault =
    frozenADCFault;

  previousRelayMismatchFault =
    relayMismatchFault;

  previousUnderVoltageFault =
    underVoltageFault;

  previousOverVoltageFault =
    overVoltageFault;

  previousRapidChangeFault =
    rapidChangeFault;
}

// ====================================================
// TASK 4
// FAULT LOG
// ====================================================

void logFault(String faultName,
              String mode)
{
  if (faultLogCount >=
      MAX_FAULT_LOGS)
  {
    return;
  }

  faultLogs[faultLogCount].timestamp =
    millis();
```

```
  faultLogs[faultLogCount].faultName =
    faultName;

  faultLogs[faultLogCount].mode =
    mode;

  // Task 5: preserve the fault event if cloud is offline.
  queueCloudEvent(faultName + " [" + mode + "]");

  faultLogCount++;

  Serial.println();
  Serial.println("******** FAULT EVENT ********");

  Serial.print("Time  : ");
  Serial.print(millis());
  Serial.println(" ms");

  Serial.print("Fault : ");
  Serial.println(faultName);

  Serial.print("Mode  : ");
  Serial.println(mode);

  Serial.println("******************************");
}

// ====================================================
// LED CONTROL
// ====================================================

void updateLEDs()
{
  digitalWrite(LED_GREEN, LOW);

  digitalWrite(LED_YELLOW, LOW);
```

```
digitalWrite(LED_ORANGE, LOW);

digitalWrite(LED_RED, LOW);

// ------------------------------------------------
// SHUTDOWN
// ------------------------------------------------

if (runtimeMode ==
  RUNTIME_SHUTDOWN)
{
 digitalWrite(LED_RED, HIGH);

 return;
}

// ------------------------------------------------
// FAILSAFE
// ------------------------------------------------

if (runtimeMode ==
  RUNTIME_FAILSAFE)
{
 digitalWrite(LED_RED, HIGH);

 return;
}

// ------------------------------------------------
// DEGRADED
// ------------------------------------------------

if (runtimeMode ==
  RUNTIME_DEGRADED)
{
 digitalWrite(LED_YELLOW, HIGH);

 return;
```

```
  }

  // ------------------------------------------------
  // NORMAL
  // ------------------------------------------------

  if (batteryHealth ==
     "HEALTHY")
  {
   digitalWrite(LED_GREEN, HIGH);
  }

  else if (batteryHealth ==
        "MINOR IMBALANCE")
  {
   digitalWrite(LED_YELLOW, HIGH);
  }

  else
  {
   digitalWrite(LED_ORANGE, HIGH);
  }
}

// ======================================================
// TASK 3
// LCD MANAGER
// ======================================================

void updateLCD()
{
  // ================================================
  // SHUTDOWN PRIORITY
  // ================================================

  if (runtimeMode ==
     RUNTIME_SHUTDOWN)
  {
```

```
  displayShutdownLCD();

  return;
}

// ================================================
// FAILSAFE PRIORITY
// ================================================

if (runtimeMode ==
    RUNTIME_FAILSAFE)
{
  displayFailsafeLCD();

  return;
}

// ================================================
// DEGRADED PRIORITY
// ================================================

if (runtimeMode ==
    RUNTIME_DEGRADED)
{
  displayDegradedLCD();

  return;
}

// ================================================
// SAFETY FAULT PRIORITY
// ================================================

if (faultDetected)
{
  displayFaultLCD();

  return;
```

```
}

// ================================================
// NORMAL HMI PAGES
// ================================================

if (currentPage == previousPage)
{
 return;
}

previousPage =
 currentPage;

lcd.clear();

switch (currentPage)
{
 case 0:
  displayCellPage();
  break;

 case 1:
  displayPackPage();
  break;

 case 2:
  displayHealthPage();
  break;

 case 3:
  displaySafetyPage();
  break;

 case 4:
  displayDiagnosticPage();
  break;
}
```

```
}

// ======================================================
// LCD PAGE 1
// ======================================================

void displayCellPage()
{
  lcd.setCursor(0, 0);

  lcd.print("C1:");
  lcd.print(cellVoltage[0], 2);

  lcd.print(" C2:");
  lcd.print(cellVoltage[1], 2);

  lcd.setCursor(0, 1);

  lcd.print("C3:");
  lcd.print(cellVoltage[2], 2);

  lcd.print(" C4:");
  lcd.print(cellVoltage[3], 2);
}

// ======================================================
// LCD PAGE 2
// ======================================================

void displayPackPage()
{
  lcd.setCursor(0, 0);

  lcd.print("PACK:");
  lcd.print(packVoltage, 2);
  lcd.print("V");

  lcd.setCursor(0, 1);
```

```
 lcd.print("AVG:");
 lcd.print(averageVoltage, 2);
 lcd.print("V");
}

// ====================================================
// LCD PAGE 3
// ====================================================

void displayHealthPage()
{
 lcd.setCursor(0, 0);

 lcd.print("IMB:");
 lcd.print(imbalancePercent, 1);
 lcd.print("%");

 lcd.setCursor(0, 1);

 lcd.print("W:C");
 lcd.print(weakestCell + 1);

 lcd.print(" S:C");
 lcd.print(strongestCell + 1);
}

// ====================================================
// LCD PAGE 4
// ====================================================

void displaySafetyPage()
{
 lcd.setCursor(0, 0);

 lcd.print("SAFETY:");

 if (safetyState ==
```

```
    SAFETY_NORMAL)
  {
   lcd.print("NORMAL");
  }

  else if (safetyState ==
        SAFETY_WARNING)
  {
   lcd.print("WARNING");
  }

  else if (safetyState ==
        SAFETY_TRIPPED)
  {
   lcd.print("TRIPPED");
  }

  else
  {
   lcd.print("RECOVERY");
  }

  lcd.setCursor(0, 1);

  lcd.print("RELAY:");

  if (relayState)
  {
   lcd.print("ON");
  }
  else
  {
   lcd.print("OFF");
  }
}

// =======================================================
// LCD PAGE 5
```

```
// =====================================================

void displayDiagnosticPage()
{
 lcd.setCursor(0, 0);

 lcd.print("SYSTEM:");

 if (runtimeMode ==
    RUNTIME_NORMAL)
 {
  lcd.print("NORMAL");
 }

 else if (runtimeMode ==
       RUNTIME_DEGRADED)
 {
  lcd.print("DEGRADE");
 }

 else if (runtimeMode ==
       RUNTIME_FAILSAFE)
 {
  lcd.print("FAILSAFE");
 }

 else
 {
  lcd.print("SHUTDOWN");
 }

 lcd.setCursor(0, 1);

 lcd.print("NO ACTIVE FAULT");
}

// =====================================================
// LCD NORMAL
```

```
// ====================================================

void displayNormalLCD()
{
 lcd.clear();

 lcd.setCursor(0, 0);

 lcd.print("RUNTIME:NORMAL");

 lcd.setCursor(0, 1);

 lcd.print("SYSTEM OK");
}

// ====================================================
// LCD DEGRADED
// ====================================================

void displayDegradedLCD()
{
 lcd.clear();

 lcd.setCursor(0, 0);

 lcd.print("RUNTIME:DEGRADE");

 lcd.setCursor(0, 1);

 if (invalidSensorFault)
 {
  lcd.print("SENSOR FAULT");
 }

 else if (frozenADCFault)
 {
  lcd.print("ADC FROZEN");
 }
```

```
 else if (rapidChangeFault)
 {
  lcd.print("RAPID DV");
 }

 else
 {
  lcd.print("CHECK SYSTEM");
 }
}

// ====================================================
// LCD FAILSAFE
// ====================================================

void displayFailsafeLCD()
{
 lcd.clear();

 lcd.setCursor(0, 0);

 lcd.print("RUNTIME:FAILSAFE");

 lcd.setCursor(0, 1);

 lcd.print("RELAY:");

 if (relayState)
 {
  lcd.print("ON");
 }
 else
 {
  lcd.print("OFF");
 }
}
```

```
// ======================================================
// LCD SHUTDOWN
// ======================================================

void displayShutdownLCD()
{
  lcd.clear();

  lcd.setCursor(0, 0);

  lcd.print("SYSTEM SHUTDOWN");

  lcd.setCursor(0, 1);

  lcd.print("RELAY:OFF");
}

// ======================================================
// LCD FAULT
// ======================================================

void displayFaultLCD()
{
  lcd.clear();

  if (underVoltageFault)
  {
    lcd.setCursor(0, 0);

    lcd.print("FAULT:UNDER V");

    lcd.setCursor(0, 1);

    lcd.print("C");
    lcd.print(weakestCell + 1);

    lcd.print(":");
    lcd.print(cellVoltage[weakestCell], 2);
```

```
  lcd.print("V");
}

else if (overVoltageFault)
{
  lcd.setCursor(0, 0);

  lcd.print("FAULT:OVER V");

  lcd.setCursor(0, 1);

  lcd.print("CHECK CELL");
}

else if (rapidChangeFault)
{
  lcd.setCursor(0, 0);

  lcd.print("FAULT:RAPID DV");

  lcd.setCursor(0, 1);

  lcd.print("CHECK BATTERY");
}

else if (sensorFault)
{
  lcd.setCursor(0, 0);

  lcd.print("FAULT:SENSOR");

  lcd.setCursor(0, 1);

  lcd.print("SAFE MODE");
}

else
{
```

```
  lcd.setCursor(0, 0);

  lcd.print("FAULT DETECTED");

  lcd.setCursor(0, 1);

  lcd.print("CHECK SYSTEM");
 }
}

// ==================================================
// SERIAL REPORT
// ==================================================

void displaySerialReport()
{
 Serial.println();

 Serial.println(
  "================================================"
 );

 Serial.println(
  "     EV BMS TASK 1 + 2 + 3 + 4"
 );

 Serial.println(
  "================================================"
 );

 // ================================================
 // BATTERY
 // ================================================

 Serial.println("BATTERY PARAMETERS");

 Serial.println(
  "------------------------------------------------"
```

```
);

for (int i = 0; i < NUM_CELLS; i++)
{
  Serial.print("Cell ");
  Serial.print(i + 1);
  Serial.print(" Voltage     ");
  Serial.print(cellVoltage[i], 2);
  Serial.println(" V");
}

Serial.print("Pack Voltage          ");
Serial.print(packVoltage, 2);
Serial.println(" V");

Serial.print("Average Voltage       ");
Serial.print(averageVoltage, 2);
Serial.println(" V");

Serial.print("Imbalance             ");
Serial.print(imbalancePercent, 2);
Serial.println(" %");

Serial.print("Strongest Cell        C");
Serial.println(strongestCell + 1);

Serial.print("Weakest Cell          C");
```

```
Serial.println(weakestCell + 1);

Serial.print("Battery Health         ");

Serial.println(batteryHealth);

// ================================================
// SAFETY
// ================================================

Serial.println();

Serial.println("SAFETY STATUS");

Serial.println(
  "------------------------------------------------"
);

Serial.print("Under Voltage          ");

Serial.println(
  underVoltageFault ? "YES" : "NO"
);

Serial.print("Over Voltage           ");

Serial.println(
  overVoltageFault ? "YES" : "NO"
);

Serial.print("Rapid Voltage Change     ");

Serial.println(
  rapidChangeFault ? "YES" : "NO"
);

Serial.print("Safety State           ");
```

```
printSafetyState();

Serial.print("Relay                ");

Serial.println(
 relayState ? "ON" : "OFF"
);

// ================================================
// TASK 4
// ================================================

Serial.println();

Serial.println("FAULT-TOLERANT RUNTIME");

Serial.println(
 "------------------------------------------------"
);

Serial.print("Runtime Mode          ");

printRuntimeMode();

Serial.print("Invalid Sensor        ");

Serial.println(
 invalidSensorFault ? "YES" : "NO"
);

Serial.print("Frozen ADC            ");

Serial.println(
 frozenADCFault ? "YES" : "NO"
);

Serial.print("Relay Mismatch        ");
```

```
Serial.println(
 relayMismatchFault ? "YES" : "NO"
);

// ================================================
// FAULT LOG
// ================================================

Serial.println();

Serial.println("FAULT LOG");

Serial.println(
 "------------------------------------------------"
);

if (faultLogCount == 0)
{
 Serial.println("No faults logged");
}
else
{
 for (int i = 0;
     i < faultLogCount;
     i++)
 {
  Serial.print("#");

  Serial.print(i + 1);

  Serial.print("  ");

  Serial.print(
   faultLogs[i].timestamp
  );

  Serial.print(" ms  ");
```

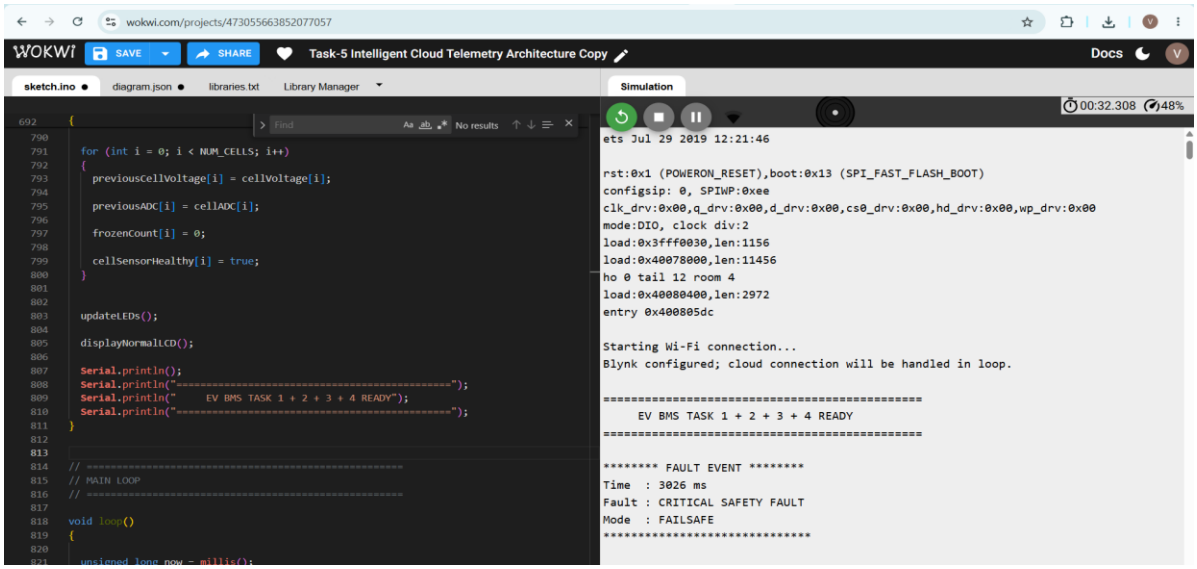
```
      Serial.print(
        faultLogs[i].faultName
      );

      Serial.print("  [");

      Serial.print(
        faultLogs[i].mode
      );

      Serial.println("]");
    }
  }

  Serial.println();
  Serial.println("CLOUD TELEMETRY");
  Serial.println("------------------------------------------------");
  Serial.print("WiFi Status            ");
  Serial.println(WiFi.status() == WL_CONNECTED ? "CONNECTED" : "OFFLINE");
  Serial.print("Blynk Status           ");
  Serial.println(Blynk.connected() ? "CONNECTED" : "OFFLINE");
  if (WiFi.status() == WL_CONNECTED)
  {
    Serial.print("WiFi RSSI              ");
    Serial.print(WiFi.RSSI());
    Serial.println(" dBm");
  }
  Serial.print("Queued Cloud Events     ");
  Serial.println(cloudEventCount);

  Serial.println(
    "================================================="
  );
}

// ======================================================
// PRINT SAFETY STATE
```

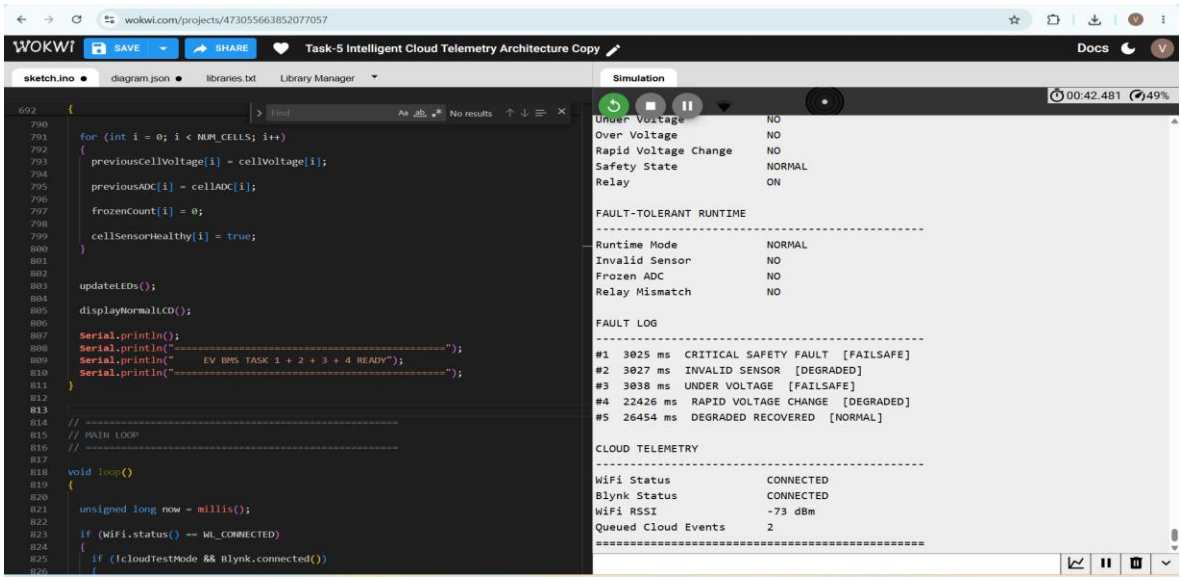
```
// ====================================================

void printSafetyState()
{
 if (safetyState ==
    SAFETY_NORMAL)
 {
  Serial.println("NORMAL");
 }

 else if (safetyState ==
       SAFETY_WARNING)
 {
  Serial.println("WARNING");
 }

 else if (safetyState ==
       SAFETY_TRIPPED)
 {
  Serial.println("TRIPPED");
 }

 else
 {
  Serial.println("RECOVERY");
 }
}

// ====================================================
// PRINT RUNTIME MODE
// ====================================================

void printRuntimeMode()
{
 if (runtimeMode ==
    RUNTIME_NORMAL)
 {
  Serial.println("NORMAL");
```

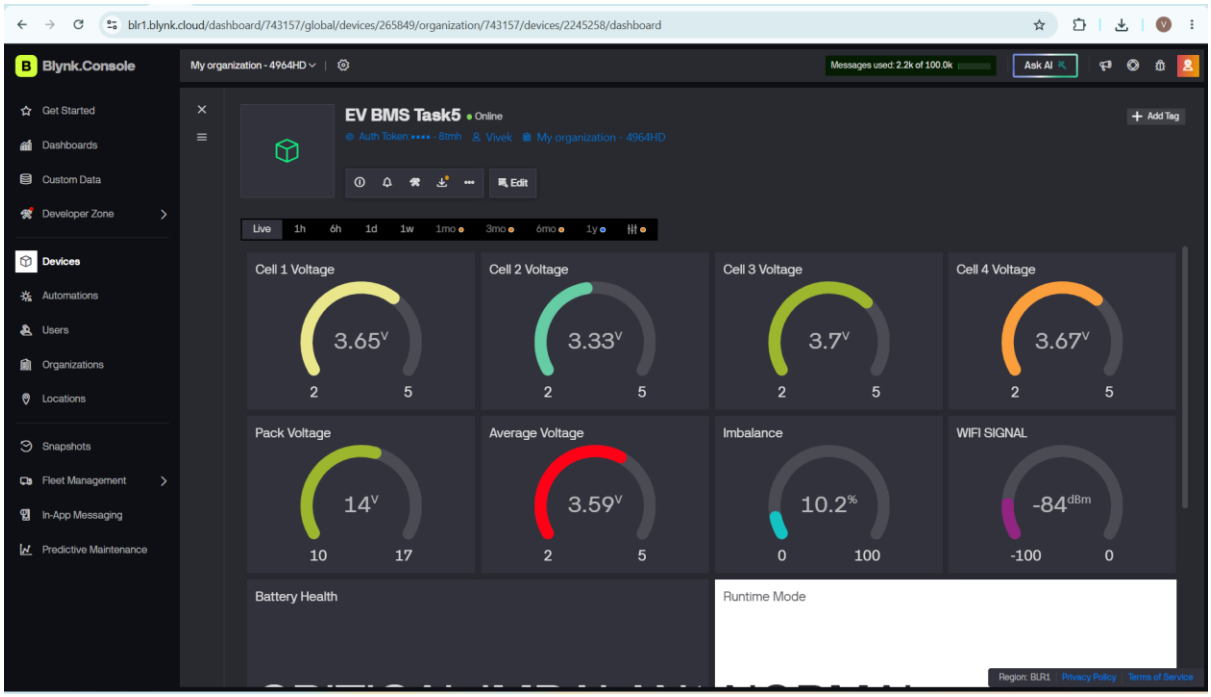
```
}

else if (runtimeMode ==
      RUNTIME_DEGRADED)
{
 Serial.println("DEGRADED");
}

else if (runtimeMode ==
      RUNTIME_FAILSAFE)
{
 Serial.println("FAILSAFE");
}

else
{
 Serial.println("SHUTDOWN");
}
}
```

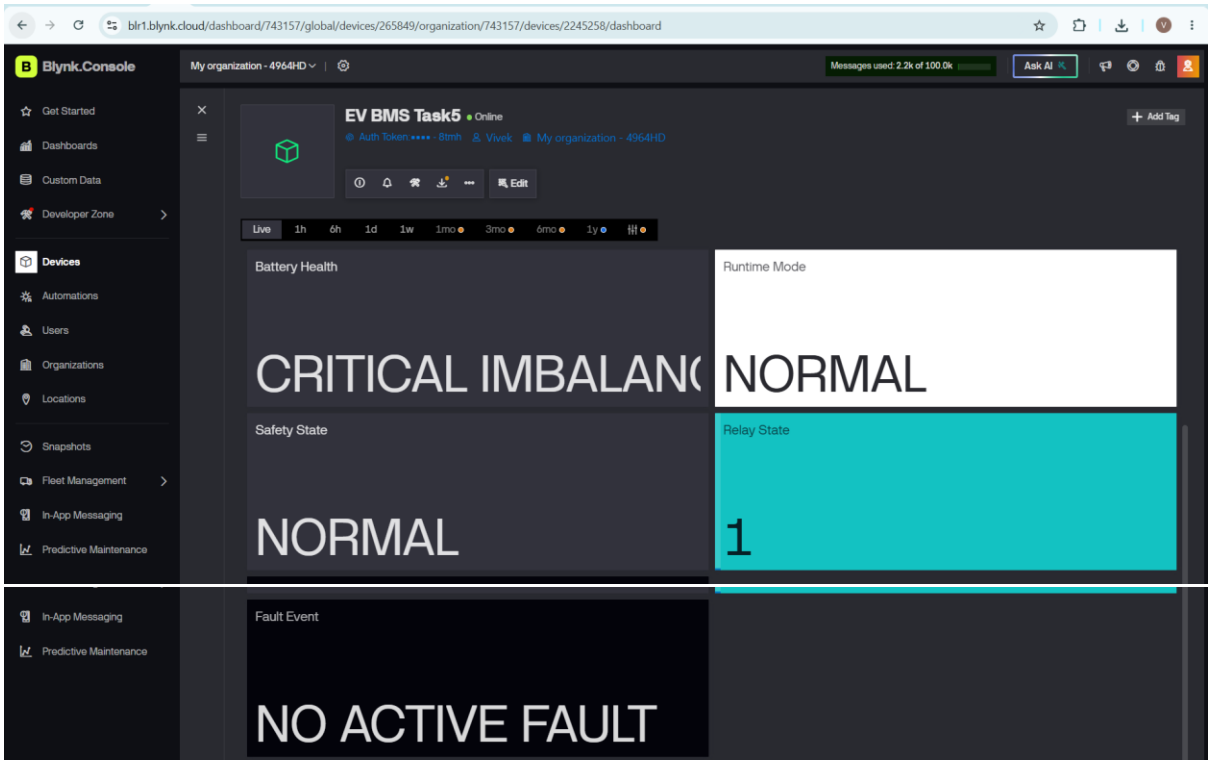
12.3 Simulation Running Photos with correct Output Of Blynk Dashboard:

1) Normal BMS and Blynk operation:

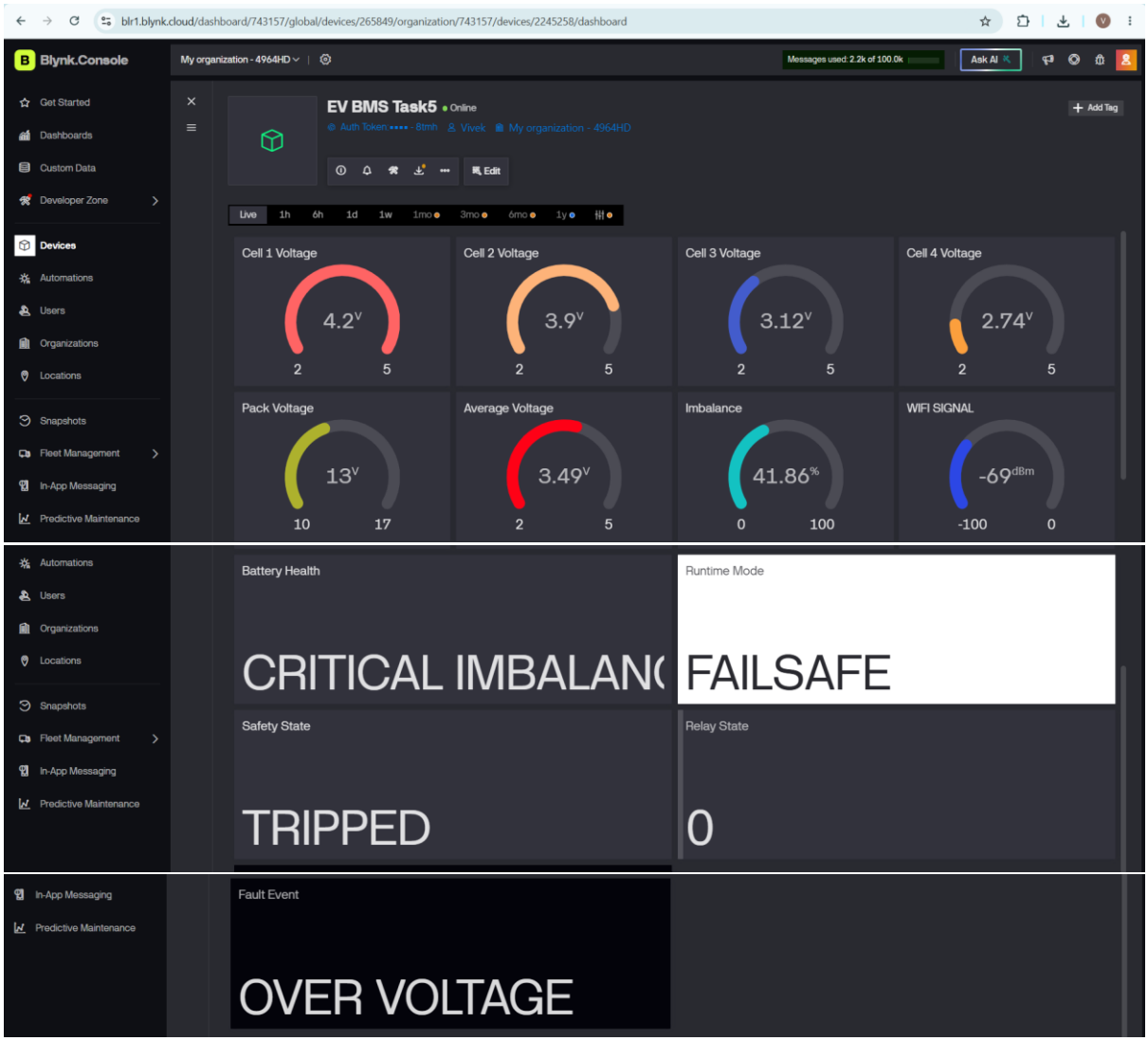



wokwi.com/projects/473055663852077057
WOKWI
SAVE
SHARE
Task-5 Intelligent Cloud Telemetry Architecture Copy
Docs
sketch.ino
diagram.json
libraries.txt
Library Manager
Simulation
00:42.481
49%
for (int i = 0; i < NUM_CELLS; i++)
previousCellVoltage[i] = cellVoltage[i];
previousADC[i] = cellADC[i];
frozenCount[i] = 0;
cellSensorHealthy[i] = true;
updateLEDs();
displayNormalLCD();
Serial.println("EV BMS TASK 1 + 2 + 3 + 4 READY");
// MAIN LOOP
void loop()
unsigned long now = millis();
if (WiFi.status() == WL_CONNECTED)
if (!cloudTestMode && Blynk.connected())
Under Voltage NO
Over Voltage NO
Rapid Voltage Change NO
Safety State NORMAL
Relay ON
FAULT-TOLERANT RUNTIME
Runtime Mode NORMAL
Invalid Sensor NO
Frozen ADC NO
Relay Mismatch NO
FAULT LOG
#1 3025 ms CRITICAL SAFETY FAULT [FAILSAFE]
#2 3027 ms INVALID SENSOR [DEGRADED]
#3 3038 ms UNDER VOLTAGE [FAILSAFE]
#4 22426 ms RAPID VOLTAGE CHANGE [DEGRADED]
#5 26454 ms DEGRADED RECOVERED [NORMAL]
CLOUD TELEMETRY
WiFi Status CONNECTED
Blynk Status CONNECTED
WiFi RSSI -73 dBm
Queued Cloud Events 2

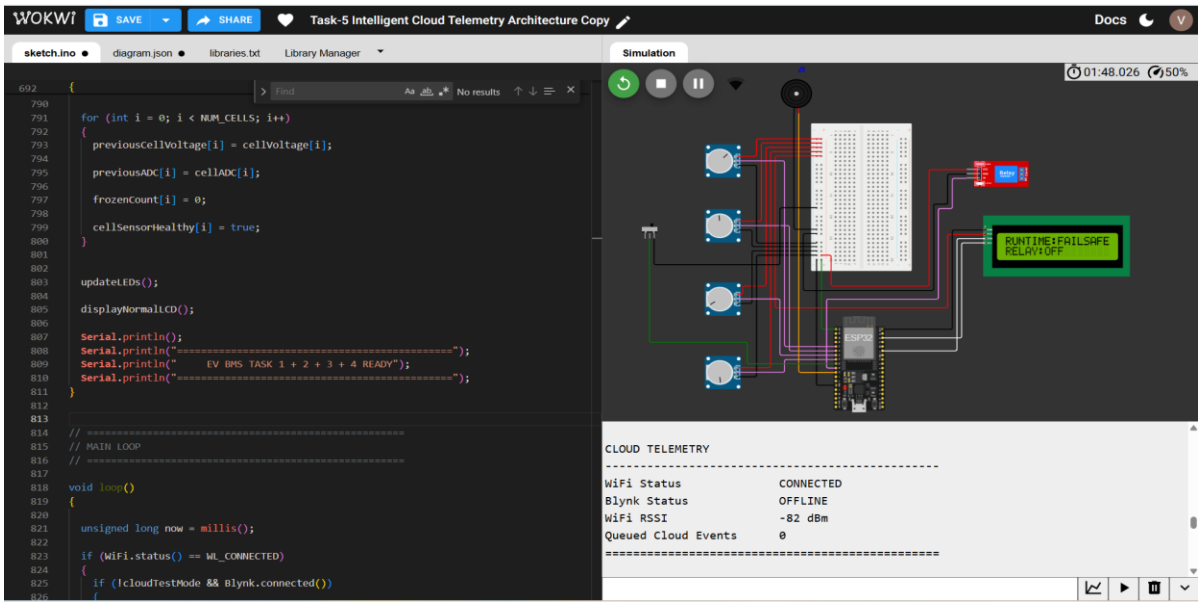

blr1.blynk.cloud/dashboard/743157/global/devices/265849/organization/743157/devices/2245258/dashboard
Blynk.Console
My organization - 4964HD
Messages used: 2.2k of 100.0k
Ask AI
Get Started
Dashboards
Custom Data
Developer Zone
Devices
Automations
Users
Organizations
Locations
Snapshots
Fleet Management
In-App Messaging
Predictive Maintenance
EV BMS Task5
Online
Add Tag
Edit
Live
1h
6h
1d
1w
Cell 1 Voltage
3.65V
Cell 2 Voltage
3.33V
Cell 3 Voltage
3.7V
Cell 4 Voltage
3.67V
Pack Voltage
14V
Average Voltage
3.59V
Imbalance
10.2%
WIFI SIGNAL
-84dBm
Battery Health
Runtime Mode
Region: BLR1
Privacy Policy
Terms of Service



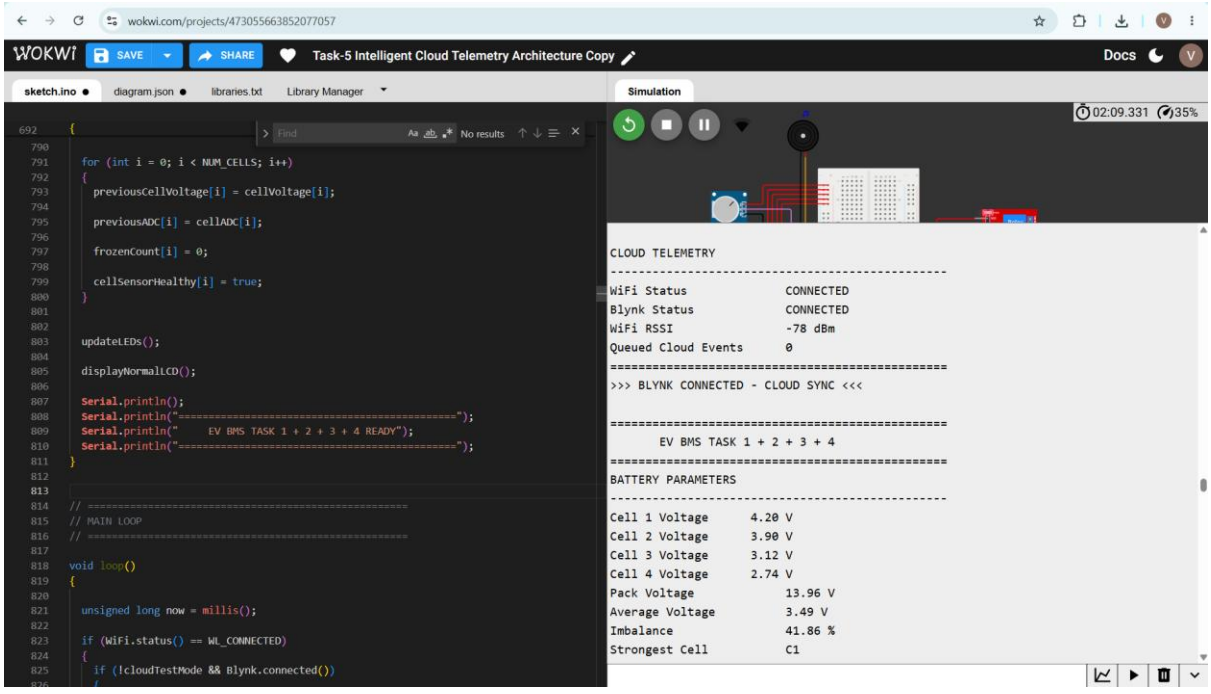
When I changed the values:

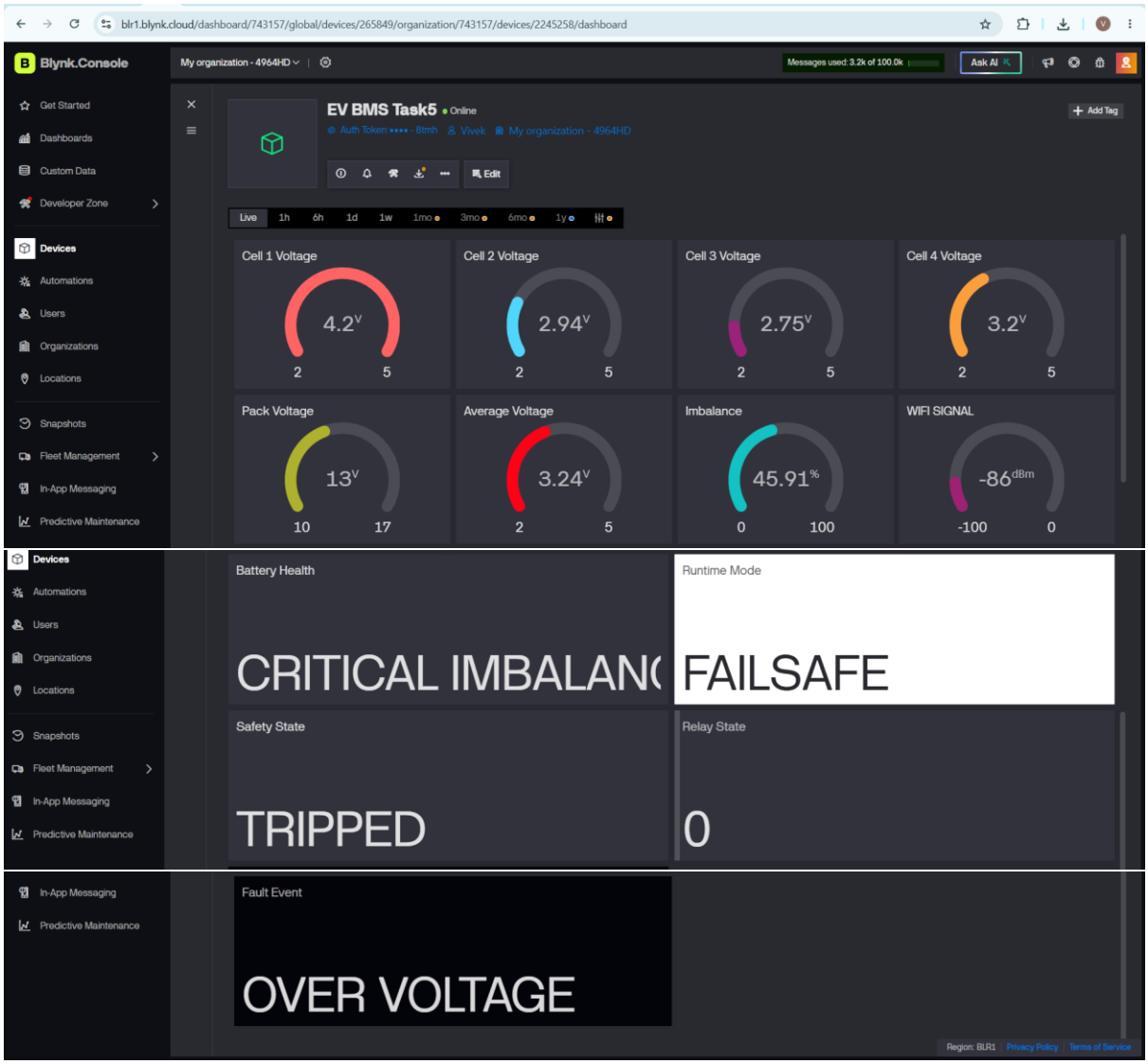


2) Blynk communication failure while Wi-Fi remains connected:

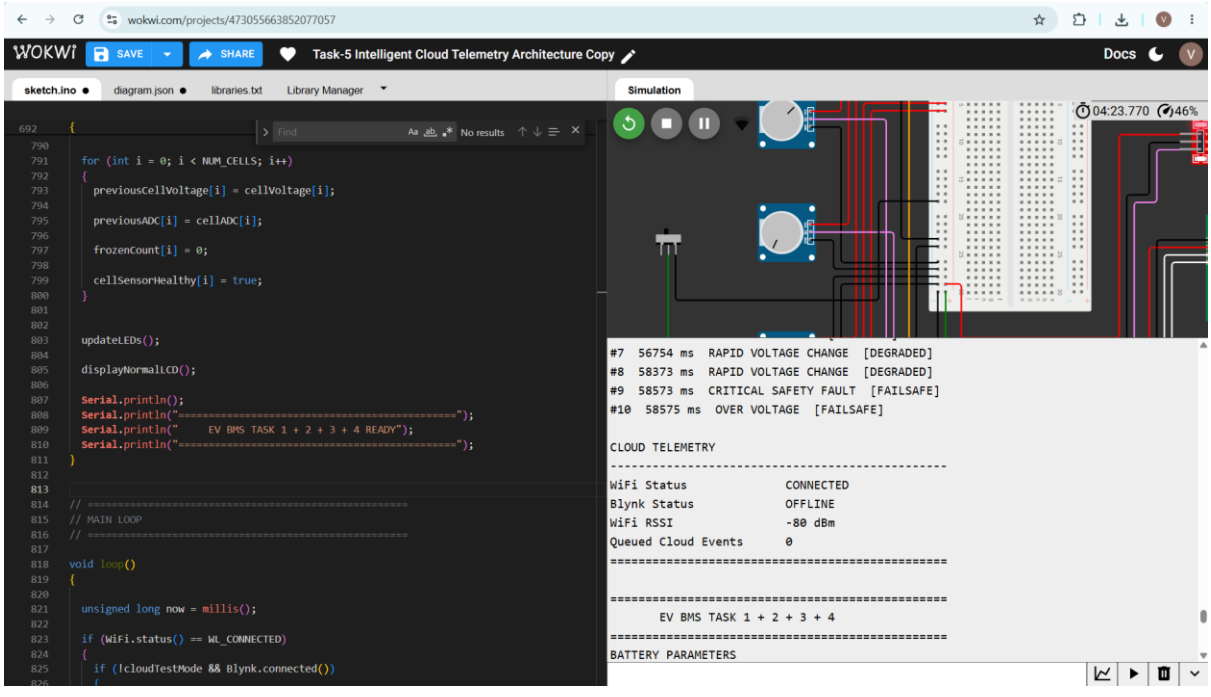


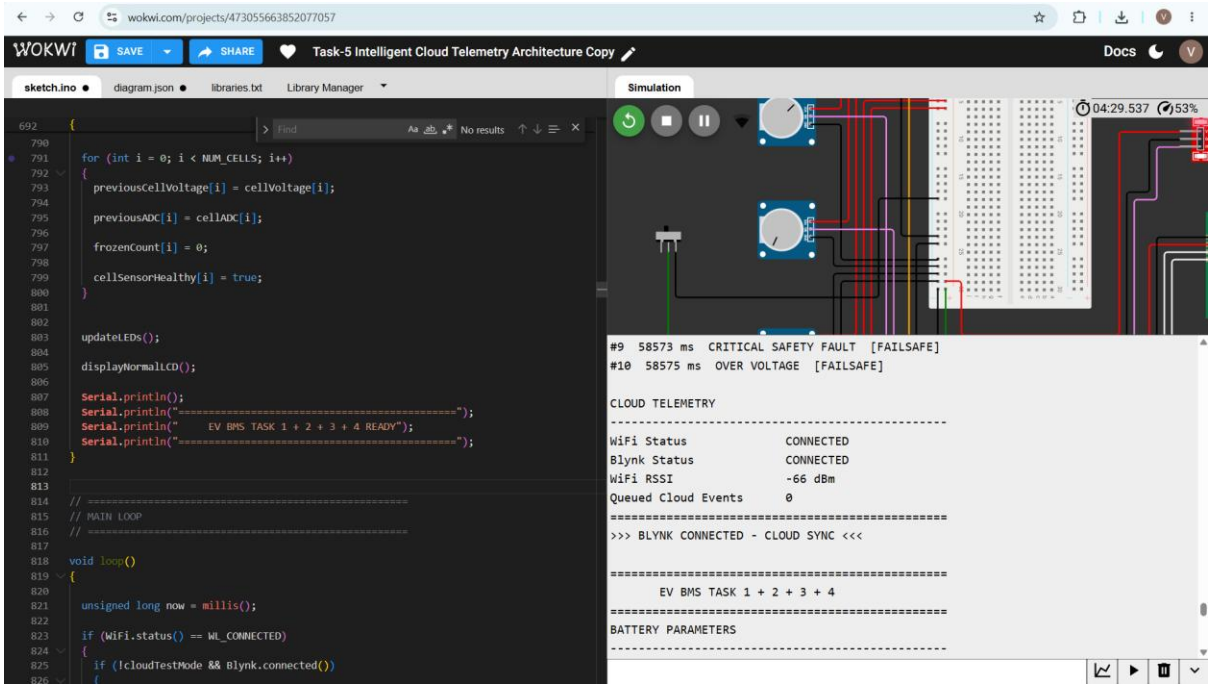
3) Blynk reconnection and cloud synchronization:

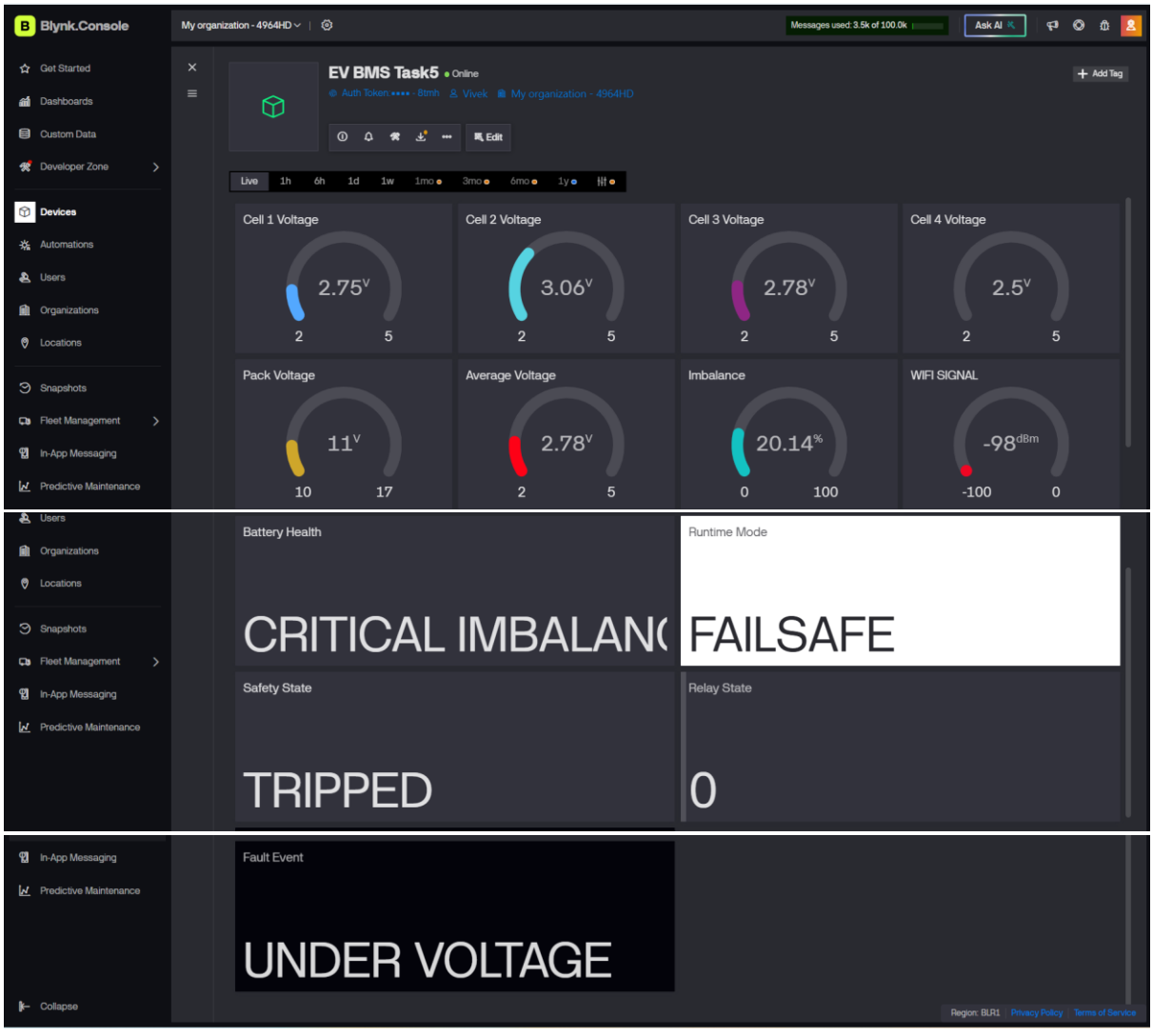



blr1.blynk.cloud/dashboard/743157/global/devices/265849/organization/743157/devices/2245258/dashboard
Blynk.Console
My organization - 4964HD
Messages used: 3.2k of 100.0k
Ask AI
Get Started
Dashboards
Custom Data
Developer Zone
Devices
Automations
Users
Organizations
Locations
Snapshots
Fleet Management
In-App Messaging
Predictive Maintenance
EV BMS Task5
Online
Add Tag
Edit
Live
1h
6h
1d
1w
1mo
3mo
6mo
1y
Cell 1 Voltage
4.2V
Cell 2 Voltage
2.94V
Cell 3 Voltage
2.75V
Cell 4 Voltage
3.2V
Pack Voltage
13V
Average Voltage
3.24V
Imbalance
45.91%
WIFI SIGNAL
-86dBm
Battery Health
CRITICAL IMBALAN(
Runtime Mode
FAILSAFE
Safety State
TRIPPED
Relay State
0
Fault Event
OVER VOLTAGE
Region: BLR1
Privacy Policy
Terms of Service

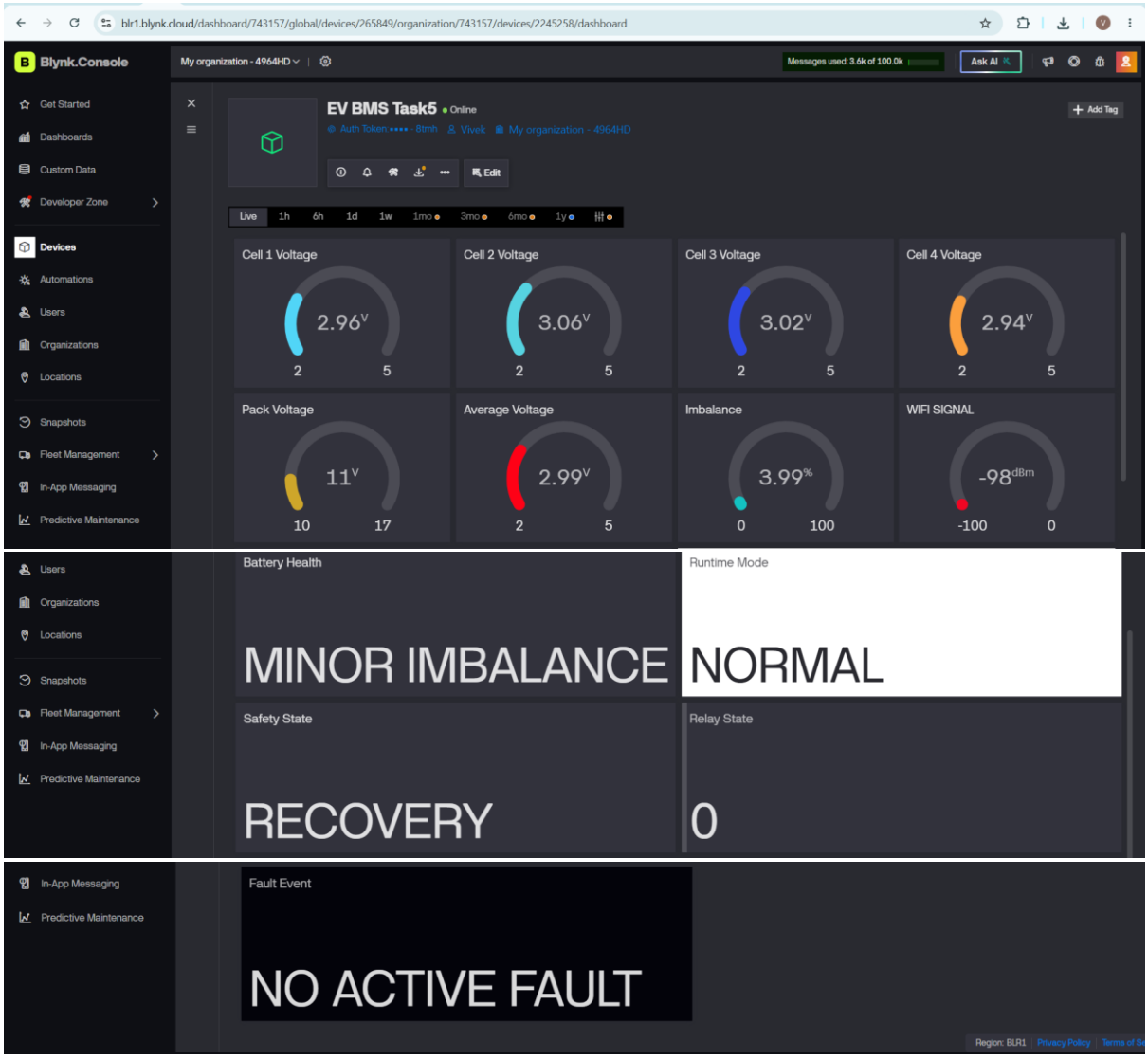
Example 2


wokwi.com/projects/473055663852077057
WOKWI
SAVE
SHARE
Task-5 Intelligent Cloud Telemetry Architecture Copy
Docs
sketch.ino
diagram.json
libraries.txt
Library Manager
Simulation
04:23.770
46%
for (int i = 0; i < NUM_CELLS; i++)
{
previousCellVoltage[i] = cellVoltage[i];
previousADC[i] = cellADC[i];
frozenCount[i] = 0;
cellSensorHealthy[i] = true;
}
updateLEDs();
displayNormalLCD();
Serial.println();
Serial.println("==========");
Serial.println("EV BMS TASK 1 + 2 + 3 + 4 READY");
Serial.println("==========");
}
// ==========
// MAIN LOOP
// ==========
void loop()
{
unsigned long now = millis();
if (WiFi.status() == WL_CONNECTED)
{
if (!cloudTestMode && Blynk.connected())
#7 56754 ms RAPID VOLTAGE CHANGE [DEGRADED]
#8 58373 ms RAPID VOLTAGE CHANGE [DEGRADED]
#9 58573 ms CRITICAL SAFETY FAULT [FAILSAFE]
#10 58575 ms OVER VOLTAGE [FAILSAFE]
CLOUD TELEMETRY
WiFi Status CONNECTED
Blynk Status OFFLINE
WiFi RSSI -80 dBm
Queued Cloud Events 0
EV BMS TASK 1 + 2 + 3 + 4
BATTERY PARAMETERS


wokwi.com/projects/473055663852077057
WOKWI
SAVE
SHARE
Task-5 Intelligent Cloud Telemetry Architecture Copy
Docs
sketch.ino
diagram.json
libraries.txt
Library Manager
Simulation
04:29.537
53%
for (int i = 0; i < NUM_CELLS; i++)
previousCellVoltage[i] = cellVoltage[i];
previousADC[i] = cellADC[i];
frozenCount[i] = 0;
cellSensorHealthy[i] = true;
updateLEDs();
displayNormalLCD();
Serial.println("EV BMS TASK 1 + 2 + 3 + 4 READY");
void loop()
unsigned long now = millis();
if (WiFi.status() == WL_CONNECTED)
#9 58573 ms CRITICAL SAFETY FAULT [FAILSAFE]
#10 58575 ms OVER VOLTAGE [FAILSAFE]
CLOUD TELEMETRY
WiFi Status CONNECTED
Blynk Status CONNECTED
WiFi RSSI -66 dBm
Queued Cloud Events 0
>>> BLYNK CONNECTED - CLOUD SYNC <<<
EV BMS TASK 1 + 2 + 3 + 4
BATTERY PARAMETERS


Blynk.Console
My organization - 4964HD
Messages used: 3.5k of 100.0k
Ask AI
Get Started
Dashboards
Custom Data
Developer Zone
Devices
Automations
Users
Organizations
Locations
Snapshots
Fleet Management
In-App Messaging
Predictive Maintenance
EV BMS Task5
Online
Add Tag
Edit
Live
1h
6h
1d
1w
1mo
3mo
6mo
1y
Cell 1 Voltage
2.75V
Cell 2 Voltage
3.06V
Cell 3 Voltage
2.78V
Cell 4 Voltage
2.5V
Pack Voltage
11V
Average Voltage
2.78V
Imbalance
20.14%
WIFI SIGNAL
-98dBm
Battery Health
CRITICAL IMBALAN(
Runtime Mode
FAILSAFE
Safety State
TRIPPED
Relay State
0
Fault Event
UNDER VOLTAGE
Collapse
Region: BLR1
Privacy Policy
Terms of Service

4) Normal BMS operation after cloud recovery:



Link of Simulation (Wokwi):

[Task 5 Click Here]

Link of Dashboard (Blynk IoT):

[Blynk Dashboard Click Here]

Task 5 Video Demonstration Link:

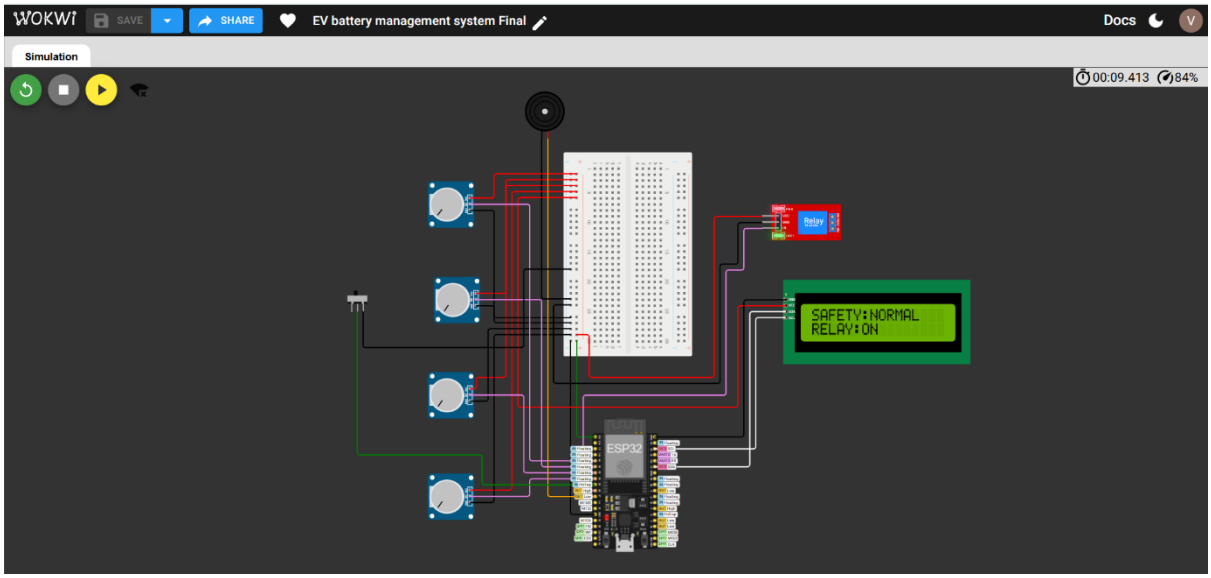
[Click Here]

13. Task 6: Executive Battery Intelligence Dashboard

13.1 Task 6 Problem

This final task involves creating an enterprise-grade monitoring dashboard that visualizes battery intelligence and operational analytics in a structured and professional manner. The dashboard should include live operational data, risk analysis, historical voltage trends, diagnostics, fault history, and intelligent operator recommendations. It must use severity-based color coding, dynamic telemetry visualization, and actionable insights to provide a professional IoT user experience suitable for real-world battery management systems.

13.2 Final Project Circuit Diagram :



13.3 Code:

```
#define BLYNK_TEMPLATE_ID "TMPL36OnLHkD5"

#define BLYNK_TEMPLATE_NAME "EV BMS Task5"

#define BLYNK_AUTH_TOKEN "nKkqxnUij0v4qdk0oJ-niWzcGSOd8tmh"

#define CLOUD_TEST_BUTTON 25


#include <WiFi.h>

#include <BlynkSimpleEsp32.h>

#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <math.h>


const char* WIFI_SSID = "Wokwi-GUEST";

const char* WIFI_PASSWORD = "";
```

```
// ====================================================
// EV BMS - TASK 1 + TASK 2 + TASK 3 + TASK 4
//
// TASK 1 : Adaptive Multi-Cell Battery Intelligence
// TASK 2 : Event-Driven Safety Protection
// TASK 3 : Intelligent HMI & Diagnostic Interface
// TASK 4 : Fault-Tolerant Embedded Runtime
//
// Hardware:
// ESP32 DevKit V1
// 4 x Potentiometers
// 16x2 I2C LCD
// 4 x LEDs
// Relay
// Buzzer
// ====================================================

// ====================================================
// 1. PIN CONFIGURATION
// ====================================================

#define CELL1_PIN 34
#define CELL2_PIN 35
#define CELL3_PIN 32
#define CELL4_PIN 33

#define SDA_PIN 21
#define SCL_PIN 22

#define LED_GREEN  4
#define LED_YELLOW 2
#define LED_ORANGE 15
#define LED_RED    5

#define RELAY_PIN  26
#define BUZZER_PIN 27
```

```
// ======================================================
// 2. LCD
// ======================================================

LiquidCrystal_I2C lcd(0x27, 16, 2);

// ======================================================
// 3. BATTERY CONFIGURATION
// ======================================================

#define NUM_CELLS 4

const int ADC_MAX = 4095;

const float ADC_MAX_VOLTAGE = 3.3;

const float CELL_MIN_VOLTAGE = 2.5;
const float CELL_MAX_VOLTAGE = 4.2;

// ======================================================
// 4. TASK 1 HEALTH THRESHOLDS
// ======================================================

const float HEALTHY_LIMIT = 3.0;
const float MINOR_LIMIT   = 10.0;
const float CRITICAL_LIMIT = 10.0;

// ======================================================
// 5. TASK 2 SAFETY THRESHOLDS
// ======================================================

const float UV_THRESHOLD = 2.8;
const float OV_THRESHOLD = 4.15;

const float RAPID_CHANGE_THRESHOLD = 0.10;

// ======================================================
// 6. TIMING
```

```
// ======================================================

const unsigned long SAMPLE_INTERVAL = 200;

const unsigned long LCD_PAGE_INTERVAL = 3000;

const unsigned long LCD_REFRESH_INTERVAL = 500;

const unsigned long SERIAL_INTERVAL = 2000;

const unsigned long FAULT_CONFIRM_TIME = 500;

const unsigned long SAFETY_RECOVERY_TIME = 5000;

const unsigned long RUNTIME_RECOVERY_TIME = 3000;

const unsigned long RELAY_LOCK_TIME = 3000;

const unsigned long BUZZER_INTERVAL = 300;

// ======================================================
// 7. TASK 4 DIAGNOSTIC SETTINGS
// ======================================================

// IMPORTANT:
//
// Keep all FALSE for NORMAL operation.
//
// Change ONE at a time for testing.
//
// ------------------------------------------------------

bool simulateFrozenADC = false;

bool simulateRelayMismatch = false;

bool simulateShutdown = false;
```

```
bool cloudTestMode = false;

// Frozen ADC threshold
const int FROZEN_LIMIT = 10;

// ====================================================
// 8. TIMERS
// ====================================================

unsigned long previousSampleTime = 0;

unsigned long previousLCDPageTime = 0;

unsigned long previousLCDRefreshTime = 0;

unsigned long previousSerialTime = 0;

unsigned long faultStartTime = 0;

unsigned long safetyRecoveryStart = 0;

unsigned long runtimeRecoveryStart = 0;

unsigned long lastRelayChangeTime = 0;

unsigned long previousBuzzerTime = 0;

// ====================================================
// 9. BATTERY DATA
// ====================================================

float cellVoltage[NUM_CELLS];

float previousCellVoltage[NUM_CELLS];

int cellADC[NUM_CELLS];

int previousADC[NUM_CELLS];
```

```
int frozenCount[NUM_CELLS];

bool cellSensorHealthy[NUM_CELLS];

float packVoltage = 0.0;

float averageVoltage = 0.0;

float maxVoltage = 0.0;

float minVoltage = 0.0;

float imbalanceVoltage = 0.0;

float imbalancePercent = 0.0;

int strongestCell = 0;

int weakestCell = 0;

String batteryHealth;

// =====================================================
// 10. TASK 2 SAFETY STATE
// =====================================================

enum SafetyState
{
 SAFETY_NORMAL,
 SAFETY_WARNING,
 SAFETY_TRIPPED,
 SAFETY_RECOVERY
};

SafetyState safetyState = SAFETY_NORMAL;

// =====================================================
```

```
// 11. TASK 2 FAULT FLAGS
// ======================================================

bool underVoltageFault = false;

bool overVoltageFault = false;

bool rapidChangeFault = false;

bool sensorFault = false;

bool faultDetected = false;

bool faultConfirmed = false;

// ======================================================
// 12. TASK 4 RUNTIME STATES
// ======================================================

enum RuntimeMode
{
 RUNTIME_NORMAL,
 RUNTIME_DEGRADED,
 RUNTIME_FAILSAFE,
 RUNTIME_SHUTDOWN
};

RuntimeMode runtimeMode = RUNTIME_NORMAL;

// ======================================================
// 13. TASK 4 FAULT FLAGS
// ======================================================

bool invalidSensorFault = false;

bool frozenADCFault = false;

bool relayMismatchFault = false;
```

```
bool multipleFailureFault = false;

// ====================================================
// 14. RELAY
// ====================================================

bool relayState = true;

// ====================================================
// 15. LCD HMI
// ====================================================

int currentPage = 0;

const int TOTAL_PAGES = 5;

bool faultPriorityActive = false;

RuntimeMode previousRuntimeMode = RUNTIME_NORMAL;

SafetyState previousSafetyState = SAFETY_NORMAL;

int previousPage = -1;

// ====================================================
// 16. FAULT LOG
// ====================================================

struct FaultLog
{
  unsigned long timestamp;

  String faultName;

  String mode;
};
```

```
#define MAX_FAULT_LOGS 10

FaultLog faultLogs[MAX_FAULT_LOGS];

int faultLogCount = 0;

// ======================================================
// 17. FAULT EVENT MEMORY
// ======================================================

bool previousInvalidSensorFault = false;

bool previousFrozenADCFault = false;

bool previousRelayMismatchFault = false;

bool previousUnderVoltageFault = false;

bool previousOverVoltageFault = false;

bool previousRapidChangeFault = false;


// ======================================================
// TASK 5 - INTELLIGENT CLOUD TELEMETRY
// ======================================================
// Blynk virtual datastream mapping - MATCHES THE USER'S DASHBOARD:
 // V0  Cell 1 Voltage
 // V1  Cell 2 Voltage
 // V2  Cell 3 Voltage
 // V3  Cell 4 Voltage
 // V4  Pack Voltage
 // V5  Average Voltage
 // V6  Imbalance %
 // V7  Battery Health
 // V8  Runtime Mode
 // V9  Safety State
 // V10 Relay State
```

```
// V11 Fault / Event Message
// V12 WiFi RSSI
//
// IMPORTANT: Create the same datastream VPINs in Blynk.
// Event code used below: bms_fault
// ====================================================

#define ENABLE_BLYNK_EVENTS 1

const unsigned long WIFI_RECONNECT_INTERVAL = 5000;
const unsigned long BLYNK_RECONNECT_INTERVAL = 5000;
const float TELEMETRY_VOLTAGE_CHANGE = 0.05;
const float TELEMETRY_IMBALANCE_CHANGE = 0.5;

unsigned long previousWiFiReconnect = 0;
unsigned long previousBlynkReconnect = 0;

bool previousCloudConnected = false;
bool cloudSyncRequired = true;

// Last values actually transmitted to Blynk
float sentCellVoltage[NUM_CELLS] = {-100, -100, -100, -100};
float sentPackVoltage = -100;
float sentAverageVoltage = -100;
float sentImbalancePercent = -100;
String sentRuntimeMode = "";
String sentSafetyState = "";
String sentHealth = "";
String sentFaultText = "";
int sentRelayState = -1;
int sentRSSI = 999;

// Small event queue for faults/state changes while cloud is offline
#define CLOUD_EVENT_QUEUE_SIZE 10
String cloudEventQueue[CLOUD_EVENT_QUEUE_SIZE];
int cloudEventHead = 0;
int cloudEventTail = 0;
int cloudEventCount = 0;
```

```
void queueCloudEvent(const String &eventText)
{
 if (cloudEventCount < CLOUD_EVENT_QUEUE_SIZE)
 {
  cloudEventQueue[cloudEventTail] = eventText;
  cloudEventTail = (cloudEventTail + 1) % CLOUD_EVENT_QUEUE_SIZE;
  cloudEventCount++;
 }
 else
 {
  // Keep the newest event when the queue is full.
  cloudEventQueue[cloudEventHead] = eventText;
  cloudEventHead = (cloudEventHead + 1) % CLOUD_EVENT_QUEUE_SIZE;
  cloudEventTail = cloudEventHead;
 }
}

bool cloudEventAvailable()
{
 return cloudEventCount > 0;
}

String popCloudEvent()
{
 if (cloudEventCount == 0) return "";

 String e = cloudEventQueue[cloudEventHead];
 cloudEventHead = (cloudEventHead + 1) % CLOUD_EVENT_QUEUE_SIZE;
 cloudEventCount--;
 return e;
}

String getRuntimeModeText()
{
 switch (runtimeMode)
 {
  case RUNTIME_NORMAL:   return "NORMAL";
```

```
    case RUNTIME_DEGRADED: return "DEGRADED";
    case RUNTIME_FAILSAFE: return "FAILSAFE";
    default:               return "SHUTDOWN";
  }
}

String getSafetyStateText()
{
  switch (safetyState)
  {
    case SAFETY_NORMAL:   return "NORMAL";
    case SAFETY_WARNING:  return "WARNING";
    case SAFETY_TRIPPED:  return "TRIPPED";
    default:              return "RECOVERY";
  }
}

String getMainFaultText()
{
  if (overVoltageFault) return "OVER VOLTAGE";
  if (underVoltageFault) return "UNDER VOLTAGE";
  if (relayMismatchFault) return "RELAY MISMATCH";
  if (invalidSensorFault) return "INVALID SENSOR";
  if (frozenADCFault) return "ADC FROZEN";
  if (rapidChangeFault) return "RAPID VOLTAGE CHANGE";
  return "NO ACTIVE FAULT";
}

void sendBlynkEvent(const String &eventText)
{
#if ENABLE_BLYNK_EVENTS
  if (Blynk.connected())
  {
    Blynk.logEvent("bms_fault", eventText);
  }
#else
  (void)eventText;
#endif
```

```
}

void syncAllTelemetry()
{
  if (!Blynk.connected()) return;

  // Full synchronization is performed ONLY after a cloud connection/reconnection.
  Blynk.virtualWrite(V0, cellVoltage[0]);
  Blynk.virtualWrite(V1, cellVoltage[1]);
  Blynk.virtualWrite(V2, cellVoltage[2]);
  Blynk.virtualWrite(V3, cellVoltage[3]);
  Blynk.virtualWrite(V4, packVoltage);
  Blynk.virtualWrite(V5, averageVoltage);
  Blynk.virtualWrite(V6, imbalancePercent);
  Blynk.virtualWrite(V7, batteryHealth);
  Blynk.virtualWrite(V8, getRuntimeModeText());
  Blynk.virtualWrite(V9, getSafetyStateText());
  Blynk.virtualWrite(V10, relayState ? 1 : 0);
  Blynk.virtualWrite(V11, getMainFaultText());
  Blynk.virtualWrite(V12, WiFi.RSSI());

  for (int i = 0; i < NUM_CELLS; i++)
    sentCellVoltage[i] = cellVoltage[i];

  sentPackVoltage = packVoltage;
  sentAverageVoltage = averageVoltage;
  sentImbalancePercent = imbalancePercent;
  sentRuntimeMode = getRuntimeModeText();
  sentSafetyState = getSafetyStateText();
  sentRelayState = relayState ? 1 : 0;
  sentRSSI = WiFi.RSSI();
  sentHealth = batteryHealth;
  sentFaultText = getMainFaultText();

  cloudSyncRequired = false;
}

void flushCloudEvents()
```

```
{
  if (!Blynk.connected()) return;

  // Send queued events after reconnect, then remove them from the queue.
  while (cloudEventAvailable())
  {
    String eventText = popCloudEvent();
    Blynk.virtualWrite(V10, eventText);
    sendBlynkEvent(eventText);
  }
}

void publishChangedTelemetry()
{
  if (!Blynk.connected()) return;

  // V0-V3: cell voltage; send only after a meaningful change.
  for (int i = 0; i < NUM_CELLS; i++)
  {
    if (fabs(cellVoltage[i] - sentCellVoltage[i]) >= TELEMETRY_VOLTAGE_CHANGE)
    {
      Blynk.virtualWrite(V0 + i, cellVoltage[i]);
      sentCellVoltage[i] = cellVoltage[i];
    }
  }

  // V4: pack voltage.
  if (fabs(packVoltage - sentPackVoltage) >= TELEMETRY_VOLTAGE_CHANGE)
  {
    Blynk.virtualWrite(V4, packVoltage);
    sentPackVoltage = packVoltage;
  }

  // V5: average voltage.
  if (fabs(averageVoltage - sentAverageVoltage) >= TELEMETRY_VOLTAGE_CHANGE)
  {
    Blynk.virtualWrite(V5, averageVoltage);
    sentAverageVoltage = averageVoltage;
```

```
}

// V6: imbalance percentage.
if (fabs(imbalancePercent - sentImbalancePercent) >= TELEMETRY_IMBALANCE_CHANGE)
{
 Blynk.virtualWrite(V6, imbalancePercent);
 sentImbalancePercent = imbalancePercent;
}

// V7: battery health.
if (batteryHealth != sentHealth)
{
 Blynk.virtualWrite(V7, batteryHealth);
 sentHealth = batteryHealth;
}

// V8: runtime mode.
String runtimeText = getRuntimeModeText();
if (runtimeText != sentRuntimeMode)
{
 Blynk.virtualWrite(V8, runtimeText);
 sentRuntimeMode = runtimeText;
}

// V9: safety state.
String safetyText = getSafetyStateText();
if (safetyText != sentSafetyState)
{
 Blynk.virtualWrite(V9, safetyText);
 sentSafetyState = safetyText;
}

// V10: relay state.
int relay = relayState ? 1 : 0;
if (relay != sentRelayState)
{
 Blynk.virtualWrite(V10, relay);
 sentRelayState = relay;
```

```
  }

  // V11: fault/event text.
  String faultText = getMainFaultText();
  if (faultText != sentFaultText)
  {
   Blynk.virtualWrite(V11, faultText);
   sentFaultText = faultText;
  }

  // V12: Wi-Fi RSSI.
  int rssi = WiFi.RSSI();
  if (abs(rssi - sentRSSI) >= 5)
  {
   Blynk.virtualWrite(V12, rssi);
   sentRSSI = rssi;
  }
}

void processCloudTelemetry()
{
  unsigned long now = millis();

  // WiFi reconnect is non-blocking: no delay() is used here.
  if (WiFi.status() != WL_CONNECTED)
  {
   if (now - previousWiFiReconnect >= WIFI_RECONNECT_INTERVAL)
   {
    previousWiFiReconnect = now;
    WiFi.disconnect();
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
   }

   previousCloudConnected = false;
   return;
  }

  // Blynk reconnect is attempted periodically,
```

```
// unless cloud test mode intentionally disables it.
if (cloudTestMode)
{
 if (Blynk.connected())
 {
  Blynk.disconnect();
 }

 previousCloudConnected = false;
 return;
}

if (!Blynk.connected())
{
 if (now - previousBlynkReconnect >= BLYNK_RECONNECT_INTERVAL)
 {
  previousBlynkReconnect = now;
  Blynk.connect(1000);
 }

 previousCloudConnected = false;
 return;
}

 // Detect a new cloud connection.
 if (!previousCloudConnected)
 {
  previousCloudConnected = true;
  cloudSyncRequired = true;
  Serial.println(">>> BLYNK CONNECTED - CLOUD SYNC <<<");
 }

 if (cloudSyncRequired)
 {
  syncAllTelemetry();
  flushCloudEvents();
 }
 else
```

```
  {
    publishChangedTelemetry();
  }
}

void connectWiFi()
{
  Serial.println();
  Serial.println("Starting Wi-Fi connection...");
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
}

void handleCloudTestButton()
{

  bool pressed = (digitalRead(CLOUD_TEST_BUTTON) == LOW);

  if (pressed && !cloudTestMode)
  {
    cloudTestMode = true;

    Serial.println();
    Serial.println("=======================================");
    Serial.println(">>> CLOUD TEST MODE: OFFLINE <<<");
    Serial.println("Blynk communication intentionally disabled");
    Serial.println("BMS operation continues normally");
    Serial.println("=======================================");
  }

  if (!pressed && cloudTestMode)
  {
    cloudTestMode = false;

    Serial.println();
    Serial.println("=======================================");
    Serial.println(">>> CLOUD TEST MODE: ONLINE <<<");
```

```
    Serial.println("Blynk communication restored");
    Serial.println("Queued events will synchronize");
    Serial.println("=====================================");
  }
}

void updateRiskDiagnosticMask()
{
  int riskMask = 0;

  // Bit 0 - Under Voltage
  if (underVoltageFault)
  {
    riskMask |= (1 << 0);
  }

  // Bit 1 - Over Voltage
  if (overVoltageFault)
  {
    riskMask |= (1 << 1);
  }

  // Bit 2 - Rapid Voltage Change
  if (rapidChangeFault)
  {
    riskMask |= (1 << 2);
  }

  // Bit 3 - Invalid Sensor
  if (invalidSensorFault)
  {
    riskMask |= (1 << 3);
  }

  if (Blynk.connected())
  {
    Blynk.virtualWrite(V13, riskMask);
  }
```

```
}

int lastPublishedFaultCount = 0;

void publishFaultHistory()
{
 if (!Blynk.connected())
 {
  return;
 }

 // Send only newly added fault events
 while (lastPublishedFaultCount < faultLogCount)
 {
  int i = lastPublishedFaultCount;

  String eventText = "";

  eventText += "#";
  eventText += String(i + 1);
  eventText += " ";
  eventText += String(faultLogs[i].timestamp);
  eventText += "ms ";
  eventText += faultLogs[i].faultName;
  eventText += " [";
  eventText += faultLogs[i].mode;
  eventText += "]";

  Blynk.virtualWrite(V14, eventText);

  lastPublishedFaultCount++;
 }
}

// ADD THIS HERE

void updateOperatorRecommendation()
{
```

```
  String recommendation;

  if (underVoltageFault)
  {
    recommendation = "UNDER VOLTAGE: Inspect affected cell and reduce load.";
  }
  else if (overVoltageFault)
  {
    recommendation = "OVER VOLTAGE: Stop charging and inspect cell.";
  }
  else if (rapidChangeFault)
  {
    recommendation = "RAPID CHANGE: Check cell and sensor.";
  }
  else if (invalidSensorFault)
  {
    recommendation = "INVALID SENSOR: Check sensor connection.";
  }
  else if (batteryHealth == "CRITICAL IMBALANCE")
  {
    recommendation = "CRITICAL IMBALANCE: Inspect and balance cells.";
  }
  else if (batteryHealth == "MINOR IMBALANCE")
  {
    recommendation = "MINOR IMBALANCE: Monitor weakest cell.";
  }
  else
  {
    recommendation = "NORMAL: Battery operating normally.";
  }

  if (Blynk.connected())
  {
    Blynk.virtualWrite(V15, recommendation);
  }
}

// ====================================================
```

```
// SETUP
// ================================================

void setup()
{
 Serial.begin(115200);

 connectWiFi();

 Blynk.config(BLYNK_AUTH_TOKEN);

 Serial.println("Blynk configured; cloud connection will be handled in loop.");

 pinMode(CLOUD_TEST_BUTTON, INPUT_PULLUP);
 // ================================================
 // ADC
 // ================================================

 analogReadResolution(12);

 pinMode(CELL1_PIN, INPUT);
 pinMode(CELL2_PIN, INPUT);
 pinMode(CELL3_PIN, INPUT);
 pinMode(CELL4_PIN, INPUT);

 // ================================================
 // LEDs
 // ================================================

 pinMode(LED_GREEN, OUTPUT);
 pinMode(LED_YELLOW, OUTPUT);
 pinMode(LED_ORANGE, OUTPUT);
 pinMode(LED_RED, OUTPUT);

 digitalWrite(LED_GREEN, LOW);
 digitalWrite(LED_YELLOW, LOW);
 digitalWrite(LED_ORANGE, LOW);
 digitalWrite(LED_RED, LOW);
```

```
// ================================================
// RELAY
// ================================================

pinMode(RELAY_PIN, OUTPUT);

digitalWrite(RELAY_PIN, HIGH);

relayState = true;

lastRelayChangeTime = millis();

// ================================================
// BUZZER
// ================================================

pinMode(BUZZER_PIN, OUTPUT);

digitalWrite(BUZZER_PIN, LOW);

// ================================================
// I2C
// ================================================

Wire.begin(SDA_PIN, SCL_PIN);

// ================================================
// LCD
// ================================================

lcd.init();

lcd.backlight();

lcd.clear();

lcd.setCursor(0, 0);
```

```
lcd.print("EV BMS TASK 4");

lcd.setCursor(0, 1);
lcd.print("Initializing...");

// ================================================
// INITIAL READING
// ================================================

readCellVoltages();

calculateBatteryParameters();

identifyStrongestWeakest();

calculateImbalance();

classifyBatteryHealth();

for (int i = 0; i < NUM_CELLS; i++)
{
  previousCellVoltage[i] = cellVoltage[i];

  previousADC[i] = cellADC[i];

  frozenCount[i] = 0;

  cellSensorHealthy[i] = true;
}

updateLEDs();

displayNormalLCD();

Serial.println();
Serial.println("================================================");
Serial.println("    EV BMS TASK 1 + 2 + 3 + 4 READY");
Serial.println("================================================");
```

```
}

// ======================================================
// MAIN LOOP
// ======================================================

void loop()
{

 unsigned long now = millis();

 if (WiFi.status() == WL_CONNECTED)
 {
  if (!cloudTestMode && Blynk.connected())
  {
   Blynk.run();
  }
 }

 // TASK 5: network/cloud manager.
 // Runs independently of battery protection logic.
 processCloudTelemetry();

 // ==================================================
 // BATTERY + SAFETY + RUNTIME
 // ==================================================

 if (now - previousSampleTime >= SAMPLE_INTERVAL)
 {
  previousSampleTime = now;

  // ------------------------------------------------
  // TASK 1
  // ------------------------------------------------

  readCellVoltages();

  calculateBatteryParameters();
```

```
    identifyStrongestWeakest();

    calculateImbalance();

    classifyBatteryHealth();

    // ------------------------------------------------
    // TASK 2
    // ------------------------------------------------

    detectSafetyConditions();

    updateSafetyState();

    updateRelay();

    updateBuzzer();

    // ------------------------------------------------
    // TASK 4
    // ------------------------------------------------

    checkSensorHealth();

    checkFrozenADC();

    checkRelayMismatch();

    updateRuntimeMode();

    updateLEDs();

    // ------------------------------------------------
    // FAULT EVENT LOGGING
    // ------------------------------------------------

    processFaultEvents();
```

```
  updateRiskDiagnosticMask();

  publishFaultHistory();

  updateOperatorRecommendation();

}

// ================================================
// TASK 3 - AUTOMATIC PAGE ROTATION
// ================================================

if (now - previousLCDPageTime >= LCD_PAGE_INTERVAL)
{
  previousLCDPageTime = now;

  // Do not rotate normal pages during critical faults

  if (!faultDetected &&
     runtimeMode == RUNTIME_NORMAL)
  {
    currentPage++;

    if (currentPage >= TOTAL_PAGES)
    {
      currentPage = 0;
    }

    previousPage = -1;
  }
}

// ================================================
// LCD REFRESH
// ================================================

if (now - previousLCDRefreshTime >= LCD_REFRESH_INTERVAL)
```

```
  {
    previousLCDRefreshTime = now;

    updateLCD();
  }

  // ==================================================
  // SERIAL
  // ==================================================

  if (now - previousSerialTime >= SERIAL_INTERVAL)
  {
    previousSerialTime = now;

    displaySerialReport();
  }
}

// ==================================================
// TASK 1
// READ CELL VOLTAGES
// ==================================================

void readCellVoltages()
{
  cellADC[0] = analogRead(CELL1_PIN);

  cellADC[1] = analogRead(CELL2_PIN);

  cellADC[2] = analogRead(CELL3_PIN);

  cellADC[3] = analogRead(CELL4_PIN);

  for (int i = 0; i < NUM_CELLS; i++)
  {
    cellVoltage[i] =
      convertADCToCellVoltage(cellADC[i]);
  }
```

```
}

// ======================================================
// TASK 1
// ADC TO VOLTAGE
// ======================================================

float convertADCToCellVoltage(int rawADC)
{
 float adcVoltage =
  ((float)rawADC / ADC_MAX)
  * ADC_MAX_VOLTAGE;

 float voltage =
  CELL_MIN_VOLTAGE
  +
  (adcVoltage / ADC_MAX_VOLTAGE)
  *
  (CELL_MAX_VOLTAGE -
   CELL_MIN_VOLTAGE);

 return voltage;
}

// ======================================================
// TASK 1
// BATTERY PARAMETERS
// ======================================================

void calculateBatteryParameters()
{
 packVoltage = 0.0;

 for (int i = 0; i < NUM_CELLS; i++)
 {
  packVoltage += cellVoltage[i];
 }
```

```
  averageVoltage =
   packVoltage / NUM_CELLS;

  maxVoltage = cellVoltage[0];

  minVoltage = cellVoltage[0];

  for (int i = 1; i < NUM_CELLS; i++)
  {
   if (cellVoltage[i] > maxVoltage)
   {
    maxVoltage = cellVoltage[i];
   }

   if (cellVoltage[i] < minVoltage)
   {
    minVoltage = cellVoltage[i];
   }
  }
}

// ======================================================
// TASK 1
// STRONGEST / WEAKEST
// ======================================================

void identifyStrongestWeakest()
{
  strongestCell = 0;

  weakestCell = 0;

  for (int i = 1; i < NUM_CELLS; i++)
  {
   if (cellVoltage[i] >
     cellVoltage[strongestCell])
   {
    strongestCell = i;
```

```
    }

    if (cellVoltage[i] <
        cellVoltage[weakestCell])
    {
      weakestCell = i;
    }
  }
}

// ====================================================
// TASK 1
// IMBALANCE
// ====================================================

void calculateImbalance()
{
  imbalanceVoltage =
    maxVoltage - minVoltage;

  if (averageVoltage > 0)
  {
    imbalancePercent =
      (imbalanceVoltage /
       averageVoltage)
      * 100.0;
  }
  else
  {
    imbalancePercent = 0.0;
  }
}

// ====================================================
// TASK 1
// HEALTH CLASSIFICATION
// ====================================================
```

```
void classifyBatteryHealth()
{
 if (imbalancePercent < 3.0)
 {
  batteryHealth = "HEALTHY";
 }

 else if (imbalancePercent < 10.0)
 {
  batteryHealth = "MINOR IMBALANCE";
 }

 else
 {
  batteryHealth = "CRITICAL IMBALANCE";
 }
}

// ======================================================
// TASK 2
// SAFETY DETECTION
// ======================================================

void detectSafetyConditions()
{
 underVoltageFault = false;

 overVoltageFault = false;

 rapidChangeFault = false;

 sensorFault = false;

 for (int i = 0; i < NUM_CELLS; i++)
 {
  // ------------------------------------------------
  // Under Voltage
  // ------------------------------------------------
```

```
  if (cellVoltage[i] < UV_THRESHOLD)
  {
    underVoltageFault = true;
  }

  // ------------------------------------------------
  // Over Voltage
  // ------------------------------------------------

  if (cellVoltage[i] > OV_THRESHOLD)
  {
    overVoltageFault = true;
  }

  // ------------------------------------------------
  // Rapid Voltage Change
  // ------------------------------------------------

  float change =
    fabs(cellVoltage[i] -
        previousCellVoltage[i]);

  if (change > RAPID_CHANGE_THRESHOLD)
  {
    rapidChangeFault = true;
  }

  // ------------------------------------------------
  // Invalid sensor
  // ------------------------------------------------

  if (cellADC[i] <= 5 ||
      cellADC[i] >= 4090)
  {
    sensorFault = true;
  }
 }
```

```
  // Store previous voltages

  for (int i = 0; i < NUM_CELLS; i++)
  {
    previousCellVoltage[i] =
      cellVoltage[i];
  }

  faultDetected =
    underVoltageFault ||
    overVoltageFault ||
    rapidChangeFault ||
    sensorFault;
}

// ======================================================
// TASK 2
// SAFETY STATE MACHINE
// ======================================================

void updateSafetyState()
{
  unsigned long now = millis();

  switch (safetyState)
  {
    case SAFETY_NORMAL:

      if (faultDetected)
      {
        safetyState = SAFETY_WARNING;

        faultStartTime = now;
      }

      break;
```

```
case SAFETY_WARNING:

 if (!faultDetected)
 {
  safetyState = SAFETY_NORMAL;

  faultConfirmed = false;

  break;
 }

 if (now - faultStartTime >=
    FAULT_CONFIRM_TIME)
 {
  faultConfirmed = true;

  safetyState = SAFETY_TRIPPED;
 }

 break;

case SAFETY_TRIPPED:

 if (!faultDetected)
 {
  safetyRecoveryStart = now;

  safetyState = SAFETY_RECOVERY;
 }

 break;

case SAFETY_RECOVERY:

 if (faultDetected)
 {
  safetyState = SAFETY_TRIPPED;
```

```
      break;
    }

    if (now - safetyRecoveryStart >=
      SAFETY_RECOVERY_TIME)
    {
      safetyState = SAFETY_NORMAL;

      faultConfirmed = false;
    }

    break;
  }
}

// ====================================================
// TASK 2
// RELAY CONTROL
// ====================================================

void updateRelay()
{
  unsigned long now = millis();

  // ------------------------------------------------
  // SAFETY TRIP
  // ------------------------------------------------

  if (safetyState == SAFETY_WARNING ||
     safetyState == SAFETY_TRIPPED)
  {
    if (relayState)
    {
      if (now - lastRelayChangeTime >=
        RELAY_LOCK_TIME)
      {
        digitalWrite(RELAY_PIN, LOW);
```

```
      relayState = false;

      lastRelayChangeTime = now;

      Serial.println(">>> RELAY CUT-OFF <<<");
    }
   }
  }

  // ------------------------------------------------
  // RECOVERY
  // ------------------------------------------------

  else if (safetyState ==
        SAFETY_NORMAL)
  {
   if (!relayState)
   {
    if (now - lastRelayChangeTime >=
       RELAY_LOCK_TIME)
    {
      digitalWrite(RELAY_PIN, HIGH);

      relayState = true;

      lastRelayChangeTime = now;

      Serial.println(">>> RELAY RESTORED <<<");
    }
   }
  }
 }

 // ======================================================
 // TASK 2
 // BUZZER
 // ======================================================
```

```
void updateBuzzer()
{
  unsigned long now = millis();

  if (safetyState == SAFETY_WARNING ||
      safetyState == SAFETY_TRIPPED)
  {
    if (now - previousBuzzerTime >=
        BUZZER_INTERVAL)
    {
      previousBuzzerTime = now;

      digitalWrite(
        BUZZER_PIN,
        !digitalRead(BUZZER_PIN)
      );
    }
  }

  else
  {
    digitalWrite(BUZZER_PIN, LOW);
  }
}

// ====================================================
// TASK 4
// SENSOR HEALTH
// ====================================================

void checkSensorHealth()
{
  invalidSensorFault = false;

  for (int i = 0; i < NUM_CELLS; i++)
  {
    if (cellADC[i] <= 5 ||
        cellADC[i] >= 4090)
```

```
    {
      invalidSensorFault = true;

      cellSensorHealthy[i] = false;
    }
    else
    {
      cellSensorHealthy[i] = true;
    }
  }
}

// ======================================================
// TASK 4
// FROZEN ADC
//
// IMPORTANT:
// A stable battery voltage is NOT automatically
// considered an ADC fault.
//
// Frozen ADC is activated only when:
// simulateFrozenADC = true
// ======================================================

void checkFrozenADC()
{
  frozenADCFault = false;

  if (!simulateFrozenADC)
  {
    for (int i = 0; i < NUM_CELLS; i++)
    {
      frozenCount[i] = 0;

      previousADC[i] =
        cellADC[i];
    }
```

```
  return;
 }

 // Diagnostic simulation enabled

 for (int i = 0; i < NUM_CELLS; i++)
 {
  if (cellADC[i] ==
     previousADC[i])
  {
   frozenCount[i]++;
  }
  else
  {
   frozenCount[i] = 0;
  }

  if (frozenCount[i] >=
     FROZEN_LIMIT)
  {
   frozenADCFault = true;

   frozenCount[i] =
    FROZEN_LIMIT;
  }

  previousADC[i] =
   cellADC[i];
 }
}

// ======================================================
// TASK 4
// RELAY MISMATCH
// ======================================================

void checkRelayMismatch()
{
```

```
 relayMismatchFault =
   simulateRelayMismatch;
}

// ======================================================
// TASK 4
// RUNTIME STATE MACHINE
// ======================================================

void updateRuntimeMode()
{
 unsigned long now = millis();

 // ================================================
 // SHUTDOWN
 // ================================================

 if (simulateShutdown)
 {
  if (runtimeMode !=
     RUNTIME_SHUTDOWN)
  {
   logFault(
    "SYSTEM SHUTDOWN",
    "SHUTDOWN"
   );
  }

  runtimeMode =
   RUNTIME_SHUTDOWN;

  digitalWrite(RELAY_PIN, LOW);

  relayState = false;

  return;
 }
```

```
// ================================================
// FAILSAFE
// ================================================

if (relayMismatchFault ||
   overVoltageFault ||
   underVoltageFault)
{
 if (runtimeMode !=
    RUNTIME_FAILSAFE)
 {
  logFault(
    "CRITICAL SAFETY FAULT",
    "FAILSAFE"
  );
 }

 runtimeMode =
  RUNTIME_FAILSAFE;

 runtimeRecoveryStart = 0;

 return;
}

// ================================================
// DEGRADED
// ================================================

if (invalidSensorFault ||
   frozenADCFault ||
   rapidChangeFault)
{
 if (runtimeMode ==
    RUNTIME_NORMAL)
 {
  runtimeRecoveryStart = 0;
```

```
    logFault(
      "SYSTEM DEGRADED",
      "DEGRADED"
    );
  }

  runtimeMode =
    RUNTIME_DEGRADED;

  return;
}

// ================================================
// RECOVERY FROM FAILSAFE
// ================================================

if (runtimeMode ==
    RUNTIME_FAILSAFE)
{
  if (!underVoltageFault &&
      !overVoltageFault &&
      !relayMismatchFault)
  {
    if (runtimeRecoveryStart == 0)
    {
      runtimeRecoveryStart = now;
    }

    if (now - runtimeRecoveryStart >=
        RUNTIME_RECOVERY_TIME)
    {
      runtimeMode =
        RUNTIME_NORMAL;

      runtimeRecoveryStart = 0;

      logFault(
        "FAILSAFE RECOVERED",
```

```
      "NORMAL"
    );
   }
  }
  else
  {
   runtimeRecoveryStart = 0;
  }

  return;
}

// ================================================
// RECOVERY FROM DEGRADED
// ================================================

if (runtimeMode ==
   RUNTIME_DEGRADED)
{
 if (!invalidSensorFault &&
    !frozenADCFault &&
    !rapidChangeFault)
 {
  if (runtimeRecoveryStart == 0)
  {
   runtimeRecoveryStart = now;
  }

  if (now - runtimeRecoveryStart >=
     RUNTIME_RECOVERY_TIME)
  {
   runtimeMode =
    RUNTIME_NORMAL;

   runtimeRecoveryStart = 0;

   logFault(
    "DEGRADED RECOVERED",
```

```
        "NORMAL"
      );
    }
   }
   else
   {
    runtimeRecoveryStart = 0;
   }

   return;
  }

  // ================================================
  // NORMAL
  // ================================================

  if (!invalidSensorFault &&
     !frozenADCFault &&
     !relayMismatchFault &&
     !underVoltageFault &&
     !overVoltageFault &&
     !rapidChangeFault)
  {
   runtimeMode =
     RUNTIME_NORMAL;
  }
 }

// ======================================================
// TASK 4
// FAULT EVENT PROCESSING
// ======================================================

void processFaultEvents()
{
  // ------------------------------------------------
  // Invalid sensor event
  // ------------------------------------------------
```

```
if (invalidSensorFault &&
    !previousInvalidSensorFault)
{
  logFault(
    "INVALID SENSOR",
    "DEGRADED"
  );
}

// ------------------------------------------------
// Frozen ADC event
// ------------------------------------------------

if (frozenADCFault &&
    !previousFrozenADCFault)
{
  logFault(
    "ADC FROZEN",
    "DEGRADED"
  );
}

// ------------------------------------------------
// Relay mismatch
// ------------------------------------------------

if (relayMismatchFault &&
    !previousRelayMismatchFault)
{
  logFault(
    "RELAY MISMATCH",
    "FAILSAFE"
  );
}

// ------------------------------------------------
// Under voltage
```

```
// ------------------------------------------------

if (underVoltageFault &&
    !previousUnderVoltageFault)
{
  logFault(
    "UNDER VOLTAGE",
    "FAILSAFE"
  );
}

// ------------------------------------------------
// Over voltage
// ------------------------------------------------

if (overVoltageFault &&
    !previousOverVoltageFault)
{
  logFault(
    "OVER VOLTAGE",
    "FAILSAFE"
  );
}

// ------------------------------------------------
// Rapid voltage change
// ------------------------------------------------

if (rapidChangeFault &&
    !previousRapidChangeFault)
{
  logFault(
    "RAPID VOLTAGE CHANGE",
    "DEGRADED"
  );
}

// Save previous states
```

```
  previousInvalidSensorFault =
    invalidSensorFault;

  previousFrozenADCFault =
    frozenADCFault;

  previousRelayMismatchFault =
    relayMismatchFault;

  previousUnderVoltageFault =
    underVoltageFault;

  previousOverVoltageFault =
    overVoltageFault;

  previousRapidChangeFault =
    rapidChangeFault;
}

// =====================================================
// TASK 4
// FAULT LOG
// =====================================================

void logFault(String faultName,
              String mode)
{
  if (faultLogCount >=
      MAX_FAULT_LOGS)
  {
    return;
  }

  faultLogs[faultLogCount].timestamp =
    millis();

  faultLogs[faultLogCount].faultName =
```

```
  faultName;

 faultLogs[faultLogCount].mode =
  mode;

 // Task 5: preserve the fault event if cloud is offline.
 queueCloudEvent(faultName + " [" + mode + "]");

 faultLogCount++;

 Serial.println();
 Serial.println("******** FAULT EVENT ********");

 Serial.print("Time  : ");
 Serial.print(millis());
 Serial.println(" ms");

 Serial.print("Fault : ");
 Serial.println(faultName);

 Serial.print("Mode  : ");
 Serial.println(mode);

 Serial.println("******************************");
}

// ====================================================
// LED CONTROL
// ====================================================

void updateLEDs()
{
 digitalWrite(LED_GREEN, LOW);

 digitalWrite(LED_YELLOW, LOW);

 digitalWrite(LED_ORANGE, LOW);
```

```
digitalWrite(LED_RED, LOW);

// ------------------------------------------------
// SHUTDOWN
// ------------------------------------------------

if (runtimeMode ==
   RUNTIME_SHUTDOWN)
{
 digitalWrite(LED_RED, HIGH);

 return;
}

// ------------------------------------------------
// FAILSAFE
// ------------------------------------------------

if (runtimeMode ==
   RUNTIME_FAILSAFE)
{
 digitalWrite(LED_RED, HIGH);

 return;
}

// ------------------------------------------------
// DEGRADED
// ------------------------------------------------

if (runtimeMode ==
   RUNTIME_DEGRADED)
{
 digitalWrite(LED_YELLOW, HIGH);

 return;
}
```

```
 // ------------------------------------------------
 // NORMAL
 // ------------------------------------------------

 if (batteryHealth ==
    "HEALTHY")
 {
  digitalWrite(LED_GREEN, HIGH);
 }

 else if (batteryHealth ==
       "MINOR IMBALANCE")
 {
  digitalWrite(LED_YELLOW, HIGH);
 }

 else
 {
  digitalWrite(LED_ORANGE, HIGH);
 }
}

// ======================================================
// TASK 3
// LCD MANAGER
// ======================================================

void updateLCD()
{
 // ================================================
 // SHUTDOWN PRIORITY
 // ================================================

 if (runtimeMode ==
    RUNTIME_SHUTDOWN)
 {
  displayShutdownLCD();
```

```
  return;
}

// ================================================
// FAILSAFE PRIORITY
// ================================================

if (runtimeMode ==
    RUNTIME_FAILSAFE)
{
  displayFailsafeLCD();

  return;
}

// ================================================
// DEGRADED PRIORITY
// ================================================

if (runtimeMode ==
    RUNTIME_DEGRADED)
{
  displayDegradedLCD();

  return;
}

// ================================================
// SAFETY FAULT PRIORITY
// ================================================

if (faultDetected)
{
  displayFaultLCD();

  return;
}
```

```
 // ================================================
 // NORMAL HMI PAGES
 // ================================================

 if (currentPage == previousPage)
 {
  return;
 }

 previousPage =
  currentPage;

 lcd.clear();

 switch (currentPage)
 {
  case 0:
   displayCellPage();
   break;

  case 1:
   displayPackPage();
   break;

  case 2:
   displayHealthPage();
   break;

  case 3:
   displaySafetyPage();
   break;

  case 4:
   displayDiagnosticPage();
   break;
 }
}
```

```
// ====================================================
// LCD PAGE 1
// ====================================================

void displayCellPage()
{
  lcd.setCursor(0, 0);

  lcd.print("C1:");
  lcd.print(cellVoltage[0], 2);

  lcd.print(" C2:");
  lcd.print(cellVoltage[1], 2);

  lcd.setCursor(0, 1);

  lcd.print("C3:");
  lcd.print(cellVoltage[2], 2);

  lcd.print(" C4:");
  lcd.print(cellVoltage[3], 2);
}

// ====================================================
// LCD PAGE 2
// ====================================================

void displayPackPage()
{
  lcd.setCursor(0, 0);

  lcd.print("PACK:");
  lcd.print(packVoltage, 2);
  lcd.print("V");

  lcd.setCursor(0, 1);

  lcd.print("AVG:");
```

```
 lcd.print(averageVoltage, 2);
 lcd.print("V");
}

// ========================================================
// LCD PAGE 3
// ========================================================

void displayHealthPage()
{
 lcd.setCursor(0, 0);

 lcd.print("IMB:");
 lcd.print(imbalancePercent, 1);
 lcd.print("%");

 lcd.setCursor(0, 1);

 lcd.print("W:C");
 lcd.print(weakestCell + 1);

 lcd.print(" S:C");
 lcd.print(strongestCell + 1);
}

// ========================================================
// LCD PAGE 4
// ========================================================

void displaySafetyPage()
{
 lcd.setCursor(0, 0);

 lcd.print("SAFETY:");

 if (safetyState ==
    SAFETY_NORMAL)
 {
```

```
  lcd.print("NORMAL");
 }

 else if (safetyState ==
       SAFETY_WARNING)
 {
  lcd.print("WARNING");
 }

 else if (safetyState ==
       SAFETY_TRIPPED)
 {
  lcd.print("TRIPPED");
 }

 else
 {
  lcd.print("RECOVERY");
 }

 lcd.setCursor(0, 1);

 lcd.print("RELAY:");

 if (relayState)
 {
  lcd.print("ON");
 }
 else
 {
  lcd.print("OFF");
 }
}

// =======================================================
// LCD PAGE 5
// =======================================================
```

```
void displayDiagnosticPage()
{
 lcd.setCursor(0, 0);

 lcd.print("SYSTEM:");

 if (runtimeMode ==
    RUNTIME_NORMAL)
 {
  lcd.print("NORMAL");
 }

 else if (runtimeMode ==
       RUNTIME_DEGRADED)
 {
  lcd.print("DEGRADE");
 }

 else if (runtimeMode ==
       RUNTIME_FAILSAFE)
 {
  lcd.print("FAILSAFE");
 }

 else
 {
  lcd.print("SHUTDOWN");
 }

 lcd.setCursor(0, 1);

 lcd.print("NO ACTIVE FAULT");
}

// =======================================================
// LCD NORMAL
// =======================================================
```

```
void displayNormalLCD()
{
 lcd.clear();

 lcd.setCursor(0, 0);

 lcd.print("RUNTIME:NORMAL");

 lcd.setCursor(0, 1);

 lcd.print("SYSTEM OK");
}

// ====================================================
// LCD DEGRADED
// ====================================================

void displayDegradedLCD()
{
 lcd.clear();

 lcd.setCursor(0, 0);

 lcd.print("RUNTIME:DEGRADE");

 lcd.setCursor(0, 1);

 if (invalidSensorFault)
 {
  lcd.print("SENSOR FAULT");
 }

 else if (frozenADCFault)
 {
  lcd.print("ADC FROZEN");
 }

 else if (rapidChangeFault)
```

```
 {
   lcd.print("RAPID DV");
 }

 else
 {
   lcd.print("CHECK SYSTEM");
 }
}

// ====================================================
// LCD FAILSAFE
// ====================================================

void displayFailsafeLCD()
{
 lcd.clear();

 lcd.setCursor(0, 0);

 lcd.print("RUNTIME:FAILSAFE");

 lcd.setCursor(0, 1);

 lcd.print("RELAY:");

 if (relayState)
 {
   lcd.print("ON");
 }
 else
 {
   lcd.print("OFF");
 }
}

// ====================================================
// LCD SHUTDOWN
```

```
// ====================================================

void displayShutdownLCD()
{
  lcd.clear();

  lcd.setCursor(0, 0);

  lcd.print("SYSTEM SHUTDOWN");

  lcd.setCursor(0, 1);

  lcd.print("RELAY:OFF");
}

// ====================================================
// LCD FAULT
// ====================================================

void displayFaultLCD()
{
  lcd.clear();

  if (underVoltageFault)
  {
    lcd.setCursor(0, 0);

    lcd.print("FAULT:UNDER V");

    lcd.setCursor(0, 1);

    lcd.print("C");
    lcd.print(weakestCell + 1);

    lcd.print(":");
    lcd.print(cellVoltage[weakestCell], 2);
    lcd.print("V");
  }
```

```
else if (overVoltageFault)
{
  lcd.setCursor(0, 0);

  lcd.print("FAULT:OVER V");

  lcd.setCursor(0, 1);

  lcd.print("CHECK CELL");
}

else if (rapidChangeFault)
{
  lcd.setCursor(0, 0);

  lcd.print("FAULT:RAPID DV");

  lcd.setCursor(0, 1);

  lcd.print("CHECK BATTERY");
}

else if (sensorFault)
{
  lcd.setCursor(0, 0);

  lcd.print("FAULT:SENSOR");

  lcd.setCursor(0, 1);

  lcd.print("SAFE MODE");
}

else
{
  lcd.setCursor(0, 0);
```

```
    lcd.print("FAULT DETECTED");

    lcd.setCursor(0, 1);

    lcd.print("CHECK SYSTEM");
  }
}

// ====================================================
// SERIAL REPORT
// ====================================================

void displaySerialReport()
{
  Serial.println();

  Serial.println(
    "================================================"
  );

  Serial.println(
    "      EV BMS TASK 1 + 2 + 3 + 4"
  );

  Serial.println(
    "================================================"
  );

  // ================================================
  // BATTERY
  // ================================================

  Serial.println("BATTERY PARAMETERS");

  Serial.println(
    "------------------------------------------------"
  );
```

```
for (int i = 0; i < NUM_CELLS; i++)
{
  Serial.print("Cell ");
  Serial.print(i + 1);
  Serial.print(" Voltage     ");
  Serial.print(cellVoltage[i], 2);
  Serial.println(" V");
}

Serial.print("Pack Voltage          ");
Serial.print(packVoltage, 2);
Serial.println(" V");
Serial.print("Average Voltage       ");
Serial.print(averageVoltage, 2);
Serial.println(" V");
Serial.print("Imbalance             ");
Serial.print(imbalancePercent, 2);
Serial.println(" %");
Serial.print("Strongest Cell        C");
Serial.println(strongestCell + 1);
Serial.print("Weakest Cell          C");
Serial.println(weakestCell + 1);
```

```
Serial.print("Battery Health         ");

Serial.println(batteryHealth);

// ================================================
// SAFETY
// ================================================

Serial.println();

Serial.println("SAFETY STATUS");

Serial.println(
  "------------------------------------------------"
);

Serial.print("Under Voltage          ");

Serial.println(
  underVoltageFault ? "YES" : "NO"
);

Serial.print("Over Voltage           ");

Serial.println(
  overVoltageFault ? "YES" : "NO"
);

Serial.print("Rapid Voltage Change     ");

Serial.println(
  rapidChangeFault ? "YES" : "NO"
);

Serial.print("Safety State           ");

printSafetyState();
```

```
Serial.print("Relay                ");

Serial.println(
 relayState ? "ON" : "OFF"
);

// ================================================
// TASK 4
// ================================================

Serial.println();

Serial.println("FAULT-TOLERANT RUNTIME");

Serial.println(
 "------------------------------------------------"
);

Serial.print("Runtime Mode          ");

printRuntimeMode();

Serial.print("Invalid Sensor        ");

Serial.println(
 invalidSensorFault ? "YES" : "NO"
);

Serial.print("Frozen ADC            ");

Serial.println(
 frozenADCFault ? "YES" : "NO"
);

Serial.print("Relay Mismatch        ");

Serial.println(
```

```
  relayMismatchFault ? "YES" : "NO"
);

// ================================================
// FAULT LOG
// ================================================

Serial.println();

Serial.println("FAULT LOG");

Serial.println(
  "------------------------------------------------"
);

if (faultLogCount == 0)
{
  Serial.println("No faults logged");
}
else
{
  for (int i = 0;
      i < faultLogCount;
      i++)
  {
    Serial.print("#");

    Serial.print(i + 1);

    Serial.print("  ");

    Serial.print(
      faultLogs[i].timestamp
    );

    Serial.print(" ms  ");

    Serial.print(
```

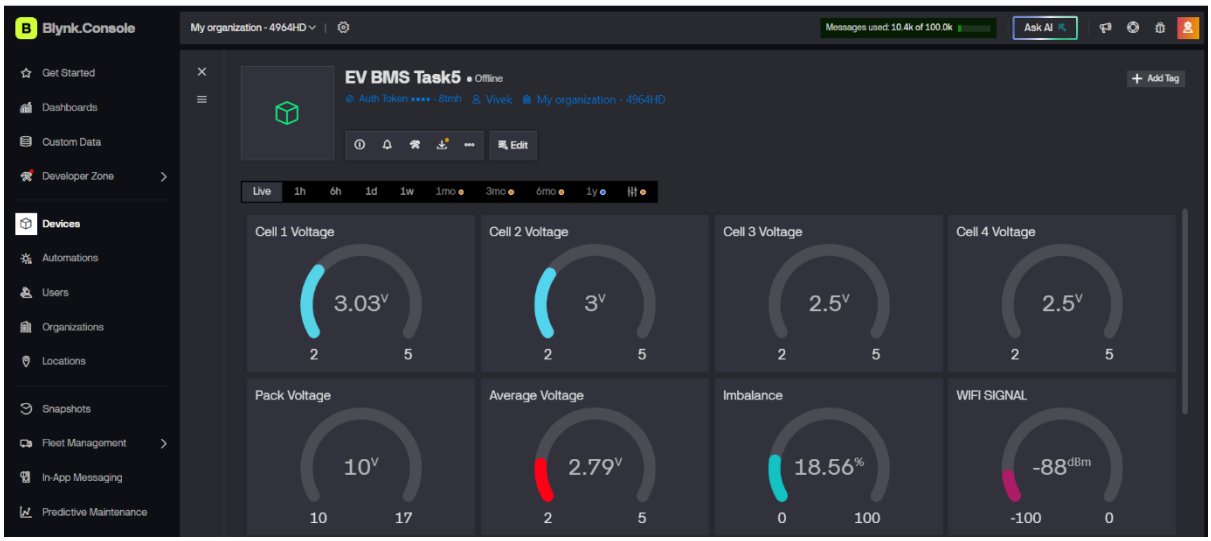
```
      faultLogs[i].faultName
    );

    Serial.print("  [");

    Serial.print(
      faultLogs[i].mode
    );

    Serial.println("]");
  }
 }

 Serial.println();
 Serial.println("CLOUD TELEMETRY");
 Serial.println("------------------------------------------------");
 Serial.print("WiFi Status           ");
 Serial.println(WiFi.status() == WL_CONNECTED ? "CONNECTED" : "OFFLINE");
 Serial.print("Blynk Status          ");
 Serial.println(Blynk.connected() ? "CONNECTED" : "OFFLINE");
 if (WiFi.status() == WL_CONNECTED)
 {
  Serial.print("WiFi RSSI             ");
  Serial.print(WiFi.RSSI());
  Serial.println(" dBm");
 }
 Serial.print("Queued Cloud Events     ");
 Serial.println(cloudEventCount);

 Serial.println(
  "================================================"
 );
}

// ======================================================
// PRINT SAFETY STATE
// ======================================================
```

```
void printSafetyState()
{
  if (safetyState ==
      SAFETY_NORMAL)
  {
    Serial.println("NORMAL");
  }

  else if (safetyState ==
         SAFETY_WARNING)
  {
    Serial.println("WARNING");
  }

  else if (safetyState ==
         SAFETY_TRIPPED)
  {
    Serial.println("TRIPPED");
  }

  else
  {
    Serial.println("RECOVERY");
  }
}

// ================================================
// PRINT RUNTIME MODE
// ================================================

void printRuntimeMode()
{
  if (runtimeMode ==
      RUNTIME_NORMAL)
  {
    Serial.println("NORMAL");
  }
```

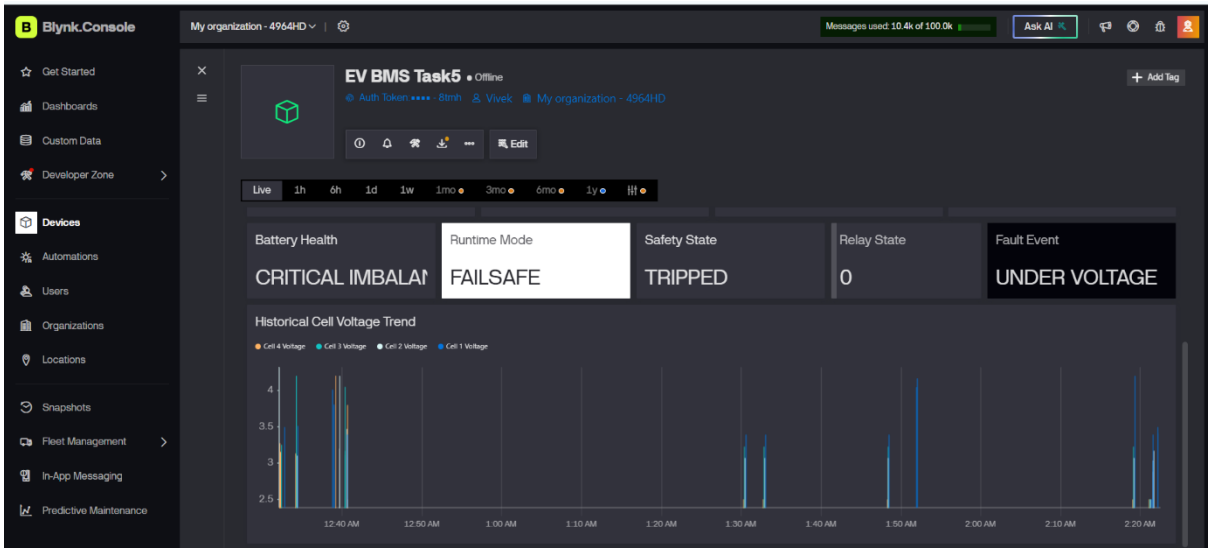
```
else if (runtimeMode ==
      RUNTIME_DEGRADED)
{
 Serial.println("DEGRADED");
}

else if (runtimeMode ==
      RUNTIME_FAILSAFE)
{
 Serial.println("FAILSAFE");
}

else
{
 Serial.println("SHUTDOWN");
}
}
```

13.4 Blynk Dashboard Overview photos :

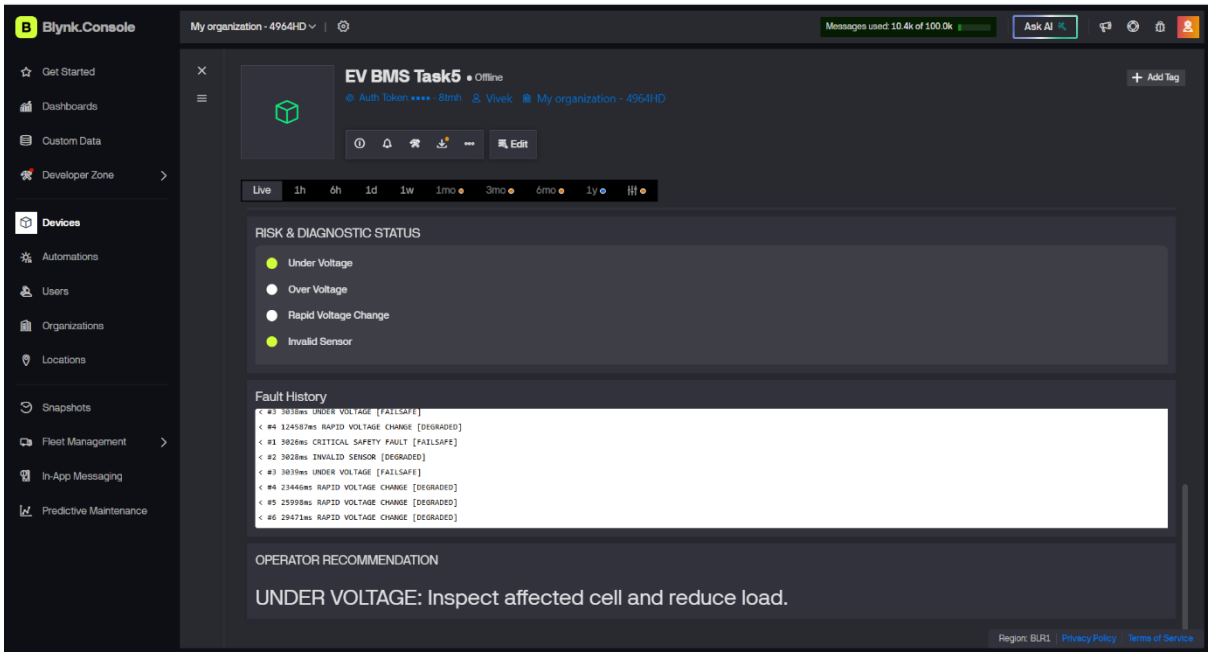
Case-1 Operational data & risk analysis:



Case-2 historical voltage trends:



Case-3 diagnostics, fault history, and intelligent operator recommendations:



13.5 Link of Dashboard :

[Blynk Dashboard Click Here]

13.6 Link of Final Wokwi Simulation :

[Task 6 Click Here]

13.7 Task 6 Demonstration Video Link:

Click Here

14. LIMITATIONS:

i. The battery pack and cell voltages are simulated using Wokwi rather than a physical lithium battery pack.
ii. The prototype mainly focuses on voltage-based battery intelligence and does not include complete physical current and temperature sensing.
iii. The system has not been validated on an actual EV battery pack or under real automotive operating conditions.
iv. Cloud communication and dashboard performance depend on Wi-Fi and Blynk connectivity.
v. The prototype requires further hardware-level and safety validation before practical EV deployment.

15. FUTURE SCOPE:

1. Implement the system on a real multi-cell lithium battery pack.
2. Add accurate current and temperature sensing.
3. Implement SOC and SOH estimation using advanced algorithms.
4. Add active/passive cell balancing.
5. Integrate CAN communication for automotive applications.
6. Perform Hardware-in-the-Loop (HIL) and real battery testing.
7. Develop an automotive-grade PCB and improve system reliability for real-world deployment.

16. CONCLUSION:

The Intelligent EV Battery Management System was successfully developed as an integrated ESP32-based platform combining battery intelligence, safety protection, HMI, fault-tolerant runtime, cloud telemetry, and dashboard monitoring. The system demonstrated real-time four-cell monitoring, fault detection, protection response, diagnostic logging, and remote visualization using Wokwi and Blynk. The project provides a strong foundation for further development toward a hardware-based and automotive-oriented Battery Management System.

16. APPENDIX

APPENDIX-A : Task-Wise Video Demonstration Links

Task	Video Demonstration
Task 1 – Battery Intelligence	Click Here
Task 2 – Safety Protection	Click Here
Task 3 – HMI & Diagnostics	Click Here
Task 4 – Fault-Tolerant Runtime	Click Here
Task 5 – Cloud Telemetry	Click Here
Task 6 – Executive Dashboard	Click Here

APPENDIX-B : Project Resource Links

Resource	Link
Final Wokwi Simulation	Click Here
Blynk Dashboard	Click Here
Final Project Demo Video	Click Here

17. References

1) BMS Project Training Course

2) Datasheet of ESP32 Microcontroller:

Click Here